

RADIANT Technical Reference

Scripting API, Configuration, Parameters, and Architecture

RADIANT Project

v0.2.0

Built 2026-09-21

Contents

1	System Overview	5
1.1	The signal chain	5
1.2	Radiometric regimes	6
1.3	The dual spatial path	7
1.4	Geometry-first design	8
1.5	Units, provenance, and the stable surface	9
1.6	Where to go next	9
2	Conventions	10
2.1	Canonical internal units	10
2.2	Spatial coordinate system	11
2.3	Spectral conventions	12
2.4	Time conventions	13
2.5	Radiometric modeling conventions	13
2.6	Physical constants	14
2.7	Parameter naming	14
2.8	Geometry input-mode letters	14
3	Scripting Guide	15
3.1	The import surface	15
3.2	The Sensor class	15
3.3	Working with a ChainResult	19
3.4	Parameter sweeps	22
3.5	Solving for a parameter value	23
3.6	Monte Carlo tolerance analysis	23
3.7	Sensitivity analysis	24
3.8	Progress reporting and cancellation	25
3.9	Batch matrices	25
3.10	Using the data library	26
3.11	Cloning for comparison	27
3.12	Configuration sets	27
3.13	Exporting results	28
3.14	See also	28
4	Command-Line Interface	29
4.1	radiant run — evaluate a config	29
4.2	radiant validate — check a config without running it	30
4.3	radiant explain — where a value came from	31
4.4	radiant gui — launch the desktop application	31
4.5	radiant sweep — 1-D sweep from the shell	31
4.6	radiant tolerance — Monte Carlo from the shell	32

4.7	radiant compare — two files side by side	32
4.8	radiant schema — list the parameter definitions	32
4.9	radiant template — bundled starting points	33
4.10	radiant convert — scalar unit conversion	33
5	Configuration Guide	35
5.1	YAML structure	35
5.2	Parameter dot-path convention	36
5.3	Defaults and required parameters	37
5.4	Overriding parameters	37
5.5	Consistency groups	38
5.6	Units	38
5.7	Resolution precedence	38
5.8	File paths inside a config	39
5.9	Using templates	39
5.10	Session metadata — the <code>_radiant</code> block	40
5.11	Optical element trains — the <code>optical_elements</code> section	40
5.12	Named FPA presets — the <code>fpa</code> key	41
5.13	Atmosphere configuration	41
5.14	Configuration sets — several configurations in one file	42
5.15	Common patterns	44
6	Parameter Reference	45
6.1	geometry	45
6.2	source	52
6.3	atmosphere	57
6.4	optics	63
6.5	platform	67
6.6	spectral_integration	67
6.7	detector	68
6.8	readout	70
6.9	calibration	73
6.10	performance	76
6.11	Consistency Groups	77
7	Error Taxonomy	78
7.1	The actionability contract	78
7.2	The tree	79
7.3	Core — <code>radiant.core</code>	80
7.4	Physics stages	81
7.5	I/O and data — <code>radiant.io</code> , <code>radiant.data</code>	84
7.6	API and front ends	85
7.7	Catching selectively	87
8	Data Libraries	88
8.1	SpectralLibrary	88
8.2	FPALibrary — 21 presets	89
8.3	Atmosphere library	91

8.4	Mission templates	92
9	External Data Interfaces	93
9.1	MODTRAN	93
9.2	Measured curves	94
9.3	Vendor detector data	95
9.4	Zemax wavefront reports	96
9.5	Spectral libraries	96
9.6	Spreadsheets	97
9.7	Other file-valued parameters	98
10	Part 3 — Internals: A Note to the Reader	99
11	RADIANT Signal Chain Architecture	100
11.1	What Is a Chain?	100
11.2	Module Interfaces	101
11.3	Data Flow: ChainState	103
11.4	Three Radiometric Regimes	105
11.5	Backward Propagation: Reference Frames	107
11.6	Non-Radiometric Information	110
11.7	Concrete Python Interfaces	110
11.8	Worked Example: Full Forward Pass	114
11.9	Architectural Invariants	115
12	RADIANT Parameter System Design	116
12.1	Critical Decision: Spectral Arrays	116
12.2	Parameter Class Design	117
12.3	Parameter Naming Convention	119
12.4	Defaults	124
12.5	Tolerances	125
12.6	Consistency Groups	126
12.7	Dependency Tracking	129
12.8	Provenance	129
12.9	Explainability	131
12.10	Schema Introspection (Gap 70, 2026-07-11)	132
12.11	Spectral Data Store	133
12.12	Configuration File Format	134
12.13	Design Implications from Personas	135
13	RADIANT Testing and Validation Framework	137
13.1	Test Hierarchy	137
13.2	Level 0: Physics Correctness Tests	137
13.3	Level 1: Module-Level Tests	141
13.4	Level 2: End-to-End Reference Scenarios	142
13.5	Golden Regression Tests (@pytest.mark.golden)	144
13.6	Numerical Tolerances	147
13.7	Provenance Tracking	147
13.8	Error Handling Philosophy	149

13.9	Test Infrastructure	152
14	Extending RADIANT	154
14.1	Adding a parameter	154
14.2	Adding a stage	156
14.3	Import rules	157
14.4	Adding a metric	158
14.5	Testing an extension	158
14.6	Plugins — deferred	159

System Overview

RADIANT is a first-principles electro-optical sensor performance model. Given a sensor specification, an observation geometry, an atmosphere, and a target/background description, it propagates spectral radiance from the source to the digitized output and reports SNR, NEDT, NIIRS, system MTF, and detection range — accumulating every noise term and every MTF contributor at the stage where it physically originates.

This chapter orients a script author or developer in that machine: what the stages are, how data moves between them, why there are two parallel spatial paths, and why geometry runs first. Part 3 of this volume binds the stage protocol, the parameter system, and the test framework verbatim; the repository’s master architecture document is where each constraint is stated in binding form.

1.1 The signal chain

A RADIANT evaluation is one pass through ten stages, in a fixed order. The order is a physics statement, not a convenience: each stage consumes quantities the previous stages published, and no stage reaches backwards.

#	Stage	Package	Consumes	Publishes
0	geometry	radiant.geometry	Altitudes [m], look angles [rad], platform kinematics	Slant range [m], ground range [m], GSD [m], incidence [rad], ground speed [m/s]
1	source	radiant.source	Target/background temperature [K], emissivity [–], reflectance [–]	Target and background spectral radiance [W/m ² /sr/μm], tentative regime
2	atmosphere	radiant.atmosphere	Path geometry, atmosphere model selection	$\tau_{\text{atm}}(\lambda)$ [–], path radiance $L_{\text{path}}(\lambda)$ [W/m ² /sr/μm]
3	optics	radiant.optics	Aperture [m], focal length [m], wavefront error [waves], element train	Complex pupil, EffectivePSF, MTF_optics, throughput [–], final regime
4	platform	radiant.platform	Jitter [μrad RMS], smear rate [rad/s], turbulence C_n^2 [m ^{–2/3}]	Degraded EffectivePSF, jitter/smear/turbulence MTF, EE_box [–]
5	spectral_integration	radiant.spectral_integration	All spectral arrays, QE(λ) [–], t_{int} [s]	Per-pixel signal [e–] — the spectral-to-scalar boundary
6	detector	radiant.detector	Dark rate [e–/s], pixel pitch [μm], operating temperature [K]	Detector noise terms [e– RMS], detector-aperture / diffusion / IPC MTF
7	readout	radiant.readout	Read noise [e– RMS], gain [e–/DN], ADC bits [–], TDI stages [–]	Digitized signal [DN], read/quantization noise [e– RMS], TDI MTF
8	calibration	radiant.calibration	Calibration scheme, NUC residual terms	Post-NUC residual noise [e– RMS], bias budget [–] (fractional $\Delta L/L$)

#	Stage	Package	Consumes	Publishes
9	performance	radiant. performance	Everything above	SNR [-], NEDT [K], NIIRS [-], system MTF [-], detection range [m]

The stage names in the first column are the literal keys of `ChainResult.stage_outputs` and the entries of `ChainResult.history`, verified against `radiant.api.session.RadiantSession.stage_names`.

Two ordering decisions in that table are load-bearing and are **not** free to change:

- **spectral_integration sits between platform and detector.** Everything above it carries spectral arrays of length N_λ ; everything below it carries per-pixel scalars in e- and DN. The collapse happens exactly once.
- **calibration runs after readout, not before.** Post-NUC residual fixed-pattern terms are appended after TDI and coadd scaling, which is precisely what makes them structurally exempt from $\sqrt{N}$ averaging. Moving the stage earlier would silently average away the error the stage exists to model.

1.1.1 Stages are pure functions

Every stage implements one method:

```
def run(self, state: ChainState, params: ParameterSet) -> ChainState: ...
```

A stage does not mutate its inputs, does not read or write files, does not call another stage, and holds no state between runs. All inter-stage communication flows through `ChainState`, a frozen dataclass that stages extend with `with_frame(...)`, `with_noise(...)`, `with_mtf(...)`, and `with_stage_output(...)`. Fields added by an earlier stage are never removed or overwritten by a later one, so the final state is a complete audit record of the run.

File I/O happens *before* the chain: the atmosphere model, the optical-element list, a tabulated QE curve, a Zernike wavefront, and a C_n^2 profile are all built by the API layer and injected as pre-chain stage outputs. The full mechanism is in Part 3, “Signal-Chain Internals”.

1.2 Radiometric regimes

RADIANT classifies every scene into one of three radiometric regimes, because the signal equation is genuinely different in each:

Regime	Condition	Signal model
extended	Target fills the pixel footprint	Radiance-based; no <code>EE_box</code> term
sub_pixel	Target fills part of the pixel	Target term weighted by fill fraction and <code>EE_box</code> ; background pedestal fills the remainder and is not <code>EE_box</code> -weighted
point_source	Angular extent much smaller than the IFOV	Intensity-based [W/sr]; <code>EE_box</code> applied once

Classification is **tentative** in SourceStage (`stage_outputs["source"]["regime_tentative"]`) and **final** in OpticsStage (`stage_outputs["optics"]["regime"]`), because the decision needs the diffraction PSF diameter, which does not exist until the pupil has been built. Every downstream stage reads the optics value. No stage re-classifies.

`source.regime_override` forces a regime when the analyst knows better than the detection rule.

1.3 The dual spatial path

RADIANT maintains **two** parallel spatial descriptions, both rooted in the *same* complex pupil function. This is the single most frequently misunderstood part of the architecture, and it exists because spatial-domain metrics and frequency-domain budgets have genuinely different natural homes.

1.3.1 Path A — the PSF path (spatial domain)

Every spatial degradation enters as a convolution kernel on one EffectivePSF object: the diffraction-and-aberration PSF from the pupil, then the detector aperture kernel, then jitter, smear, turbulence, diffusion, and IPC. Ensquared energy (EE_box), RER, FWHM, Strehl ratio, LSF, and ERF are computed **only** from that one object.

Strehl ratio is the degraded-PSF peak divided by the diffraction-limited reference_psf peak, with the same detector kernels applied to both so detector effects cancel. The analytic Maréchal approximation survives only as a separate `strehl_marechal` diagnostic — it is not the reported Strehl.

EE_box is computed in PlatformStage from the fully degraded PSF (jitter, smear, and turbulence already folded in) and applied once, in SpectralIntegrationStage.

1.3.2 Path B — the MTF product path (frequency domain)

Optical MTF comes from the **autocorrelation of the complex pupil function**. By the Wiener–Khinchin theorem this equals $|\mathcal{F}\{\text{PSF}\}|$, but computing it directly from the pupil is what makes it correct: aberrations interact with diffraction *inside* the pupil and cannot be factored. Writing

$$\text{MTF}_{\text{optics}}(\nu) = \text{MTF}_{\text{diffraction}}(\nu) \times \text{MTF}_{\text{aberration}}(\nu)$$

is wrong, and RADIANT never does it. There is one `MTF_optics` term.

Each downstream contributor — detector aperture, jitter, smear, diffusion, IPC, turbulence — supplies an analytic or kernel-derived MTF, and the system MTF is their product:

$$\text{MTF}_{\text{sys}}(\nu) = \prod_i \text{MTF}_i(\nu)$$

MTF budgets, MTF-at-Nyquist, folded MTF, and GIQE/NIIRS all consume this path.

One term is deliberately MTF-only: **TDI mis-registration**. It is a readout-timing effect with no spatial kernel, so it enters the product and is excluded from the cross-path comparison below.

1.3.3 The consistency invariant

Both paths start from the same pupil, so the FFT of the convolved EffectivePSF must agree with the MTF product. `radiant.performance.consistency_check` verifies this on every chain execution in which the spatial path is computed:

Property	Value
Default absolute tolerance	2×10^{-2} (dimensionless MTF units)
Worst measured full-chain residual	$\sim 1 \times 10^{-2}$ (dimensionless) — about 2× margin
Comparison band	At and below detector Nyquist only
Excluded terms	TDI mis-registration (MTF-only by construction)
Failure behavior	Logged warning

A failure means a degradation was added to one path and not the other. The check is skipped only when the analyst deselects the entire Spatial-MTF metric group *and* no enabled metric needs a spatial input — there is then no spatial computation to check.

1.4 Geometry-first design

Geometry is Stage 0, not a helper called from inside the radiometry. Every downstream stage that needs a range, an angle, a GSD, or a ground speed reads it from `stage_outputs["geometry"]`; nothing recomputes it locally.

The analyst does not have to supply the geometry in one canonical form. Geometry accepts **input modes**, grouped into four independent families. Exactly one mode per family is active, and the stage solves the spherical viewing triangle from whichever door was used (`radiant.api.geometry_modes.MODE_FAMILIES`):

Family	Mode	Driving parameter(s)
viewing	V0	<code>geometry.target_range_m</code> [m]
viewing	V1	<code>geometry.path_zenith_rad</code> [rad]
viewing	V2	<code>geometry.sensor_off_boresight_rad</code> [rad]
viewing	V3	<code>geometry.ground_range_m</code> [m]
viewing	V4	<code>geometry.elevation_angle_rad</code> [rad]
solar	S1	<code>geometry.solar_zenith_rad</code> [rad]
solar	S2	<code>geometry.solar_elevation_rad</code> [rad]
solar	S3	<code>geometry.site_latitude_rad</code> [rad], <code>geometry.day_of_year</code> [–], <code>geometry.local_solar_time_h</code> [h], <code>geometry.ltan_h</code> [h]
kinematics	direct	<code>geometry.ground_speed_m_s</code> [m/s]
kinematics	circular	<code>geometry.circular_orbit</code> [–] (derives speed from altitude)
LOS rate	K0	none — static line of sight
LOS rate	K1	<code>geometry.los_angular_rate_rad_s</code> [rad/s]
LOS rate	K2	<code>geometry.target_speed_m_s</code> [m/s], <code>geometry.target_heading_rad</code> [rad], <code>geometry.target_climb_rad</code> [rad]

Because the families are independent, a ground-to-air scenario (up-looking V4 elevation, K2 target kinematics) and a nadir mapping pass (V1 path zenith, circular orbit) use the same solver with no special cases.

1.5 Units, provenance, and the stable surface

Units convert exactly once, at a boundary: `params.set()` for user input, or a file reader for external data (the MODTRAN reader’s $\times 10^4$ for $\text{W}/\text{cm}^2 \rightarrow \text{W}/\text{m}^2$ is the canonical example). Inside a physics module, a `*math.pi / 180` or a `* 1e4` is a bug unless it is computing physics. Canonical internal units: wavelength in μm , angles in rad, time in s, length in m, radiance in $\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$, noise in e- RMS. The next chapter gives the full table.

Provenance is mandatory and cannot be disabled. Every `ChainResult` carries `to_provenance_record()`: run ID, RADIANT version, git commit, Python version, dependency versions, the fully resolved parameter set with per-parameter provenance, SHA-256 hashes of every consumed input file, and the ordered list of stages that ran. Given that record, the run reproduces.

The stable surface is small. The top-level `__all__` of `radiant` is `{Sensor, RadiantError, __version__}`; `Sensor` and `RadiantError` carry stability guarantees. `ChainResult` is importable from `radiant.io.results` (and re-exported from `radiant.api`) but is not top-level. `BatchRunner` is a semi-public `radiant.api.batch` class. There are no `SensorConfig` or `ScenarioConfig` builder classes — they were dropped because `Sensor.from_yaml()` and `Sensor.from_dict()` already accept the same data. Anything under `radiant.core.*` or an individual stage package is internal.

1.6 Where to go next

You want to	Read
Drive RADIANT from Python	“Scripting Guide”, this volume
Drive it from a terminal	“Command-Line Interface”, this volume
Write or read a config file	“Configuration Guide”, this volume
Look up a parameter	“Parameter Reference”, this volume (generated from the schema)
Catch and interpret an error	“Error Taxonomy”, this volume
Understand the physics	<i>RADIANT Theory Manual</i> (Volume I)
Implement a stage or parameter	Part 3 of this volume, and “Extending RADIANT”

Conventions

Every RADIANT module obeys one set of conventions. No module defines its own local ones, and a violation is a bug rather than a style preference. This chapter is the working summary; the repository’s conventions document carries the binding statement, with the justification for each choice and the comparison against other tools.

2.1 Canonical internal units

These are the units every quantity is stored and computed in, everywhere inside the framework.

Quantity	Canonical unit	Notes
Wavelength	μm	Ascending order, always
Angle	rad	No exceptions, in any module
Time	s	<code>integration_time_s</code> , <code>frame_period_s</code>
Length	m	Altitudes, ranges, aperture, focal length
Spectral radiance $L(\lambda)$	$\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$	Not W/cm^2 — see the MODTRAN note below
In-band radiance L	$\text{W}/\text{m}^2/\text{sr}$	$\int L(\lambda) d\lambda$ over the bandpass
Spectral irradiance $E(\lambda)$	$\text{W}/\text{m}^2/\mu\text{m}$	
Spectral intensity $I(\lambda)$	$\text{W}/\text{sr}/\mu\text{m}$	Point sources
Spectral photon radiance $L_q(\lambda)$	$\text{photons}/\text{s}/\text{m}^2/\text{sr}/\mu\text{m}$	Derived: $L_q = L\lambda/(hc)$
Signal	e-	After QE and integration time
Noise	e- RMS	Every noise term, at its origin frame
Temperature	K	Entry in K, °C, or °F; converted once at <code>set()</code>
SNR	dimensionless	S/σ_{total}
NEDT	K	
NIIRS	dimensionless	
MTF	dimensionless	$0 \leq \text{MTF}(\nu) \leq 1$
Spatial frequency	cycles/mm (focal plane) or cycles/mrad (angular)	Always labeled; never “cycles/pixel” as a primary unit

2.1.1 Conversion happens exactly once

Boundary	Conversion	Owner
User input	$\text{deg} \rightarrow \text{rad}$, $\mu\text{rad} \rightarrow \text{rad}$, $\text{ms} \rightarrow \text{s}$, $^{\circ}\text{C}/^{\circ}\text{F} \rightarrow \text{K}$	<code>ParameterSet.set(..., unit=...)</code>

Boundary	Conversion	Owner
MODTRAN tape7 import	$\text{cm}^{-1} \rightarrow \mu\text{m}$, $\text{W}/\text{cm}^2 \rightarrow \text{W}/\text{m}^2 (\times 10^4)$, descending $\rightarrow$ ascending λ	<code>radiant.atmosphere.modtran</code> reader
Vendor QE CSV	$\text{nm} \rightarrow \mu\text{m}$, percent $\rightarrow$ fraction	<code>radiant.io.qe_csv</code>
Vendor dark-current CSV	$\text{A}/\text{cm}^2 \rightarrow \text{e-}/\text{s}/\text{pixel}$	<code>radiant.io.dark_current_csv</code>
Detector QE stage	$\text{W}/\text{m}^2/\text{sr}/\mu\text{m} \rightarrow \text{photons}/\text{s}/\text{m}^2/\text{sr}/\mu\text{m}$	<code>SpectralIntegrationStage</code>
Output formatting	$\text{rad} \rightarrow \text{deg}$ or μrad , $\text{s} \rightarrow \text{ms}$	Output formatter / GUI

If a conversion appears in two modules, one of them is wrong. A factor of `1e4`, `math.pi/180`, or `1e-6` inside a physics module is a red flag: verify it is physics, not unit plumbing.

The $\times 10^4$ on MODTRAN radiance is the single most consequential conversion in the tool. MODTRAN reports $\text{W}/\text{cm}^2/\text{sr}/\mu\text{m}$; RADIANT stores $\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$. It is implemented in the MODTRAN reader and nowhere else.

2.1.2 Angles: two user-facing units, one internal unit

Internally: radians only. At the user boundary, two units are supported and the threshold between them is roughly $0.1^\circ = 1745 \mu\text{rad}$:

- **Large angles in degrees** — FOV, look angle, solar zenith, elevation, azimuth.
- **Small angles in μrad** — jitter, pointing knowledge, IFOV, angular blur.

mrad is deliberately not supported: it sits in an awkward middle zone and a third angular unit triples the bug surface.

A parameter's name suffix names its *canonical stored* unit, which is not always the user-facing entry unit. Two patterns coexist:

```
# Small angles: stored and named in the user-facing unit.
sensor.set("platform.jitter_rms_urad", 5.0)           #  $\mu\text{rad} \rightarrow \text{rad}$  at the boundary

# Large geometry angles: stored and named in radians, entered in degrees.
sensor.set("geometry.solar_zenith_rad", 30.0, unit="deg") #  $\rightarrow 0.5236 \text{ rad}$  stored
```

The invariant that actually holds across the whole schema is narrower than “every angle is named in its entry unit”: **every angular parameter is unit-suffixed and is converted to radians exactly once, at the `set()` boundary.**

2.2 Spatial coordinate system

Property	Convention
Handedness	Right-handed
+Z	Toward target — along boresight / optical axis
+X	Cross-track (cross-scan; perpendicular to flight direction)
+Y	Along-track (along-scan; flight direction projected to the image plane)

Property	Convention
Euler sequence	3-2-1 (ZYX): yaw → pitch → roll
Pixel indexing	[row, col] = [y, x] = [along-track, cross-track], 0-indexed

+Z toward target matches the optics convention (Zemax, Code V), where the optical axis is +Z and the focal plane lies in the XY plane. For a nadir-pointing spacecraft, +Z is nadir; for a ground observer, +Z is toward the target in the sky. The same assignment holds for every viewing geometry, with no redefinition.

For a pushbroom sensor the linear array lies cross-track (+X) and the image builds along-track (+Y), which puts the TDI direction along +Y and completes a right-handed frame. A staring sensor maps its two focal-plane axes the same way.

[row, col] is the opposite of Cartesian (x, y) ordering, deliberately: it matches NumPy row-major storage, OpenCV, and standard image processing. Every function that takes pixel coordinates takes (row, col), never (x, y).

Conversions at boundaries: FITS stores [col, row] (NAXIS1 = column = cross-track), so FITS I/O transposes. MATLAB is 1-indexed, so a MATLAB bridge adds or subtracts 1. STK and other orbital frames transform to the sensor body frame with the ZYX angles in the transform module — never inside a physics module.

2.3 Spectral conventions

Property	Convention
Primary spectral variable	Wavelength
Primary unit	μm
Array ordering	Ascending: $\lambda_0 < \lambda_1 < \dots < \lambda_{N-1}$
Secondary variable	Wavenumber ν [cm^{-1}] — derived, never stored as primary
Conversion	$\nu [\text{cm}^{-1}] = 10000/\lambda [\mu\text{m}]$

Wavelength in μm keeps every band in single digits: UV 0.2–0.4 μm , VIS 0.4–0.7 μm , SWIR 0.7–2.5 μm , MWIR 3–5 μm , LWIR 8–14 μm . Nanometers make MWIR/LWIR unwieldy (3000–14000 nm); wavenumber inverts the intuition.

MODTRAN’s native tape7 output is ascending wavenumber, which is *descending* wavelength. On import RADIANT reverses the array and applies the Jacobian of the transformation:

$$L(\lambda) = L(\nu) \nu^2$$

The ν^2 is two factors whose powers of ten cancel: the spectral-axis Jacobian $|d\nu/d\lambda| = 10^4/\lambda^2 = \nu^2/10^4$, times the 10^4 that carries the $\text{cm}^{-2} \rightarrow \text{m}^{-2}$ area conversion. Written either factor alone, the relation is off by 10^4 .

Every spectral array must be monotonically increasing in λ . Non-monotonic input is an error, not something silently reordered.

All stages evaluate on one common wavelength grid, spanning `spectral_integration.filter_min_um` to `filter_max_um` with `Sensor.wavelength_points` samples (default 500 points). There is no resampling inside the chain.

2.4 Time conventions

Property	Convention
Integration time	s — <code>spectral_integration.integration_time_s</code>
Frame period	s — <code>readout.frame_period_s</code> (0.0 s = unset)
Frame rate	Hz — derived, <code>stage_outputs["readout"]["frame_rate_hz"]</code>
Duty cycle	dimensionless, $0 < \text{duty} \leq 1$ — derived, <code>stage_outputs["readout"]["duty_cycle"]</code>

Integration time and frame period are **independent** parameters: the first is how long charge accumulates, the second is the time between frame starts, and duty cycle = $t_{\text{int}}/T_{\text{frame}}$. Neither is derived from the other without explicit user intent. When `readout.frame_period_s` is left at 0.0 s, the frame period defaults to t_{int} (frame rate = $1/t_{\text{int}}$, duty cycle 1.0) and the chain records `frame_period_defaulted` in the readout stage outputs. A duty cycle above 1.0 — integration longer than the frame period — is rejected with an actionable error.

Display may use SI prefixes (10.0 ms, 250 μ s); internal storage is always seconds.

2.5 Radiometric modeling conventions

Energy units upstream, photons at the detector. The chain computes in W through source, atmosphere, and optics. The conversion to photons happens once, where QE is applied:

$$S[e^-] = t_{\text{int}} \int L(\lambda) A \Omega \tau_{\text{opt}}(\lambda) \tau_{\text{atm}}(\lambda) \text{QE}(\lambda) \frac{\lambda}{hc} d\lambda$$

Carrying photon quantities upstream would force a $\lambda/(hc)$ spectral weighting through stages that should be spectrally agnostic, such as geometric throughput.

Noise lives in electrons at its origin frame. Every noise term carries a value in e- RMS and the reference frame where it was generated. Conversion to another frame — DN, aperture-referred irradiance — happens at query time from the stored forward factors, never at generation time.

Emissivity of an optical element is always derived, never an input. Kirchhoff's law gives $\varepsilon = 1 - R$ for a mirror and $\varepsilon = 1 - T - R$ for a transmissive element. Supplying both reflectance and emissivity for an optical surface over-specifies the energy balance and is rejected with a `KirchhoffViolationError`. This does **not** apply to scene targets and backgrounds, where emissivity is a legitimate independent material property.

2.6 Physical constants

All constants are CODATA 2018 exact values, defined once in `radiant.core.constants` and imported everywhere else.

Constant	Symbol	Value	Unit
Speed of light	c	2.99792458×10^8	m/s
Planck constant	h	$6.62607015 \times 10^{-34}$	J·s
Boltzmann constant	k_B	1.380649×10^{-23}	J/K
Stefan–Boltzmann constant	σ_{SB}	$5.670374419 \times 10^{-8}$	W/m ² /K ⁴
Elementary charge	q	$1.602176634 \times 10^{-19}$	C

No module hardcodes a constant, and no “close enough” approximation is acceptable — $c \approx 3 \times 10^8$ m/s is forbidden.

2.7 Parameter naming

Parameters are dot-paths of the form `namespace.parameter_name`, lowercase with underscores, mapping directly onto YAML nesting:

Dot-path	YAML location
<code>optics.aperture_diameter_m</code>	<code>optics: → aperture_diameter_m:</code>
<code>source.target.temperature</code>	<code>source: → target: → temperature:</code>
<code>spectral_integration.filter_min_um</code>	<code>spectral_integration: → filter_min_um:</code>

The ten namespaces are the ten stage names. A unit suffix on the name (`_m`, `_um`, `_rad`, `_s`, `_e_rms`) names the canonical stored unit; a parameter with no natural unit carries none. Every parameter has exactly one `ParameterDef` in its owning stage’s `_schema.py`, and nothing tuneable is hardcoded in a physics module.

2.8 Geometry input-mode letters

The Parameter Reference tags each geometry entry with a mode letter — $V1 \dots V4$, $V0$ for the viewing family, $S1 \dots S3$ for the solar family, $V6$ for the circular-orbit kinematics door, $K0 \dots K2$ for the line-of-sight rate — and does not expand them. They are the doors of the ADR-0006 input-mode manifest, one active per family, detected from which parameter you set: $V1$ path zenith at the lower endpoint, $V2$ off-boresight angle, $V3$ ground range, $V4$ signed elevation, $V0$ direct slant range; $S1$ solar zenith, $S2$ solar elevation, $S3$ site and time; $V6$ a circular orbit in place of a direct ground speed; $K0$ platform motion only, $K1$ a direct LOS rate, $K2$ a target velocity. The User’s Guide’s scene chapter walks each card; `RADIANT_Geometry.md` is the normative source.

Scripting Guide

Using the Python API for programmatic sensor modeling, sweeps, and analysis.

3.1 The import surface

Two names are guaranteed stable and live at the top level:

```
from radiant import Sensor, RadiantError
```

Everything else is reached through `radiant.api` (`ChainResult`, `SweepResult`, `MonteCarloResult`, `SensitivityResult`, `SolveResult`, `ConfigurationSet`, `ErrorBudget`, `compare_configs`, the plotting helpers) or, for the batch matrix, `radiant.api.batch`. `from radiant.api import Sensor` resolves to the same class as the top-level import.

There are no `SensorConfig` or `ScenarioConfig` builder classes. `Sensor.from_yaml()` and `Sensor.from_dict()` already accept everything such a wrapper would have carried.

3.2 The Sensor class

`Sensor` is the primary Python entry point. It wraps configuration, parameter resolution, and chain execution into a single object.

3.2.1 Creating a sensor

From a YAML file:

```
from radiant.api import Sensor
```

```
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
```

From a Python dict:

```
from radiant.api import Sensor
```

```
config = {
    "source": {"target": {"temperature": 300.0, "emissivity": 0.95}},
    "geometry": {"sensor_altitude_m": 8000.0},
    "optics": {"aperture_diameter_m": 0.30, "focal_length_m": 1.20,
               "transmission_scalar": 0.70},
    "detector": {"pixel_pitch_x_um": 18.0, "pixel_pitch_y_um": 18.0,
```

```

        "qe_value": 0.70, "dark_rate_e_per_s": 100.0},
    "spectral_integration": {"filter_min_um": 3.5, "filter_max_um": 5.0,
        "integration_time_s": 0.005},
    "readout": {"read_noise_e_rms": 5.0, "gain_e_per_dn": 1.0, "adc_bits": 16},
}
sensor = Sensor.from_dict(config)

```

Both constructors take `wavelength_points` (default 500 points), the number of samples on the common spectral grid. The grid always spans the sensor's own resolved `spectral_integration.filter_min_um ... filter_max_um`; only the density is set here.

```
from radiant.api import Sensor
```

```

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml", wavelength_points=1000)
sensor.wavelength_points          # 1000 points
coarse = sensor.with_wavelength_points(200)  # a clone; `sensor` is untouched

```

`with_wavelength_points(n)` rejects `n < 2` with an `ApiValidationError` — the same check `Sensor.load()` applies to the `_radiant.wavelength_points` metadata field.

`Sensor.load(path)` is the counterpart of `Sensor.save()`: it restores parameters, tolerance distributions, and the grid size from the file's `_radiant` block. A plain config loads exactly as through `from_yaml`.

3.2.2 Setting and getting parameters

```
from radiant.api import Sensor
```

```

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("optics.aperture_diameter_m", 0.50)
diameter = sensor.get("optics.aperture_diameter_m")

```

`set()` returns the sensor for chaining:

```

from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("optics.aperture_diameter_m", 0.50).set("detector.qe_value", 0.80)
result = sensor.evaluate()

```

3.2.3 Resetting a parameter to its default

```
from radiant.api import Sensor
```

```

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("detector.dark_rate_e_per_s", 500.0)
sensor.reset("detector.dark_rate_e_per_s")  # back to config-file value

```

`reset_all(scope="user_set")` (the default) drops every input whose provenance is `USER_SET`; inputs still carrying config-file provenance survive. Note that an edit *replaces* an input's provenance, so a config value that was later edited reverts to its schema default, not to the file value — there is no layered history. To get the file back exactly, reload it. `reset_all(scope="all")` removes every explicit input, leaving pure schema defaults.

3.2.4 Entering a value in your own unit

`set()` takes a `unit=` keyword and converts at that boundary — the single conversion point for user input. The value you pass is in *your* unit; what gets stored is canonical.

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("optics.aperture_diameter_m", 30.0, unit="cm")      # 30 cm -> 0.30 m
sensor.set("geometry.solar_zenith_rad", 30.0, unit="deg")      # 30 deg -> 0.5236 rad
sensor.set("source.target.temperature", 27.0, unit="degC")     # 27 degC -> 300.15 K
```

`set()` also takes `source=`, the human-readable provenance label recorded with the input and shown by `resolved()` and `explain()`. It defaults to "Sensor.set"; a caller setting values on behalf of a named context passes its own label. The provenance *class* stays `USER_SET` either way.

3.2.5 Inspecting parameters and provenance

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")

sensor.get("optics.f_number")          # resolved canonical value (derived here)
sensor.get_input("optics.aperture_diameter_m") # the value as entered, input units
sensor.inputs()                        # read-only mapping of every explicit input
sensor.resolved("optics.f_number")     # ResolvedValue: value, units, provenance
sensor.provenance("optics.f_number")   # Provenance enum member (DERIVED here)

sensor.parameter_defs()                # dot-path -> ParameterDef, the whole schema
sensor.parameter_def("detector.qe_value") # one ParameterDef

print(sensor.summary())                # every resolved parameter, grouped, with units
print(sensor.explain("optics.f_number")) # one parameter's value and derivation chain
print(sensor.explain())                 # full chain walkthrough of the latest run
```

Provenance members are `USER_SET`, `CONFIG_FILE`, `DEFAULT`, `DERIVED`, `SAMPLED`, and `PRESET`.

3.2.6 Saving a sensor

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("optics.aperture_diameter_m", 0.50)

sensor.save("my_design.yaml")          # explicit inputs + _radiant metadata block
yaml_text = sensor.to_yaml()           # the same document as a string, no temp file
full = sensor.to_yaml(scope="resolved") # every resolved parameter, for documentation
```

`save()` writes only the explicitly-set inputs, in input units, plus a `_radiant` block carrying `wavelength_` points and any tolerance distributions. Reloading reproduces the sensor exactly: defaults re-apply, consistency groups re-derive, and the provenance distinction between explicit and defaulted values survives. The file is an ordinary RADIANT config — `from_yaml` and the CLI read it too.

`to_yaml(relative_to=...)` rewrites absolute file-path parameters relative to the directory the emitted YAML will live in, so an element-bearing config stays portable.

3.2.7 Applying an FPA preset

```
from radiant.api import Sensor
```

```
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
report = sensor.apply_fpa("teledyne-h2rg-2p5")
```

```
report.part           # 'teledyne-h2rg-2p5'
report.applied        # dot-paths the preset set, with Provenance.PRESET
report.skipped_existing # dot-paths the config had set explicitly – those won
report.qe_material    # the QE library curve the preset selected, or None
sensor.fpa_applications() # the live report(s), for provenance inspection
sensor.remove_fpa()    # returns the dot-paths cleared
```

Presets seed; explicit values win, in any order. Values applied by a preset persist through `save()` as ordinary explicit inputs — the `fpa:` key itself is not re-serialized. The bundled part list is in the **Data Libraries** chapter.

3.2.8 Attaching an optical element train

```
from radiant.api import Sensor
```

```
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set_optical_elements([
    {"name": "M1", "transfer_mode": "REFLECTIVE", "reflectance": 0.97,
     "temperature_K": 293.0},
    {"name": "cold_filter", "transfer_mode": "REFRACTIVE", "kind": "FILTER",
     "transmittance": 0.90, "temperature_K": 240.0},
])
sensor.optical_elements() # the stored document, or None
```

The document is validated immediately through the io parser — including the Kirchhoff check, so an entry carrying both reflectance and emissivity is rejected — normalized, and parsed onto the current wavelength grid at every evaluation. Unlike raw stage-output injections it *is* written by `save()` and restored by `load()`. Pass `None` to remove it.

3.2.9 Injecting non-scalar inputs

Stages never read files, so file-derived objects are built before the chain and injected as pre-chain stage outputs:

```
from radiant.api import Sensor
```

```
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set_stage_output("optics_config", "psf_weighting_spectrum", spectrum)
```

Or one-off, for a single run:

```
result = sensor.evaluate(extra_stage_outputs={"optics_config": {"element_list": elements}})
```

Injects set via `set_stage_output` apply to every evaluation, including sweeps and Monte Carlo runs. They are **not** serialized by `save()` — only the declarative `optical_elements` document is.

3.3 Working with a ChainResult

`sensor.evaluate()` returns a `ChainResult` with several views into the completed signal chain.

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
```

3.3.1 Metrics

`result.metrics` is a dict of computed performance numbers:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
snr = result.metrics["snr"]
contrast_snr = result.metrics["contrast_snr"]
mtf_nyq = result.metrics["mtf_at_nyquist"]
rer = result.metrics["rer"]
ee_1x1 = result.metrics["ee_1x1"]
```

`result.metrics` is a bare name → value mapping and carries no units. For anything a human will read, use `metric_records()` instead — every value arrives with its unit and description from the metric registry:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()

for rec in result.metric_records():
    print(f"{rec.name} = {rec.value:.4g} {rec.unit} ({rec.kind}) - {rec.description}")
```

Each `MetricRecord` carries name, value, unit (always a non-empty string), description, and kind — "float", "flag" (a 0/1 boolean), or "code" (an enumeration encoded as a float, with the levels named in the description). Records come back in sorted-name order.

Two export views build on the same records:

```
result.to_records()           # list of {name, value, unit, description} dicts
result.to_csv("metrics.csv")  # name,value,unit,description - UTF-8
```

There are also convenience accessors for the three headline metrics; each raises `KeyError` if that metric was not computed for the run:

```
result.snr()      # dimensionless
result.nedt()     # K      - reads metrics['nedt_K']
result.niirs()    # dimensionless
```

3.3.2 Stage outputs

Each stage stores intermediate results accessible via `result.stage_outputs`:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
regime = result.stage_outputs["optics"]["regime"]
tau_atm = result.stage_outputs["atmosphere"]["tau_atm"]
```

3.3.3 Radiometric frames

Frames are snapshots of the signal at key propagation points:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
frame_names = list(result.frames.keys())
# For examples/mwir_leo_minimal.yaml: ['at_aperture_target', 'at_aperture',
# 'at_source_target', 'at_source_target_reflected', 'post_optics', 'photoelectrons']
```

Which frames exist depends on the scene: a reflective scene registers `at_source_target_reflected`, a point-source run registers the intensity frames. Use `result.signal_at(...)` (below) when you want a value at a *named reference frame* regardless of which snapshots this particular run happened to register.

3.3.4 Noise terms

Individual noise contributions (in electrons):

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
for nt in result.noise_terms:
    if nt.value_e > 0.1:
        pass # nt.name, nt.value_e are the key attributes
```

`explain_noise(name)` returns a structured account of one term rather than prose fields you have to assemble yourself:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()

exp = result.explain_noise("signal_shot")
exp.value_e           # e- RMS at its origin frame
exp.origin_frame      # where the noise was generated
exp.physical_basis    # the mechanism tag
exp.contributes_to    # which budgets it sums into
exp.share_of_variance # value^2 / sum(value^2) – the pie-chart fraction
print(exp.description) # the same fields pre-formatted, with units
```

An unknown term name raises `KeyError` listing the available terms — never a `None`.

3.3.5 Reading a value at another reference frame

Signal and noise can be read at any point in the chain, propagated through the stored transfer factors:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()

q = result.signal_at("at_aperture")    # ChainQuantity: .value, .frame, .unit, .name
n = result.noise_at("dn")              # total noise expressed in DN
n_shot = result.noise_at("dn", "signal_shot") # one term at that frame
```

The six frames are `at_target`, `at_aperture`, `post_optics`, `photoelectrons`, `post_readout`, and `dn` (the `ReferenceFrame` enum in `radiant.core.quantity`; a plain string works too). Pre-integration frames are spectral-only by design — the chain allows exactly one spectral collapse — so `result.frames["at_aperture"].in_band_value` is deliberately `None` and `signal_at` is the only way to read an in-band scalar there.

3.3.6 Saturation status

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()

well = result.well_status()
well.status          # the clip state string
well.fill_fraction   # dimensionless, 0-1
well.total_well_e     # e-
well.full_well_capacity_e # e-
well.is_saturated     # bool
```

3.3.7 Browsing the whole result

```
print(result.inspect())    # the full readable result tree
print(result.inspect("optics")) # scoped to one stage
```

3.3.8 Saving and reloading a result

```
from radiant.api import ChainResult

path = result.save("run_042.zip")    # single-file zip archive: JSON + npz
again = ChainResult.load("run_042.zip")
again.metrics["snr"]                # every state accessor works as before
```

The archive holds the full `ChainState` — frames, noise terms, stage outputs, MTF terms, metrics, history — plus the provenance record frozen at save time. A reloaded result has no attached `ParameterSet`; the run's resolved parameters live in the provenance record, returned unchanged by `to_provenance_record()`.

3.3.9 Provenance

```
rec = result.to_provenance_record()
rec["run_id"]           # UUID4 minted by the chain runner
rec["radiant_version"]  # e.g. '0.2.0'
rec["git_commit"]       # short SHA, or 'unknown' outside a git repo
rec["python_version"]   # 'MAJOR.MINOR.PATCH'
rec["dependency_versions"] # {name: version} for numpy, scipy, pyyaml, click
rec["parameter_set"]    # {dotpath: ResolvedValue.to_dict()} – every parameter
rec["input_file_hashes"] # [{path: ..., 'sha256': ...}] for every config consumed
rec["active_models"]    # the ordered stage names that ran
```

Provenance is mandatory and cannot be disabled. The helpers never raise on environmental edge cases — no git, a missing dependency — they degrade to "unknown" rather than blocking a run.

3.4 Parameter sweeps

3.4.1 1-D sweep

```
from radiant.api import Sensor
import numpy as np

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.sweep(
    "optics.aperture_diameter_m",
    np.linspace(0.10, 0.60, 11),
    metric="snr",
)
# result.values – array of aperture values
# result.metric_values – corresponding SNR values
```

SweepResult carries param_name, values, metric_values, metric_name, and — when keep_results=True (the default) — the full ChainResult at every point in results. Any other metric can then be pulled across the whole sweep, and the sweep exports directly:

```
snr_curve = result["snr"]           # KeyError if keep_results was False
nedt_curve = result["nedt_K"]       # K
result.to_csv("aperture_sweep.csv") # param column + every kept metric
first = result.at_metric_threshold(50.0) # (param_value, metric_value) or None
```

metric also accepts a callable (ChainResult) -> float, so a derived quantity can be swept without adding a metric:

```
result = sensor.sweep(
    "optics.aperture_diameter_m",
    np.linspace(0.10, 0.60, 11),
    metric=lambda r: r.metrics["snr"] / r.metrics["mtf_at_nyquist"],
)
```

n_workers above 1 runs the points in parallel processes; keep_results=False drops the per-point ChainResult objects when only the metric curve is wanted.

3.4.2 2-D sweep

```
from radiant.api import Sensor
import numpy as np

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.sweep_2d(
    "optics.aperture_diameter_m", np.linspace(0.10, 0.60, 6),
    "spectral_integration.integration_time_s", np.array([0.001, 0.005, 0.010]),
    metric="snr",
)
# result.grid - 2D array, shape (len(values1) x len(values2))
```

Sweep2DResult carries param1_name, param2_name, values1, values2, grid, and metric_name. result.to_csv(path) writes the grid in long form (param1, param2, metric).

3.5 Solving for a parameter value

The inverse of a sweep: find the parameter value that hits a target metric. Brent root finding runs on the forward model over the given bracket, in input units.

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
res = sensor.solve_for(
    "optics.aperture_diameter_m",
    50.0, # target SNR (dimensionless)
    bounds=(0.05, 1.0), # m
    metric="snr",
)
res.solution # aperture [m] that gives SNR = 50
```

If the target is not bracketed by the two endpoints, solve_for raises radiant.api.solve.SolveBracketError carrying both endpoint metric values — so the message tells you which way to widen the bracket.

3.6 Monte Carlo tolerance analysis

Assign statistical tolerances to parameters and run a Monte Carlo ensemble:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set_tolerance("optics.transmission_scalar", "gaussian", mean=0.70, std=0.03)
sensor.set_tolerance("detector.qe_value", "gaussian", mean=0.70, std=0.05)

mc = sensor.monte_carlo(n_trials=200, seed=42)
# mc.mean("snr"), mc.std("snr") - summary statistics
# mc.metric_names - tuple of available metric names
```

```
# mc.metric_array - 2D array (n_trials x n_metrics)
```

The supported distributions are "gaussian", "uniform", "truncated_gaussian", and "log_normal"; the keyword arguments are the distribution's own parameters. `sensor.tolerances()` is the read-back view and `clear_tolerance(dotpath)` removes one. Tolerances persist through `save()` / `to_yaml()` in the `_radiant.tolerances` block.

`MonteCarloResult` carries `n_trials`, `seed`, `metric_names`, `metric_array`, `sampled_params` (dot-path → the per-trial sampled values), and — with `keep_results=True` — every `ChainResult`. Its query surface:

```
mc.mean("snr")                # dimensionless
mc.std("snr")                  # dimensionless
mc.percentile("snr", 5.0)      # 5th-percentile SNR
mc.probability_of_exceeding("snr", 20.0) # fraction of trials with SNR > 20
mc.correlation("snr")          # {param: Pearson r} against each sampled input
mc.to_dict()                   # {metric or param: 1-D array}
mc.to_csv("tolerance_trials.csv") # one row per trial
```

The run is fully deterministic in seed: the same seed and the same tolerances give identical trials.

3.7 Sensitivity analysis

Determine which parameters have the largest impact on a metric:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sa = sensor.sensitivity(
    metric="snr",
    param_names=[
        "optics.aperture_diameter_m",
        "optics.focal_length_m",
        "optics.transmission_scalar",
        "detector.qe_value",
        "spectral_integration.integration_time_s",
    ],
)
for e in sa.entries:          # sorted by |sensitivity|, descending
    print(f"{e.param_name}: {e.sensitivity:+.3f} "
          f"(nominal {e.nominal_value:g}, metric {e.metric_nominal:.4g})")

sa.param_names      # dot-paths in ranked order
sa.sensitivities     # the ranked sensitivity values
sa.to_dict()         # {dot-path: sensitivity}
```

Each `SensitivityEntry` carries `param_name`, `nominal_value`, `metric_nominal`, `metric_plus`, `metric_minus`, `sensitivity`, and `delta_fraction`. The sensitivity is normalized and therefore dimensionless:

$$S = \frac{\Delta M/M}{\Delta p/p}$$

so an S of +2.0 means a 1 % increase in the parameter raises the metric by about 2 %. With `param_names=None` the analysis perturbs the tolerated parameters if any are set, otherwise every float parameter. `delta_fraction=0.01` is a $\pm 1\%$ perturbation.

3.8 Progress reporting and cancellation

`sweep`, `sweep_2d`, `monte_carlo`, `sensitivity`, and `BatchRunner.run` all accept the same two callbacks:

- `progress(done, total)` is called after each completed unit of work — a sweep point, a grid cell, an MC trial, a perturbed parameter.
- `cancel()` -> `bool` is polled *before* each unit; returning `True` aborts with `OperationCancelledError`.

```
from radiant.api import OperationCancelledError, Sensor
import numpy as np

stop = False
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")

try:
    result = sensor.sweep(
        "optics.aperture_diameter_m",
        np.linspace(0.10, 0.60, 51),
        progress=lambda done, total: print(f"{done}/{total} points"),
        cancel=lambda: stop,
    )
except OperationCancelledError as exc:
    print(f"{exc.operation} stopped after {exc.done}/{exc.total} evaluations")
```

Both callbacks run on the calling thread — a GUI typically flips a flag from its event loop and reads progress into a bar. An exception raised inside `progress` is **not** swallowed: a broken callback fails the operation loudly. A canceled operation returns no partial result; sweep in chunks if partials are needed.

3.9 Batch matrices

`BatchRunner` evaluates the cartesian product of labeled axes — N targets $\times$ M atmospheres $\times$ K sensors — and returns a tidy table. The runner owns the mechanics (product, per-cell overrides, per-cell failure capture); the caller supplies the physics in a callback.

```
from radiant.api.batch import BatchRunner

base = {
    "source": {"target": {"temperature": 300.0, "emissivity": 0.95}},
    "geometry": {"sensor_altitude_m": 8000.0},
```

```

"optics": {"aperture_diameter_m": 0.30, "focal_length_m": 1.20,
           "transmission_scalar": 0.70},
"detector": {"pixel_pitch_x_um": 18.0, "pixel_pitch_y_um": 18.0,
             "qe_value": 0.70, "dark_rate_e_per_s": 100.0},
"spectral_integration": {"filter_min_um": 3.5, "filter_max_um": 5.0,
                        "integration_time_s": 0.005},
"readout": {"read_noise_e_rms": 5.0, "gain_e_per_dn": 1.0, "adc_bits": 16},
}

axes = [
    ("target", {
        "tank": {"source.target.temperature": 310.0},    # K
        "truck": {"source.target.temperature": 305.0},   # K
    }),
    ("altitude", {
        "low": {"geometry.sensor_altitude_m": 5000.0},   # m
        "high": {"geometry.sensor_altitude_m": 12000.0}, # m
    }),
]

batch = BatchRunner(base, axes).run(
    lambda sensor, labels: {"snr": sensor.evaluate().metrics["snr"]}
)

batch.rows          # one dict per cell: axis labels + outputs + 'error'
batch.n_failed      # cells whose evaluation raised a RadiantError
batch.pivot("snr", rows="target", cols="altitude")

```

The last axis varies fastest, and axis names become result columns — so "error" is reserved. A cell whose evaluation raises a `RadiantError` becomes a row with a populated error column: recorded data, never a silent drop. Any *other* exception is a programming bug and propagates. Failed cells appear as `None` in a pivot.

3.10 Using the data library

The `SpectralLibrary` provides bundled material emissivity, detector QE, and solar irradiance data:

```

from radiant.data import SpectralLibrary

lib = SpectralLibrary()
materials = lib.materials()    # list of 19 material names
emiss = lib.material("aluminum") # SpectralData object

detectors = lib.detectors()    # list of 6 detector names
qe = lib.detector_qe("silicon") # SpectralData object

solar = lib.solar()            # AM0 solar irradiance

```

Each returns a `SpectralData` object with `.wavelength_um` and `.values` arrays, plus `.unit`, `.source`, and `.name` metadata.

3.11 Cloning for comparison

```
from radiant.api import Sensor

baseline = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
upgraded = baseline.clone()
upgraded.set("optics.aperture_diameter_m", 0.50)

r_base = baseline.evaluate()
r_upgrade = upgraded.evaluate()
delta_snr = r_upgrade.metrics["snr"] - r_base.metrics["snr"]
```

3.12 Configuration sets

Cloning gives you two independent sensors. A ConfigurationSet instead keeps **one** document with up to twelve named *configurations* of the same problem: a parameter is **shared** by default and carries one value per configuration only once you `configure()` it. Shared edits move every configuration at once, and the whole study saves as a single config file.

```
from radiant.api import ConfigurationSet, Sensor

base = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
cs = ConfigurationSet(base, names=["MWIR", "LWIR"])

# Configured: one value per configuration, in names() order, in input units.
cs.configure("spectral_integration.filter_min_um", [3.5, 8.0])           # um
cs.configure("spectral_integration.filter_max_um", [5.0, 12.0])         # um
cs.configure("spectral_integration.integration_time_s", [0.005, 0.0005]) # s
cs.configure("readout.full_well_capacity_e", [2.0e6, 6.0e6])            # e-
cs.configure("detector.qe_value", [0.70, 0.55])                        # dimensionless
cs.set_value("detector.qe_value", "LWIR", 0.62)                       # one
configuration
cs.base.set("optics.aperture_diameter_m", 0.35)                       # m – shared: moves both
cs.baseline = "MWIR"                                                  # delta reference

cs.names()                    # ('MWIR', 'LWIR') – set order
cs.configured()               # dot-path -> one value per configuration
cs.validate_all()              # {name: None or error} – resolve-only, no physics
lwir = cs.sensor_for("LWIR")   # materialize one configuration as a Sensor

run = cs.evaluate_all()        # every configuration, active first
run.result_for("LWIR").metrics["snr"] # dimensionless
run.warnings                   # {name: messages} – attributed per configuration
print(run.summary())           # one triage line per configuration, with units
print(cs.compare(run).to_table()) # metric x configuration, deltas vs the baseline

yaml_text = cs.to_yaml()       # the whole study as one document
```

`cs.save(path)` writes that document and `ConfigurationSet.load(path)` reads it back — names and order, active / baseline, per-configuration `wavelength_points`, and the configured table all round-trip. A plain config file loads as the degenerate one-configuration set; a study file loaded through `Sensor.from_yaml` raises an error pointing at `ConfigurationSet.load`.

The repository's scripting-API specification carries the full member list, and `examples/scripts/dual_band_configuration_set.py` is the worked study.

3.13 Exporting results

3.13.1 CLI export

```
radiant run examples/mwir_leo_minimal.yaml --format json --output result.json
radiant run examples/mwir_leo_minimal.yaml --format csv --output result.csv
```

3.13.2 Python — extract to dict

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
result = sensor.evaluate()
metrics_dict = dict(result.metrics)
# Write to CSV/JSON with standard Python libraries
```

3.14 See also

Example scripts in `examples/scripts/`:

- `basic_evaluation.py` — load, evaluate, inspect
- `aperture_sweep.py` — 1D sweep with plotting
- `tolerance_analysis.py` — Monte Carlo workflow
- `compare_configs.py` — side-by-side comparison of two `Sensor` objects
- `custom_loop.py` — advanced iteration patterns
- `dual_band_configuration_set.py` — a `ConfigurationSet` study (MWIR vs LWIR on one telescope), worked end to end: see **Configuration sets** above

Plot functions are available in `radiant.api.plot` for sweep results, noise budgets, PSF images, MTF curves, and spectral data visualization.

Command-Line Interface

Installing RADIANT puts a single console entry point on the path, `radiant` (`radiant.cli.main:cli`). It is a Click command group: `radiant --help` lists the subcommands, and `radiant <subcommand> --help` gives that subcommand's options.

Usage: `radiant [OPTIONS] COMMAND [ARGS]...`

RADIANT – first-principles E0 sensor performance modeling.

Options:

`--version` Show the version, load path, and git commit, then exit.
`--help` Show this message and exit.

Commands:

`compare` Compare two configurations side-by-side.
`convert` Convert a value between RADIANT-supported units.
`explain` Explain how a parameter got its value.
`gui` Launch the RADIANT desktop GUI, optionally on a YAML config file.
`run` Run the RADIANT signal chain from a YAML config file.
`schema` List all parameter definitions.
`sweep` Run a 1-D parameter sweep.
`template` Manage sensor configuration templates.
`tolerance` Run Monte Carlo tolerance analysis.
`validate` Validate a YAML config file without running the chain.

Exit codes. Every command exits 0 on success and 1 on a RADIANT-level failure — a missing file, a validation error, an unknown parameter, an unresolvable config. Click itself exits 2 for a malformed command line. That makes the CLI usable in a shell script without parsing its text output.

--version is a provenance tool, not a banner. It prints the version, the directory the running package was imported from, and the git commit — which is what a stale-install mismatch actually needs:

```
$ radiant --version
radiant 0.1.0
  loaded from: /Users/example/ssr/src/radiant
  git commit: 738ad8c0 (dirty)
```

4.1 `radiant run` — evaluate a config

Usage: `radiant run [OPTIONS] CONFIG`

Option	Meaning
<code>--set TEXT</code>	Parameter override, key=value, repeatable
<code>--configuration NAME</code>	Which configuration of a study file to evaluate
<code>--wavelength-min FLOAT</code>	Override the spectral grid minimum [μm]
<code>--wavelength-max FLOAT</code>	Override the spectral grid maximum [μm]
<code>--wavelength-points INTEGER</code>	Grid point count (default 500 points)
<code>--output PATH</code>	Write results to a file
<code>--provenance PATH</code>	Write the provenance record to a JSON file
<code>--format [text json csv]</code>	Output format (default text)
<code>--quiet</code>	Suppress everything except the final metric summary

```

radiant run examples/mwir_leo_minimal.yaml
radiant run examples/mwir_leo_minimal.yaml --set optics.aperture_diameter_m=0.5
radiant run examples/mwir_leo_minimal.yaml --format json --output result.json
radiant run examples/mwir_leo_minimal.yaml --provenance run_provenance.json
radiant run study.yaml --configuration LWIR

```

The default text output is the signal and the full noise budget, every term with units, followed by the RSS total and the headline metric:

```

Signal: 1263548.28 e-
  signal_shot      1124.0766 e- RMS
  background_shot  0.0000 e- RMS
  dark_shot        0.7071 e- RMS
  read_noise       5.0000 e- RMS
  quantization     9.2376 e- RMS
  ...
Noise (RSS): 1124.1259 e- RMS
SNR:      1124.03

```

`--configuration` is **required** for a study file (one carrying a `configurations:` section) and **rejected** for a plain config — a study is never silently run as if its shared body were the whole model.

`--set` overrides sit at the top of the precedence ladder: schema default → config-file body → programmatic `Sensor.set()` → CLI `--set`.

4.2 radiant validate — check a config without running it

Usage: `radiant validate [OPTIONS] CONFIG`

Checks that the file parses, every parameter name is known, types are correct, and every required parameter is set. It reports **all** errors at once rather than stopping at the first — this collect-all behavior is specific to this command; the scripting API and the GUI are fail-fast by design.

```

$ radiant validate examples/mwir_leo_minimal.yaml
Config OK: examples/mwir_leo_minimal.yaml
218 parameters resolved.

```

A study file validates **every** configuration and prints one line each, so no configuration’s failure hides another’s. The exit status is non-zero if any failed.

--set is accepted here too, so an override can be validated before it is run.

4.3 radiant explain — where a value came from

Usage: radiant explain [OPTIONS] CONFIG PARAM

Prints the value, the unit, the provenance class, the source label, and the derivation chain for one parameter:

```
$ radiant explain examples/mwir_leo_minimal.yaml optics.f_number
optics.f_number = 4.0 (canonical: 4.0 )
  Description: Dimensionless f/# = focal_length_m / aperture_diameter_m. Part of the
               {D, f, f/#} consistency group; supply any two and the third is derived.
  Provenance: derived
  Source: derived: f_number = focal_length_m / aperture_diameter_m
  Derived from:
    optics.aperture_diameter_m = 0.3
    optics.focal_length_m = 1.2
```

This is the command to reach for when a number is not what you expected: it distinguishes “you set it”, “the file set it”, “the schema defaulted it”, and “a consistency group derived it from these two inputs”.

4.4 radiant gui — launch the desktop application

Usage: radiant gui [OPTIONS] [CONFIG]

```
radiant gui
radiant gui examples/mwir_leo_minimal.yaml
```

The GUI ships with the base install (PySide6 is a core dependency). If a GUI dependency fails to import, the command exits 1 with an actionable message naming what is missing and how to reinstall.

With no CONFIG the window opens on the welcome screen — mission-template cards, **Blank config**, the worked examples and the recent-files list — exactly the surface **File ▶ New** returns to. Passing a study file opens it with its configuration tab strip.

4.5 radiant sweep — 1-D sweep from the shell

Usage: radiant sweep [OPTIONS] CONFIG PARAM

Option	Meaning
--min FLOAT	Sweep minimum (required), in the parameter’s input unit
--max FLOAT	Sweep maximum (required), in the parameter’s input unit

Option	Meaning
<code>--steps</code> INTEGER	Number of points (default 10)
<code>--metric</code> TEXT	Metric key to report (default snr)
<code>--output</code> PATH	Save to JSON or CSV — the extension picks the format
<code>--plot</code> PATH	Save a PNG plot (requires matplotlib)
<code>--set</code> TEXT	Parameter override, repeatable

```
radiant sweep examples/mwir_leo_minimal.yaml optics.aperture_diameter_m \
  --min 0.15 --max 0.60 --steps 10 --metric snr --output sweep.csv
```

4.6 radiant tolerance — Monte Carlo from the shell

Usage: `radiant tolerance [OPTIONS] CONFIG`

Option	Meaning
<code>--trials</code> INTEGER	Number of MC trials (default 100)
<code>--seed</code> INTEGER	Random seed (default 42)
<code>--tolerance</code> TEXT	"param distribution key=val ...", repeatable
<code>--output</code> PATH	Save results to JSON
<code>--set</code> TEXT	Parameter override, repeatable

```
radiant tolerance examples/mwir_leo_minimal.yaml --trials 50 \
  --tolerance "optics.aperture_diameter_m gaussian std_fraction=0.02" \
  --tolerance "optics.transmission_scalar gaussian std_fraction=0.05"
```

Tolerances may also live in the config's `_radiant.tolerances` block, in which case no `--tolerance` flag is needed. The run is reproducible from `--seed`.

4.7 radiant compare — two files side by side

Usage: `radiant compare [OPTIONS] CONFIG1 CONFIG2`

Evaluates both configs and prints a metric diff table; `--output PATH` saves it as JSON.

```
radiant compare config_a.yaml config_b.yaml
```

This compares two *files* — two separate designs. To compare named configurations *within* one study file, use `ConfigurationSet` from Python (see the Configuration Guide).

4.8 radiant schema — list the parameter definitions

Usage: `radiant schema [OPTIONS]`

Option	Meaning
<code>--format [text json]</code>	Output format (default text)
<code>--stage TEXT</code>	Filter to one stage prefix, e.g. optics, detector

```

radiant schema
radiant schema --stage optics
radiant schema --format json > schema.json

```

The same registry that backs this command generates the Parameter Reference chapter of this volume, so the two never disagree.

4.9 radiant template — bundled starting points

Usage: `radiant template [OPTIONS] COMMAND [ARGS]...`

Commands:

```

create  Copy a template YAML to a file as a starting point.
list    List the bundled mission templates (the GUI welcome-screen set).
show    Print a template configuration as YAML.

```

```

radiant template list
radiant template show leo_mapping_extended
radiant template create leo_mapping_extended

```

The nine bundled mission templates are the same set the GUI welcome screen offers. Each one is a complete, runnable config spanning a distinct band, altitude, and regime:

Template	Band	Geometry	Regime
<code>aerial_vnir_imaging</code>	VNIR 0.4–0.9 μm	3 km nadir	extended reflective
<code>airborne_lwir_surveillance</code>	LWIR 8–12 μm	8 km airborne	extended
<code>geo_lwir_staring</code>	LWIR 8–12 μm	GEO nadir	extended
<code>ground_to_air_mwir_detection</code>	MWIR 3–5 μm	up-looking, 60° zenith	point source
<code>lab_blackbody_calibration</code>	MWIR 3.5–5 μm	5 m horizontal path	extended blackbody
<code>leo_mapping_extended</code>	MWIR 3.5–5 μm	500 km nadir	extended
<code>leo_swir_mapping</code>	SWIR 1.0–2.5 μm	500 km nadir	extended reflective
<code>maritime_subpixel_lwir</code>	LWIR 8–12 μm	500 km LEO	sub-pixel vessel
<code>sda_space_to_space</code>	MWIR 3.5–5 μm	LEO → GEO, exo path	point source

They ship inside the package at `radiant/data/templates/`, so they arrive with `pip install radiant`; in a source checkout the same files sit at `src/radiant/data/templates/`.

4.10 radiant convert — scalar unit conversion

Usage: `radiant convert [OPTIONS] VALUE FROM_UNIT TO_UNIT`

```
$ radiant convert 18 um m  
18.0 um = 1.8e-05 m
```

```
$ radiant convert 45 deg rad  
45.0 deg = 0.785398 rad
```

```
$ radiant convert 5 ms s  
5.0 ms = 0.005 s
```

A convenience calculator over the same unit registry the parameter boundary uses. It converts scalars only — it does not read or write config files.

Configuration Guide

How to define, customize, and manage RADIANT config files.

5.1 YAML structure

A RADIANT config file is a YAML document whose top-level keys are the signal-chain stage names. There are ten parameter namespaces in all — geometry, source, atmosphere, optics, platform, spectral_integration, detector, readout, calibration, performance — and a config only names the ones it needs. Here is an annotated example using the seven most commonly populated sections:

```
# --- Source (target and background) ---
source:
  target:
    temperature: 300.0      # K – target kinetic temperature
    emissivity: 0.95        # dimensionless, 0–1
    # fill_fraction: 0.5    # for sub-pixel targets (0–1)
    # projected_area_m2: 4.0 # target cross-section (m^2)
    # range_m: 10000.0       # slant range to target (m)
  background:
    temperature: 290.0      # K – background temperature (default)
    emissivity: 0.95        # background emissivity (default)
    # regime_override: auto  # auto | extended | sub_pixel | point_source

# --- Atmosphere ---
atmosphere:
  standard_atmosphere: midlat_summer # midlat_summer | midlat_winter | ...
  # model: simple                    # simple | interpolated | tabulated | modtran | exo
  # visibility_km: 23.0              # meteorological visibility
  # precipitable_water_cm: 1.4       # precipitable water vapor

# --- Geometry ---
geometry:
  sensor_altitude_m: 8000.0 # m – sensor altitude above ground
  # target_altitude_m: 0.0   # m – target elevation
  # path_zenith_rad: 0.0     # rad – off-nadir angle (0 = nadir)

# --- Optics ---
optics:
  aperture_diameter_m: 0.30 # m
  focal_length_m: 1.20      # m → f/4.0 (derived)
  transmission_scalar: 0.70 # end-to-end optical transmission
```

```

# obscuration_ratio: 0.0      # central obscuration ratio
# wfe_rms_waves: 0.0          # wavefront error in waves

# --- Detector ---
detector:
  pixel_pitch_x_um: 18.0      # um - pixel pitch cross-track
  pixel_pitch_y_um: 18.0      # um - pixel pitch along-track
  qe_value: 0.70              # quantum efficiency (flat)
  dark_rate_e_per_s: 100.0    # e-/s - dark current
  # read_noise handled in readout section
  # fill_factor: 1.0          # pixel fill factor (default)

# --- Spectral Integration ---
spectral_integration:
  filter_min_um: 3.5          # um - bandpass start
  filter_max_um: 5.0          # um - bandpass end
  integration_time_s: 0.005    # s - detector integration time

# --- Readout ---
readout:
  read_noise_e_rms: 5.0       # e- RMS
  gain_e_per_dn: 1.0          # e-/DN - system gain
  adc_bits: 16                # ADC bit depth
  # n_tdi: 1                  # TDI stages (default = 1 = off)
  # n_coadds: 1               # number of coadded frames

```

Besides the parameter namespaces, four top-level keys carry something other than parameters. Each is covered in its own section below:

Key	What it is
<code>_radiant</code>	Session metadata: format marker, spectral grid density, tolerance distributions
<code>optical_elements</code>	A declarative optical-element train
<code>fpa</code>	The name of a bundled FPA preset to apply
<code>configurations</code>	Turns the file into a multi-configuration <i>study</i>

Three further top-level keys — `_extends`, `_imports`, `_vars` — are **reserved** for config-inheritance features that are designed but not implemented. A config containing one is rejected with an actionable error rather than loading with the directive silently ignored, which would produce physics from a different parameter set than intended.

The Parameter Reference chapter of this volume lists all 218 parameters with types, defaults, bounds, and descriptions. That chapter is generated from the schema registry, so it cannot drift from the code.

5.2 Parameter dot-path convention

Every parameter has a dot-separated path that maps directly to YAML nesting:

Dot-path	YAML location
optics.aperture_diameter_m	optics: aperture_diameter_m:
source.target.temperature	source: target: temperature:
spectral_integration.filter_min_um	spectral_integration: filter_min_um:

This path is used everywhere: CLI overrides, Python API, explain, sweep.

5.3 Defaults and required parameters

Most parameters have sensible defaults. Ten carry none, and a configuration that omits one cannot resolve:

- geometry.sensor_altitude_m
- optics.aperture_diameter_m, optics.focal_length_m, optics.f_number
- detector.pixel_pitch_x_um, detector.pixel_pitch_y_um
- detector.qe_value
- spectral_integration.filter_min_um, spectral_integration.filter_max_um
- spectral_integration.integration_time_s

The three optics parameters are a consistency group, so you state any two and the third is derived — which is why all three appear here and no configuration sets all three. The **required** column of the Parameter Reference is the authoritative list; it is generated from the schema.

Separately, a handful of parameters *do* default but should be stated explicitly in any configuration whose results you intend to defend, because the default is a convention rather than a property of your sensor: source.target.temperature, source.target.emissivity, readout.read_noise_e_rms, readout.gain_e_per_dn, and readout.adc_bits.

Everything else defaults to a physically reasonable value. Run `radiant validate <config>` to check completeness.

5.4 Overriding parameters

5.4.1 CLI: --set

```
radiant run config.yaml --set optics.aperture_diameter_m=0.50
radiant run config.yaml --set optics.aperture_diameter_m=0.50 \
    --set detector.qe_value=0.80
```

5.4.2 Python: sensor.set()

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set("optics.aperture_diameter_m", 0.50)
```

```
sensor.set("detector.qe_value", 0.80)
result = sensor.evaluate()
```

Or set multiple at once:

```
from radiant.api import Sensor

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
sensor.set_many({
    "optics.aperture_diameter_m": 0.50,
    "detector.qe_value": 0.80,
})
result = sensor.evaluate()
```

5.5 Consistency groups

Some parameters are linked by physical relationships. The f-number consistency group enforces:

$$f/\# = f/D$$

If you specify `aperture_diameter_m` and `focal_length_m`, then `f_number` is derived automatically. If you specify all three and they conflict, RADIANT raises an error.

Check how a derived value was computed:

```
radiant explain examples/mwir_leo_minimal.yaml optics.f_number
```

5.6 Units

RADIANT uses canonical internal units (meters, radians, seconds, etc.) but accepts common input units. The `radiant convert` utility helps:

```
radiant convert 18 um m      # 18 um = 1.8e-05 m
radiant convert 45 deg rad   # 45 deg = 0.785398 rad
radiant convert 5 ms s       # 5 ms = 0.005 s
```

Parameter values in YAML are in the input units the Parameter Reference chapter documents — for example, pixel pitch is specified in micrometers, altitude in meters.

5.7 Resolution precedence

Parameters resolve in a fixed priority order, lowest to highest:

Priority	Source	Provenance tag
1 (lowest)	Schema default (<code>ParameterDef.default</code>)	DEFAULT
2	An applied FPA preset	PRESET
3	The config-file body	CONFIG_FILE
4	<code>Sensor.set()</code> / <code>set_many()</code>	USER_SET
5 (highest)	CLI <code>--set</code>	USER_SET

An FPA preset sits below the config file deliberately: presets seed, explicit values win, in any key order.

Every resolved parameter carries its provenance tag plus a source label — the config file path, `Sensor.set`, or the CLI. Derived parameters carry `DERIVED` and a `derived_` from record naming the inputs and their values, which is what `radiant explain` prints.

Setting every member of a consistency group explicitly triggers a consistency check; a value conflicting with the derived one beyond the group tolerance is an error.

5.8 File paths inside a config

Parameters that name a file (a QE curve, a measured emissivity spectrum, a `tape7`, a Zernike export) are resolved **relative to the config file's own directory**, not the process working directory. A config and its data files therefore move together as a unit.

`Sensor.save()` and `to_yaml(relative_to=...)` write the reverse transform: an absolute path stored in memory comes back out relative to the directory the YAML will live in. That keeps a round-tripped config portable rather than pinned to one machine.

5.9 Using templates

RADIANT ships nine mission templates spanning VNIR, SWIR, MWIR, and LWIR bands at altitudes from a 5 m lab bench to GEO, covering all three radiometric regimes. They are the same set the GUI welcome screen offers.

```
radiant template list           # see all available templates
radiant template show leo_mapping_extended # print the YAML
radiant template create leo_mapping_extended # write leo_mapping_extended.yaml
```

The templates ship inside the installed package at `radiant/data/templates/`; in a source checkout the same files sit at `src/radiant/data/templates/`. Either way a template can be run in place:

```
radiant run src/radiant/data/templates/leo_mapping_extended.yaml
```

The per-template table is in the Command-Line Interface chapter, under `radiant template`.

5.10 Session metadata — the `_radiant` block

`Sensor.save(path)` writes an optional top-level `_radiant` mapping holding session state that is not a chain parameter:

```
_radiant:
  format: 1
  wavelength_points: 500          # spectral grid density
  tolerances:                     # only present when tolerances are set
    detector.qe_value:
      distribution: gaussian
      params: {std: 0.02}
  optics:
    aperture_diameter_m: 0.3      # m
```

The loader strips the block before parameter flattening, applies tolerances, and raises a `ConfigError` on a malformed block — a non-mapping, a missing `distribution` or `params`, or an unknown parameter name. `wavelength_points` is session level: `Sensor.load(path)` consumes it; a bare parameter load ignores it. Configs without a `_radiant` block are unaffected, and a file `Sensor.save` wrote remains loadable by `from_yaml` and the CLI.

Note what a saved file contains: **explicitly-set inputs only**, in input units. Defaults and derived values are not written, so reloading reproduces the original resolution and provenance exactly rather than freezing today's defaults into the file.

5.11 Optical element trains — the `optical_elements` section

A config may carry a declarative optical-element document instead of (or alongside) the scalar `optics.transmission_scalar`. Each entry names an element, its transfer mode, its temperature, and its reflectance or transmittance:

```
optics:
  aperture_diameter_m: 0.3      # m
optical_elements:
  - {name: M1, transfer_mode: REFLECTIVE, reflectance: 0.97, temperature_K: 293.0}
  - {name: cold_filter, transfer_mode: REFRACTIVE, kind: FILTER,
    transmittance: 0.90, temperature_K: 240.0}
```

R and T values may be scalars, paths to a spectral CSV, or an inline spectral table (`{wavelength_um: [...], values: [...]}` — the form the GUI's spectrum dialog writes, which persists in the YAML with no external file).

Emissivity never appears in an entry. It is derived from Kirchhoff's law — $\varepsilon = 1 - R$ for a mirror, $\varepsilon = 1 - T - R$ for a transmissive element. An entry that supplies both R and ε over-specifies the energy balance and is rejected.

`Sensor.from_yaml` / `Sensor.load` / `Sensor.from_dict` parse and attach the section, and `Sensor.save` writes it back out. Spectral-file references are made absolute on attach — so the document evaluates from

any working directory — and relative to the destination directory on save, so a saved element-bearing config stays portable: move the config and its data files together and the references still resolve.

`radiant run` and `radiant validate` both act on the section: `run` parses the document onto the run grid, `validate` normalizes it and reports its errors.

5.12 Named FPA presets — the `fpa` key

A scalar section naming one preset from the bundled FPA library:

```
fpa: teledyne-h2rg-2p5
readout:
  read_noise_e_rms: 6.0  # e- RMS — explicit, wins over the preset's value
```

Every `detector.* / readout.*` value the preset carries is applied with provenance PRESET **unless the config sets that dot-path explicitly — explicit values always win**, in any key order. The applied set and the kept set are both reported through `Sensor.fpa_applications()`.

`Sensor.save` writes the applied values as ordinary explicit inputs and does *not* re-serialize the `fpa:` key: reloading a saved config reproduces the same numbers with config-file provenance, while the preset attribution lives in the preset document. The part list is in the Data Libraries chapter.

5.13 Atmosphere configuration

RADIANT ships a real MODTRAN-derived atmosphere library, so a high-fidelity atmosphere needs no MODTRAN license and no external files:

```
atmosphere:
  model: interpolated  # measured MODTRAN data, interpolated over the scene geometry
```

With `interpolated` and no `atmosphere.interpolated_data_dir`, the loader selects the bundled family that matches the scene's line-of-sight direction and the configured `atmosphere.interpolation_axes`. The model does not extrapolate: outside a family's nodes it refuses, by design, rather than inventing an answer. Point it at your own MODTRAN run matrix with:

```
atmosphere:
  model: interpolated
  interpolated_data_dir: path/to/my_runs  # directory of NPZ runs
```

`atmosphere.model` takes five values. `simple` is the always-works analytic baseline — the only backend that can serve an arbitrary path topology, so it is the right answer when you are unsure. `exo` is an explicitly vacuum path. The two file-driven backends are geometry-agnostic by construction — they do not respond to the scene at all — and need explicit paths:

```
atmosphere:
  model: tabulated
  tabulated_transmittance_file: path/to/transmittance.csv  # or a single .npz
  tabulated_path_radiance_file: path/to/path_radiance.csv
```

```

    tabulated_downwelling_file: path/to/downwelling.csv      # optional
atmosphere:
  model: modtran
  modtran:
    tape7_path: path/to/tape7

```

CSV files must have `wavelength_um` as the first column. Choosing between the five backends for a given scene is the subject of the repository’s atmosphere-selection guide.

The shipped families, their coverage, and the provenance of the underlying MODTRAN run matrix are documented in the Data Libraries chapter and in the shipped library’s own manifest. Ingest of external atmosphere data is covered in the External Data Interfaces chapter.

5.14 Configuration sets — several configurations in one file

One config file can describe **one modeling problem in up to twelve named variants of itself**: MWIR vs. LWIR on the same telescope, nominal vs. as-built, three off-nadir geometries. Add a top-level `configurations:` section and the file becomes a **study**.

Three words, used consistently everywhere in RADIANT:

- **config file** — the YAML artifact on disk.
- **configuration** — one member of a configuration set (MWIR, LWIR).
- **configuration set** (or **study**) — the whole document: the shared parameters plus the per-configuration ones.

Everything in the ordinary body of the file is **shared** — one value for every configuration. The `configurations:` section names the configurations and lists only the parameters that *differ*, each as a dense list of values aligned with the names:

```

# --- shared body: an ordinary RADIANT config, one value for ALL configurations ---
_radiant:
  format: 1
  wavelength_points: 500      # shared spectral grid density
geometry:
  sensor_altitude_m: 8000.0   # m
optics:
  aperture_diameter_m: 0.30   # m
  focal_length_m: 1.20        # m -> f/4.0
  transmission_scalar: 0.70   # dimensionless
detector:
  pixel_pitch_x_um: 18.0      # um
  pixel_pitch_y_um: 18.0      # um
source:
  target:
    temperature: 300.0        # K
    emissivity: 0.95          # dimensionless

# --- what makes this file a study ---

```

```

configurations:
  names: [MWIR, LWIR]           # 1-12 unique names; defines the value order below
  active: MWIR                  # configuration the GUI opens on (optional)
  baseline: MWIR                # delta reference for comparisons (optional)
  wavelength_points:           # optional per-configuration grid density
    LWIR: 300                   # points (MWIR inherits the shared 500)
  parameters:                  # dot-path -> one value per name, in input units
    spectral_integration.filter_min_um: [3.5, 8.0]           # um
    spectral_integration.filter_max_um: [5.0, 12.0]         # um
    spectral_integration.integration_time_s: [0.005, 0.0005] # s
    detector.qe_value: [0.70, 0.55]                         # dimensionless
    readout.full_well_capacity_e: [2.0e6, 6.0e6]            # e-

```

Read that as a table: MWIR is 3.5–5.0 μm integrated 5.0 ms at QE 0.70, LWIR is 8.0–12.0 μm integrated 0.5 ms at QE 0.55, and both look through the same 0.30 m f/4 telescope from 8000 m at the same 300 K scene. Change `optics.aperture_diameter_m` once and both configurations move together — the study states what differs, not what is repeated.

The binding rules, all checked at load time with an error naming the file, the configuration, and the parameter:

- **names** — 1 to 12 unique, non-empty names (`ConfigurationSet.MAX_CONFIGS`). This list defines the order of every value list below it.
- **parameters** — every list has exactly as many values as there are names. The lists are dense by construction: there is no “unset for this configuration” and nothing is padded for you.
- **Shared or configured, never both.** A dot-path that appears in `configurations.parameters` must *not* also appear in the shared body — the shared value would be silently shadowed. Move a parameter into the section, do not copy it.
- **Values are in input units** — exactly the units the shared body uses, so `filter_min_um` is micrometers here too. Type, bounds, and enum checks run per configuration.
- **Shared regardless:** tolerance distributions, the `optical_elements` document, and the default `_radiant.wavelength_points`. Only the grid *density* is per configuration; each configuration’s grid *span* already follows its own resolved band.

Running and validating a study from the CLI:

```

# One configuration by name --- required for a study file.
radiant run study.yaml --configuration LWIR

# Validate EVERY configuration; one line each, non-zero exit if any failed.
radiant validate study.yaml

```

In the GUI, a study file opens with a configuration tab strip above the signal-chain strip, and **Edit -> Configurations...** adds, renames, reorders, and duplicates configurations.

Plain config files are unchanged. A file with no `configurations:` key is byte-for-byte today’s format and loads everywhere exactly as before — nothing in this section is required, and nothing about it changed existing output. A study file, conversely, is only loaded by tools that understand the section: `radiant run -configuration / radiant validate` and, in Python, `ConfigurationSet.load`. Loading one as a plain single sensor is refused with an error pointing at the right entry point, so a study is never silently run as if its shared body were the whole model.

For building, evaluating, and comparing a study, see the repository’s trade-studies guide; the complete section specification is in its config-format document.

5.15 Common patterns

5.15.1 Change one parameter and re-run

```
radiant run config.yaml --set optics.aperture_diameter_m=0.40
```

5.15.2 Compare two config files

```
radiant compare config_a.yaml config_b.yaml
```

This compares two *files* — two separate designs. To compare named configurations *within* one study file, see **Configuration sets** above and the repository’s trade-studies guide.

5.15.3 Batch many scenarios (Python)

```
from radiant.api import Sensor

base = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
altitudes = [5000, 8000, 12000, 20000]

for alt in altitudes:
    s = base.clone()
    s.set("geometry.sensor_altitude_m", alt)
    r = s.evaluate()
    snr = r.metrics["snr"]
    # Process snr for each altitude
```

Parameter Reference

Auto-generated from the parameter registry. Do not edit by hand — re-run `python scripts/gen_param_reference.py` to update.

Total parameters: 218

6.1 geometry

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.circular_orbit</code>	bool	False	—	—	Declare the platform a circular orbit at <code>geometry.sensor_altitude_m</code> — mode V6 entry. When true, the ground-track speed (and orbital period) are derived from the altitude via <code>core.orbit</code> ; do not also set <code>geometry.ground_speed_m_s</code> unless it agrees. False = generic platform (airborne or static); ground speed is taken from <code>geometry.ground_speed_m_s</code> as before.
<code>geometry.day_of_year</code>	int	80	—	(1, 366)	Day of year, 1–366 — mode S3 entry (site + time solar geometry).
<code>geometry.elevation_angle_rad</code>	float	1.5707963	rad	(-1.5708, 1.5708)	Elevation of the line of sight above the local horizontal AT THE PATH'S LOWER ENDPOINT [rad] — mode V4 entry (grazing-angle framing). lower-endpoint zenith = $\pi/2$ - elevation; for the classic sensor-above-target scene the lower endpoint is the target, so this is the sensor's elevation seen from the target and path zenith = $\pi/2$ - elevation exactly as before. NEGATIVE elevation is legal — the path leaves its lower endpoint on a descending shoulder (a level or near-level arm sags below the horizontal). Unused unless explicitly set.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.ground_range_m</code>	float	0.0	m	(0.0, 20000000.0)	Surface arc distance between the sensor's ground point and the target's ground point [m] — mode V3 entry. Direction-free: the arc fixes the central angle $\Delta = \text{arc} / R_E$ whichever endpoint is higher, and the spherical viewing triangle is solved for that ordering. Unused unless explicitly set.
<code>geometry.ground_speed_m_s</code>	float	0.0	m/s	(0.0, 50000.0)	Ground-track speed [m/s]. For LEO at 600 km: ~6900 m/s.
<code>geometry.local_solar_time_h</code>	float	12.0	h	(0.0, 24.0)	Local solar time at the target [hours, 0–24; 12.0 = solar noon] — mode S3 entry. Mutually exclusive with <code>geometry.ltan_h</code> .
<code>geometry.los_angular_rate_rad_s</code>	float	0.0	rad/s	(0.0, 10000.0)	Line-of-sight angular rate [rad/s] entered DIRECTLY — mode K1, the first of the two kinematics doors. This is the rate at which the sensor-target LOS direction rotates, $ v_{\text{rel,perp}} / \text{slant_range}$, including both platform and target motion. Use it when the rate is known (a tracker's measured slew) rather than the velocities. The alternative door is the target-velocity triple (<code>geometry.target_speed_m_s / target_heading_rad / target_climb_rad</code>), from which the same rate is derived; setting both is legal only if they agree within 1%. Unused unless explicitly set: with neither door set the published rate is the platform-only value derived from <code>geometry.ground_speed_m_s</code> .
<code>geometry.ltan_h</code>	float	12.0	h	(0.0, 24.0)	Local time of ascending node [hours] for a sun-synchronous orbit — mode S3 entry; the local solar time is derived via <code>core.solar_geometry.local_solar_time_from_ltan</code> . Mutually exclusive with <code>geometry.local_solar_time_h</code> .

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.path_zenith_rad</code>	float	0.0	rad	(0.0, 3.141592653589793)	Line-of-sight zenith angle at the path's LOWER endpoint [rad] — mode V1 entry. When the sensor is above the target (every classic scene) the target IS the lower endpoint, so this is the canonical target-side zenith θ_o and 0 = sensor at the target's zenith (nadir view). When the sensor is below the target it is the sensor's own zenith angle, and $\theta_o = \pi - \theta_{up}$ is derived from it (0 = target directly overhead). The sensor-side off-boresight angle is <code>geometry.sensor_off_boresight_rad</code> .
<code>geometry.scene_class</code>	str	auto	—	—	OPTIONAL assertion of the scene class — the observer x target altitude-band label of the scene taxonomy (ground: $h < 1$ km; air: 1-100 km; space: $h > 100$ km, the <code>h_atm_top</code> convention). The class is ALWAYS derived from <code>geometry.sensor_altitude_m</code> and <code>geometry.target_altitude_m</code> and published as <code>stage_outputs['geometry']['scene_class']</code> ; setting this parameter does not select anything, it ASSERTS what the altitudes should derive. If the assertion disagrees with the derivation the stage raises <code>GeometrySpecificationError</code> naming both — which is how a wrong-magnitude altitude typo (600 m where 600 km was meant) is caught, since pure derivation renders it as a self-consistent scene of the wrong class. Physics NEVER branches on the class: it drives defaults, metric relevance, validation, and GUI composition only. 'auto' = unset (no assertion, the default).

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.sensor_altitude_m</code>	float	required	m	(0.0, 100000000.0)	Sensor altitude above mean sea level [m]. 0 = a ground-based sensor (legal; the LOS direction is derived from this altitude against <code>geometry.target_altitude_m</code> , never declared). Also the altitude the <code>no_atmosphere</code> ‘space’ sub-case uses for the Earth-limb intercept check (formerly the separate <code>platform.h_sensor</code> stop-gap — folded as a deprecated alias).
<code>geometry.sensor_off_boresight_rad</code>	float	0.0	rad	(0.0, 1.5707)	Sensor off-BORESIGHT angle [rad] — mode V2 entry. The reference axis is resolved from the altitudes, never declared: the sensor’s NADIR when the sensor is above the target (the classic off-nadir look angle η , converted to the target-side path zenith by the spherical-Earth sine rule, <code>core.los_geometry.theta_o_from_eta</code>), and the sensor’s ZENITH when the sensor is at or below the target — where the sensor is the path’s lower endpoint, so the entered angle already is the lower-endpoint zenith and <code>theta_o</code> is derived from it. Unused unless explicitly set; do not also set <code>geometry.path_zenith_rad</code> unless the two agree.

Parameter	Type	Default	Input Unit	Bounds	Description
geometry.site_elevation_m	float	0.0	m	(0.0, 10000.0)	Terrain elevation of the scene's ground site above mean sea level [m] — the altitude of the SURFACE beneath the line of sight, which is NOT the same thing as the lowest point of the line of sight. It is the reference the surface boundary layer sits on: the Hufnagel-Valley Cn2 surface term is evaluated at (h - site_elevation_m) so that a mountain-top observatory keeps its own boundary layer, while the tropopause and middle-atmosphere terms stay on mean sea level. Whose site it is follows the topology: down-looking = the terrain under the TARGET; up-looking = the terrain under the SENSOR; level = the terrain under the arm (shared - both endpoints are at the same altitude). An endpoint that is airborne over that terrain needs no special case: the 100 m scale height suppresses the surface term on its own, so a level air-to-air path more than a few hundred metres above the site carries no surface term at all. Consumed today ONLY by atmosphere.cn2_profile = 'hufnagel_valley'; a 'tabulated' Cn2 profile is taken as given against MSL and is not shifted, and 'direct' has no profile at all. A non-zero value set against either of those is inert and emits a UserWarning naming why rather than being dropped silently. The default 0 places the site at mean sea level.
geometry.site_latitude_rad	float	0.0	rad	(-1.5708, 1.5708)	Target geodetic latitude [rad], north positive — mode S3 entry (site + time solar geometry). Combined with geometry.day_of_year and geometry.local_solar_time_h (or geometry.ltan_h) to derive the solar zenith angle. Unused unless an S3 parameter is explicitly set.
geometry.solar_azimuth_rad	float	0.0	rad	(-6.2832, 6.2832)	Sun-to-sensor relative azimuth [rad].

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.solar_elevation_rad</code>	float	1.0707963	rad	(0.0, 1.5708)	Sun elevation above the target's local horizon [rad] — mode S2 entry. solar zenith = $\pi/2$ – elevation. Unused unless explicitly set.
<code>geometry.solar_illumination</code>	str	day	—	—	Day/night solar toggle. 'day' (default) illuminates reflective and mixed (T2/T3) targets with the sun at <code>geometry.solar_zenith_rad</code> — the historical behavior, in which the 0.5 rad zenith default meant every T2/T3 scene carried a daytime sun. 'night' removes the solar terms entirely ($\theta_s = \text{None}$: no direct-solar reflection, no single-scatter solar sky) while thermal self-emission and reflected THERMAL downwelling remain — the physically correct nighttime mixed scene. Pure-thermal targets (T1) never carry a solar term either way.
<code>geometry.solar_zenith_rad</code>	float	0.5	rad	(0.0, 3.141592653589793)	Solar zenith angle [rad]. Domain is the full closed interval $[0, \pi]$ $\theta_s > \pi/2$ puts the sun below the *local* horizontal, which a target high enough to clear the terminator shadow can still see. Whether a given point is lit is decided per-altitude by the shadow-height test in <code>radiant.atmosphere.solar_shadow</code> , not by this bound. The pre-Phase-2 bound was 1.5707 rad (a hard sun-above-horizon rule that made sunlit-target-over-dark-ground inexpressible); widening it changes no down-looking result, because every scene that was legal before keeps the same solar column.
<code>geometry.target.projected_area_m2</code>	float	0.0	m2	(0.0, 1000000000000.0)	Projected area of target facing the observer [m²]. 0.0 = not specified (extended-scene default).
<code>geometry.target.shape</code>	str	none	—	—	Geometric primitive for sub-pixel / point-source projected-area computation. 'none' = not specified (back-compat: use <code>geometry.target.projected_area_m2</code>). Explicit values select a concrete shape; dimensional parameters are read from the <code>geometry.target.shape_*</code> scalars per the matrix §3 catalog.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.target.shape_base_radius_m</code>	float	0.0	m	(0.0, 1000000.0)	Base-circle radius [m] for the cone shape. Separate from <code>geometry.target.shape_radius_m</code> because the cone class names the parameter <code>base_radius_m</code> (see <code>shapes/cone.py</code>). Ignored for non-cone shapes.
<code>geometry.target.shape_height_m</code>	float	0.0	m	(0.0, 1000000.0)	Height [m] for shapes with a body-Z extent (box height, cone apex-to-base height). Ignored for sphere, cylinder, flat_plate.
<code>geometry.target.shape_length_m</code>	float	0.0	m	(0.0, 1000000.0)	Length [m] for shapes with a length axis (cylinder axial extent, flat_plate body-X extent, box body-X extent). Ignored for sphere and cone.
<code>geometry.target.shape_pitch_rad</code>	float	0.0	rad	(-6.283185307179586, 6.283185307179586)	Target body pitch [rad] about scene +Y (ZYX Euler).
<code>geometry.target.shape_radius_m</code>	float	0.0	m	(0.0, 1000000.0)	Radius [m] for shapes with a single radius parameter (sphere, cylinder). Must be > 0 when the selected shape requires it; validation is enforced by the Step 1.2 shape factory, not here. Ignored for shapes that do not take a radius (flat_plate, box).
<code>geometry.target.shape_roll_rad</code>	float	0.0	rad	(-6.283185307179586, 6.283185307179586)	Target body roll [rad] about scene +X (ZYX Euler).
<code>geometry.target.shape_width_m</code>	float	0.0	m	(0.0, 1000000.0)	Width [m] for rectangular shapes (flat_plate body-Y extent, box body-Y extent). Ignored for sphere, cylinder, cone.
<code>geometry.target.shape_yaw_rad</code>	float	0.0	rad	(-6.283185307179586, 6.283185307179586)	Target body yaw [rad] about scene +Z (ZYX Euler). Applied to the selected TargetShape's <code>orientation_rad</code> tuple in Step 1.2.
<code>geometry.target_altitude_m</code>	float	0.0	m	(0.0, 100000000.0)	Target altitude above mean sea level [m].
<code>geometry.target_climb_rad</code>	float	0.0	rad	(-1.5707963267948966, 1.5707963267948966)	Target climb angle [rad] — mode K2 entry. Elevation of the target's velocity above its own local horizontal plane, positive up: 0 = level flight, +pi/2 = vertical ascent, -pi/2 = vertical descent. Combined with <code>geometry.target_speed_m_s</code> and <code>geometry.target_heading_rad</code> into the target velocity vector. Unused unless explicitly set.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>geometry.target_heading_rad</code>	float	0.0	rad	(-6.283185307179586, 6.283185307179586)	Target heading [rad] — mode K2 entry. Azimuth of the target's horizontal velocity in the TARGET'S local horizontal plane, measured from the observer's ground azimuth: the same zero <code>geometry.solar_azimuth_rad / delta_phi</code> is measured from ($\text{delta_phi} = \text{phi_s} - \text{phi_o}$ with $\text{phi_o} = 0$), and the same rotational sense (counter-clockwise seen from above). 0 = the target moves horizontally TOWARD the sensor's ground point; π = directly away; $+\pi/2$ = crossing, in the platform's ground-track direction (RADIANT models the platform track as cross-track to the LOS azimuth plane — see <code>radiant.geometry.los_rate</code> for that simplification). DEGENERATE for a radial line of sight ($\text{path_zenith_rad} = 0$ or π , i.e. the sensor or the target straight overhead): the zero azimuth is the sensor's ground point, which does not exist at zero ground range, and every heading then gives the same LOS rate. Any value is equally correct there (0 is conventional) — see <code>radiant.geometry.los_rate</code> 'Degenerate azimuth at a radial LOS'. Unused unless explicitly set.
<code>geometry.target_range_m</code>	float	0.0	m	(0.0, 1000000000000.0)	Observer-to-target slant range [m]. 0.0 = not specified (extended-scene default).
<code>geometry.target_speed_m_s</code>	float	0.0	m/s	(0.0, 1000000.0)	Target speed [m/s] — mode K2 entry (the target-velocity door). Magnitude only; direction is <code>geometry.target_heading_rad</code> (azimuth) and <code>geometry.target_climb_rad</code> (elevation). 0 = a stationary target, which is the default and reduces the published LOS rate exactly to the platform-only value. Combined with the platform ground-track velocity into the relative LOS rate by <code>radiant.geometry.los_rate</code> .

6.2 source

Parameter	Type	Default	Input Unit	Bounds	Description
source.background.emissivity	float	0.95	—	(0.0, 1.0)	Background surface emissivity for sub-pixel regime.
source.background.emissivity_path	str		—	—	Two-column CSV (wavelength_um, emissivity) giving a measured background emissivity spectrum $\epsilon_g(\lambda)$ for the sub-pixel/point-source GroundBackground. Loaded by the API layer before chain execution and resampled onto the chain grid. Takes precedence over source.background.material.
source.background.material	str	grey	—	—	Named spectral-library material for the sub-pixel/point-source GroundBackground emissivity $\epsilon_g(\lambda)$. ‘grey’ (default) uses the scalar source.background.emissivity as a flat spectrum — the back-compat path. Any other name is resolved against radiant.data.SpectralLibrary (vegetation_green, snow, soil_dry, asphalt, ...) by the API layer before chain execution; unknown names are rejected with the legal vocabulary. source.background.emissivity_path overrides this when set.
source.background.temperature	float	290.0	K	(0.0, 5000.0)	Background surface temperature [K] for sub-pixel regime.
source.contrast_reference.emissivity	float	0.95	—	(0.0, 1.0)	Emissivity of the extended contrast-reference scene. Only used when source.contrast_reference.temperature > 0.
source.contrast_reference.temperature	float	0.0	K	(0.0, 5000.0)	Temperature [K] of the reference (background) scene in the neighbouring extended pixel, used only for the extended contrast_snr differential. 0 = no contrast reference (default). Never enters the noise budget.
source.lab_test_mode	str		—	—	Positive dark/lit assertion for the ground_test / lab_test sub-cases. ‘dark’ declares a no-external-illumination configuration (no lamp, no solar — thermal self-emission only, the D-lab dark-cal sub-mode) and is VALIDATED: a user-set source.target.reflectance contradicts it and is rejected. ‘lit’ positively asserts an externally illuminated lab scene (recorded for readability; unvalidated until a lamp surface exists). Empty string = unasserted (back-compat).

Parameter	Type	Default	Input Unit	Bounds	Description
source.no_atmosphere_subcase	str		—	—	Matrix §3.3 sub-case selector for target_location='no_atmosphere'. Empty string = not set (paired with target_location != 'no_atmosphere'). Allowed explicit values: 'space', 'ground_test', 'lab_test'.
source.regime_override	str	auto	—	—	Force regime classification. 'auto' = use detection rule. 'extended', 'point_source', 'sub_pixel' = force that regime.
source.scene_type	str	auto	—	—	Matrix §3.2 scene-type axis. 'auto' = infer from fill_fraction / geometry (default). 'extended' = fills the pixel; 'sub_pixel' = partial fill; 'point_source' = angular size $\ll$ IFOV.
source.target.albedo	float	0.0	—	(0.0, 1.0)	User-facing alias for source.target.reflectance with identical semantics (Lambertian ρ , 0–1). Accepted for scenarios where 'albedo' is the natural label; the inferrer rejects pairing with source.target.reflectance (pick one).
source.target.albedo_path	str		—	—	Alias of source.target.reflectance_path with identical CSV format. Rejected when paired with the reflectance_path surface.
source.target.brightness_temperature_K	float	0.0	K	(0.0, 10000.0)	Scalar brightness temperature T _B [K] — the equivalent-blackbody temperature that produces the same spectral radiance as the source. When user-set (provenance != DEFAULT), routes through the S11 converter to a T1Thermal descriptor with $\epsilon \equiv 1$. Mutually exclusive with source.target.brightness_temperature_path.
source.target.brightness_temperature_path	str		—	—	Path to a 2-column CSV (wavelength_um, T _B _K) carrying a wavelength-dependent brightness temperature. When set, routes through the S11 converter; λ -varying T _B emits T6TabulatedAtSource with $L_{\text{source}} = B(\lambda, T_B(\lambda))$. Mutually exclusive with source.target.brightness_temperature_K.
source.target.emissivity	float	0.95	—	(0.0, 1.0)	Scalar target emissivity used when no spectral emissivity table is supplied. Graybody approximation: $\epsilon(\lambda) = \text{const}$.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>source.target.emissivity_path</code>	str		—	—	Path to a 2-column CSV (wavelength_um, emissivity) carrying a λ -dependent emissivity $\epsilon(\lambda)$ for a thermal target. When set, the inferer builds the thermal descriptor with $L_t(\lambda) = \epsilon(\lambda) \cdot B(\lambda, \text{source.target.temperature})$ instead of a grey ϵ . Mutually exclusive with the scalar <code>source.target.emissivity</code> and with every reflective / radiance / brightness-temperature surface.
<code>source.target.fill_fraction</code>	float	1.0	—	(0.0, 1.0)	Target fill fraction within the pixel. 1.0 = extended scene (default). Values in (0, 1) activate the sub-pixel regime.
<code>source.target.is_hot_target</code>	bool	False	—	—	Hot-target opt-out for MWIR routing. Per matrix §3.2 the legacy scalar- ϵ surface defaults MWIR scenes to T3Mixed (Kirchhoff emit+reflect) because ambient MWIR scenes are reflective-relevant. Set true for $\rho \approx 0$ hot-target scenes (engine plumes, missile signatures, calibration sources) where self-emission dominates and the legacy T1Thermal pure-emit treatment is the correct physics. Ignored for non-MWIR wavelength grids.
<code>source.target.point_intensity_area_m2</code>	float	0.0	m ²	(0.0, 1000000000000.0)	Projected emitting area [m ²] of a blackbody point source — the A in $I(\lambda) = \epsilon \cdot A \cdot B(\lambda, T)$. Distinct from <code>geometry.target.projected_area_m2</code> (which sizes a resolved/sub-pixel target); this one only scales the point-source intensity.
<code>source.target.point_intensity_band_W_per_sr</code>	float	0.0	W/sr	(0.0, 1000000000000.0)	Scalar band-integrated radiant intensity [W/sr] of a point source — the in-band integral $\int I(\lambda) d\lambda$ over the filter band [<code>spectral_integration.filter_min_um</code> , <code>filter_max_um</code>]. Modeled as a spectrally flat intensity $I(\lambda) = \text{value} / (\text{filter_max} - \text{filter_min})$ inside the band, zero outside, so the band integral recovers the specified value. The simplest point-source input (star-tracker / SDA when only a band flux is known); requires <code>scene_type='point_source'</code> . Mutually exclusive with the blackbody point-intensity params and the CSV intensity path.

Parameter	Type	Default	Input Unit	Bounds	Description
source.target.point_intensity_emissivity	float	1.0	—	(0.0, 1.0)	Scalar emissivity $\epsilon \in [0, 1]$ of a blackbody point source (the ϵ in $I(\lambda) = \epsilon \cdot A \cdot B(\lambda, T)$). An independent material property: the Kirchhoff-derived emissivity rule binds optical elements, not scene targets.
source.target.point_intensity_temperature_K	float	0.0	K	(0.0, 5000.0)	Point-source emitter temperature [K]. With point_intensity_area_m2 (and point_intensity_emissivity) defines a blackbody radiant intensity $I(\lambda) = \epsilon \cdot A \cdot B(\lambda, T)$ [W/sr/ μm] for an unresolved target (SDA thermal object, S10 convenience — no surface radiance $\times$ area needed). Requires scene_type='point_source'.
source.target.radiance_temperature_K	float	0.0	K	(0.0, 10000.0)	Band-averaged radiance temperature T_R [K] — the scalar equivalent-blackbody temperature that matches the in-band integrated radiance of the target over $[\lambda_{lo}, \lambda_{hi}]$. Must be paired with source.target.radiance_temperature_band_lo_um and source.target.radiance_temperature_band_hi_um when user-set. Mutually exclusive with S11 brightness_temperature parameters.
source.target.radiance_temperature_band_hi_um	float	0.0	um	(0.0, 1000.0)	Upper band edge [μm] for the S12 radiance-temperature specification. Required when source.target.radiance_temperature_K is user-set; must satisfy $\lambda_{lo} < \lambda_{hi}$.
source.target.radiance_temperature_band_lo_um	float	0.0	um	(0.0, 1000.0)	Lower band edge [μm] for the S12 radiance-temperature specification. Required when source.target.radiance_temperature_K is user-set; must satisfy $\lambda_{lo} < \lambda_{hi}$.
source.target.reflectance	float	0.0	—	(0.0, 1.0)	Scalar target reflectance ρ [dimensionless, 0–1] — the fraction of incident radiance that is reflected from the target. When user-set (provenance != DEFAULT), routes through the Step 3.2 inferrer to a T2Reflective descriptor with $\rho(\lambda) \equiv$ this scalar. Mutually exclusive with source.target.albedo, source.target.reflectance_path, and the legacy (ϵ , T) surface.
source.target.reflectance_path	str	—	—	—	Path to a 2-column CSV (wavelength_um, rho) carrying a λ -dependent reflectance $\rho(\lambda)$. When set, routes through the Step 3.2 inferrer to a T2Reflective descriptor (S5/S6). Mutually exclusive with the scalar reflectance / albedo surfaces.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>source.target.temperature</code>	float	300.0	K	(0.0, 5000.0)	Target surface kinetic temperature (blackbody / graybody).
<code>source.target.user_intensity_path</code>	str		—	—	Path to a 2-column CSV (wavelength_um, I_t_source [W/sr/μm]) carrying a user-supplied spectral intensity at the target plane, for unresolved (point-source) targets. When set, routes through the Phase 5 inferer to a T7IntensityAtSource descriptor (S10 — no physical model applied, the user owns the physics). Mutually exclusive with every other target spec form ((ε, T), reflectance/albedo, brightness_temperature, radiance_temperature, user_radiance). Requires scene_type='point_source'.
<code>source.target.user_radiance_path</code>	str		—	—	Path to a 2-column CSV (wavelength_um, L_t_source [W/m²/sr/μm]) carrying a user-supplied spectral radiance at the target plane. When set, routes through the Phase 4 inferer to a T6TabulatedAtSource descriptor (S8 — no physical model applied; the user owns the physics). Mutually exclusive with every other target spec form ((ε, T), reflectance/albedo, brightness_temperature, radiance_temperature).
<code>source.target_location</code>	str	auto	—	—	Matrix §3.2 target-location axis. 'auto' = infer from atmosphere.model (default). Allowed explicit values: 'at_aperture', 'terrestrial', 'airborne', 'no_atmosphere'.

6.3 atmosphere

Parameter	Type	Default	Input Unit	Bounds	Description
<code>atmosphere.aerosol_type</code>	str	rural	—	—	Aerosol type label. Selects an Ångström exponent and single-scatter albedo: rural (α=1.3), urban (α=1.5), maritime (α=0.7).
<code>atmosphere.cn2_hv_ground_strength</code>	float	1.7e-14	m ^(-2/3)	(0.0, 1e-10)	Hufnagel-Valley ground-level structure constant A [m ^(-2/3)]. Scales the 100 m-scale-height surface layer of Cn²(h). Used only when atmosphere.cn2_profile = 'hufnagel_valley'.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>atmosphere.cn2_hv_wind_rms_m_s</code>	float	21.0	m/s	(0.0, 100.0)	Hufnagel-Valley RMS upper-atmosphere (5–20 km) wind speed w [m/s]. Scales the jet-stream term of $Cn^2(h)$. Used only when <code>atmosphere.cn2_profile = 'hufnagel_valley'</code> .
<code>atmosphere.cn2_profile</code>	str	direct	—	—	Optical-turbulence profile $Cn^2(h)$ used to derive the Fried parameter for turbulence. 'direct' (default) uses <code>atmosphere.r0_m</code> as given, with no profile at all. 'hufnagel_valley' integrates the analytic HV profile along the line of sight (parameters <code>cn2_hv_wind_rms_m_s</code> and <code>cn2_hv_ground_strength</code> ; the defaults are HV-5/7). 'tabulated' uses the two-column CSV named by <code>atmosphere.cn2_tabulated_file</code> .
<code>atmosphere.cn2_tabulated_file</code>	str		—	—	Two-column CSV 'altitude_m,cn2_m^-2/3' (ascending altitude, '#' comments allowed) defining a measured or externally-modelled $Cn^2(h)$ profile. Required when <code>atmosphere.cn2_profile = 'tabulated'</code> . Read before chain execution and injected at <code>stage_outputs['atmosphere_config']['cn2_profile']</code> .

Parameter	Type	Default	Input Unit	Bounds	Description
<code>atmosphere.interpolated_data_dir</code>	str		—	—	Directory containing pre-computed atmosphere runs (NPZ files) at discrete geometry points for the interpolated model. Empty (the default) uses the bundled atmosphere library family matching the scene's line-of-sight direction and <code>atmosphere.interpolation_axes</code> : down-looking axes= <code>'path_zenith_rad'</code> → <code>us_standard_zenith_fan</code> , down-looking axes= <code>'sensor_altitude_m'</code> → <code>midlat_summer_sensor_ladder</code> , down-looking axes= <code>'sensor_altitude_m,path_zenith_rad'</code> → <code>midlat_summer_upwelling_offnadir</code> , down-looking axes= <code>'sensor_altitude_m,target_altitude_m'</code> → <code>midlat_summer_ladders</code> , down-looking axes= <code>'sensor_altitude_m,target_altitude_m,path_zenith_rad'</code> → <code>midlat_summer_boost_offnadir</code> , up-looking axes= <code>'sensor_altitude_m'</code> → <code>midlat_summer_uplooking_sensor_ladder</code> , up-looking axes= <code>'target_altitude_m'</code> → <code>midlat_summer_uplooking_ladder</code> , up-looking axes= <code>'target_altitude_m,path_zenith_rad'</code> → <code>midlat_summer_uplooking_zenith_fan</code> . Any other (direction, axes) combination requires an explicit directory. Per-family coverage limits and the MODTRAN profile each was rendered with: <code>radiant.api.shipped_atmosphere_families()</code> .
<code>atmosphere.interpolation_axes</code>	str	<code>path_zenith_rad</code>	—	—	Comma-separated list of geometry fields to interpolate over, e.g. <code>'path_zenith_rad'</code> or <code>'path_zenith_rad,sensor_altitude_m'</code> .
<code>atmosphere.interpolation_method</code>	str	linear	—	—	Interpolation method: <code>'linear'</code> or <code>'nearest'</code> .
<code>atmosphere.model</code>	str	simple	—	—	Atmosphere model selector. <code>'simple'</code> uses a closed-form Beer-Lambert; <code>'exo'</code> uses a vacuum; <code>'tabulated'</code> loads from files; <code>'modtran'</code> wraps the MODTRAN binary; <code>'interpolated'</code> interpolates between pre-computed runs at discrete geometry points.
<code>atmosphere.modtran.aerosol_model</code>	str	rural	—	—	Aerosol model for MODTRAN (maps to IHAZE card).

Parameter	Type	Default	Input Unit	Bounds	Description
atmosphere.modtran.allow_fallback	bool	False	—	—	If True and MODTRAN binary is unavailable, fall back to SimpleAtmosphere with translated parameters.
atmosphere.modtran.atmosphere_profile	str	us_standard	—	—	Standard atmosphere profile for MODTRAN (maps to MODEL card). Same enum as atmosphere.standard_atmosphere.
atmosphere.modtran.binary_path	str		—	—	Path to the MODTRAN executable. Empty (the default) resolves at run time: 'modtran' on PATH if present, else the platform's conventional install location.
atmosphere.modtran.cache_dir	str	~/.radiant/modtran_cache	—	—	Directory for caching MODTRAN tape7 results. Keyed by SHA-256 hash of the rendered tape5 deck.
atmosphere.modtran.flux_path	str		—	—	Optional MODTRAN spectral flux CSV (a Block E irradiance run's *_flux.csv sidecar) supplying the downwelling sky irradiance for the tape7-import path. Requires atmosphere.modtran.tape7_path. When set, the ground-level DOWN column (thermal emission + scattered solar) feeds the sky-reflection terms: E_sky_scattered from the reflective-solar band and E_sky_thermal from the thermal band — replacing the zero downwelling a flux-less import would carry. Unset, a standard IEMSCT=2 tape7 has no downwelling column, so both terms stay zero.
atmosphere.modtran.h2o_scale	float	1.0	—	(0.01, 10.0)	Water vapor column scaling factor for MODTRAN.
atmosphere.modtran.o3_scale	float	1.0	—	(0.01, 10.0)	Ozone column scaling factor for MODTRAN.
atmosphere.modtran.spectral_resolution_cm1	float	1.0	cm-1	(0.1, 100.0)	Spectral resolution [cm-1] for the MODTRAN computation.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>atmosphere.modtran.tape7_path</code>	str		—	—	Path to a MODTRAN tape7 output file produced elsewhere. When set (with <code>atmosphere.model='modtran'</code>), the atmospheric state is built from this file — parsed before chain execution — and the MODTRAN binary, cache, and fallback are never consulted. Unset (empty) leaves the binary/cache/fallback behavior unchanged. Like tabulated files, an imported tape7 is geometry-agnostic: the arrays are served as-is for any query geometry. Airborne targets (<code>h_tgt > 0</code>) additionally require the <code>target→sensor</code> leg via <code>atmosphere.modtran.tape7_up_path</code> ; with only this single (ground→sensor) file they are rejected.
<code>atmosphere.modtran.tape7_sun_path</code>	str		—	—	Optional sun-leg tape7 file for the two-leg split (file flavor). Requires <code>atmosphere.modtran.tape7_path</code> . When set, <code>tau_sun</code> (the sun→target down-leg transmittance) comes from this file's TOT TRANS column — a MODTRAN run along the solar-zenith slant path — instead of aliasing the up-leg transmittance, and the single-tau collapse warning is not emitted. Unset, <code>tau_sun</code> aliases <code>tau_up</code> with a <code>UserWarning</code> , as before.
<code>atmosphere.modtran.tape7_up_path</code>	str		—	—	Optional <code>target→sensor</code> up-leg tape7 file for airborne targets (file flavor). Requires <code>atmosphere.modtran.tape7_path</code> (which then supplies the ground→sensor full column the background branch needs). When set, <code>tau_up</code> and <code>L_path_up</code> (the <code>target→sensor</code> partial column) come from this file — a MODTRAN run with <code>H2</code> = the target altitude — enabling <code>h_tgt > 0</code> on the file-import path. The user owns geometry consistency: nothing in the file records <code>H2</code> , so this file must actually be the run matching the scenario's target altitude and LOS zenith. Unset, airborne targets are rejected on the file-import path.
<code>atmosphere.precipitable_water_cm</code>	float	1.4	cm	(0.0, 10.0)	Total column precipitable water in centimetres of liquid-equivalent water vapour. Drives the 5-band water-vapor extinction fit.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>atmosphere.r0_m</code>	float	0.0	m	(0.0, 10.0)	Fried coherence diameter r_0 [m] at the operating wavelength. Controls long-exposure Kolmogorov turbulence MTF. Set to 0 to disable turbulence effects.
<code>atmosphere.r0_reference_wavelength_um</code>	float	0.0	um	(0.0, 100.0)	Wavelength [um] that <code>atmosphere.r0_m</code> is quoted at. Seeing is habitually reported at 0.5 um, but r_0 is strongly wavelength-dependent ($r_0 \propto \lambda^{(6/5)}$), so an r_0 entered at 500 nm and applied at 4 um is ~8x too small — a silent order-of-magnitude error in the turbulence MTF. When set, <code>r0_resolution</code> rescales the entered value to the band centre by $(\lambda_{\text{band}} / \lambda_{\text{ref}})^{(6/5)}$. The default 0.0 means ‘the value I entered is already at the operating wavelength’, which rescales nothing. Ignored unless <code>atmosphere.r0_m</code> is set directly (a C_n^2 profile derives r_0 at the band centre already).
<code>atmosphere.standard_atmosphere</code>	str	us_standard	—	—	Standard atmosphere profile selector. Used by the simple model for the path-mean temperature lookup and aerosol/H2O scale heights.
<code>atmosphere.tabulated_downwelling_file</code>	str		—	—	Path to CSV file containing tabulated downwelling atmospheric emission [W/m ² /sr/um]. Optional; defaults to zeros if not provided.
<code>atmosphere.tabulated_path_radiance_file</code>	str		—	—	Path to CSV or NPZ file containing tabulated path radiance [W/m ² /sr/um]. Required when <code>atmosphere.model</code> =‘tabulated’.
<code>atmosphere.tabulated_transmittance_file</code>	str		—	—	Path to CSV or NPZ file containing tabulated atmospheric transmittance. CSV: two columns (wavelength_um, value). NPZ: key ‘transmittance’ with matching ‘wavelength_um’. Required when <code>atmosphere.model</code> =‘tabulated’.
<code>atmosphere.turbulence_wave_type</code>	str	plane	—	—	Path weighting for the profile-driven Fried parameter. ‘plane’ (default) weights every slab equally — the source is effectively at infinity, which is the imaging case and the convention published r_0 values use. ‘spherical’ weights each slab by (fractional distance from the target) ^(5/3) , peaking at the aperture — the finite-range point-source case. Ignored when <code>atmosphere.cn2_profile</code> = ‘direct’.

Parameter	Type	Default	Input Unit	Bounds	Description
atmosphere.visibility_km	float	23.0	km	(0.1, 500.0)	Meteorological visibility at 550 nm in kilometres. Drives the Koschmieder aerosol extinction $\sigma_{\text{aer}}(550 \text{ nm}) = 3.912 / V_{\text{km}}$.

6.4 optics

Parameter	Type	Default	Input Unit	Bounds	Description
optics.aperture_diameter_m	float	required	m	(0.0001, 20.0)	Clear entrance-pupil diameter of the primary [m].
optics.cold_stop_obscuratio_ratio	float	0.0	—	(0.0, 0.99)	Central obscuration ratio [-] imposed by the cold shield itself (D_blocked / D_eff). The effective obscuration is the LARGER of this and optics.obscuration_ratio, so a cold shield that blocks more than the secondary governs. Default 0 = the cold shield adds no obscuration beyond the telescope's own.
optics.cold_stop_undersize_frac	float	0.0	—	(0.0, 0.49)	Fractional reduction of the pupil DIAMETER imposed by the cold stop [-]: $D_{\text{eff}} = (1 - u) * \text{aperture_diameter_m}$. The cold stop is the aperture stop, undersized for alignment and thermal tolerancing, so the effective pupil sets the collecting area, the working f/#, the diffraction PSF/MTF, and the near-field acceptance cone together. $u = 0.05$ (a 5 % undersized stop) collects $(1 - 0.05)^2 = 90 \%$ of the light. 0 = the cold stop matches the primary (the default: no undersizing modelled).
optics.defocus_um	float	0.0	um	(-500.0, 500.0)	Linear defocus: displacement of the detector plane from best focus [μm]. Positive = behind focus, negative = in front. Both produce identical blur (absolute value used). Zero = no defocus.
optics.f_number	float	required	—	(0.3, 200.0)	Dimensionless f/# = focal_length_m / aperture_diameter_m. Part of the {D, f, f/#} consistency group; supply any two and the third is derived.
optics.field_position_x	float	0.0	deg	(-10.0, 10.0)	Normalized cross-track field coordinate for field-dependent WFE evaluation. 0.0 = on-axis. Maps to field_x_deg via the field table.
optics.field_position_y	float	0.0	deg	(-10.0, 10.0)	Normalized along-track field coordinate for field-dependent WFE evaluation. 0.0 = on-axis. Maps to field_y_deg via the field table.

Parameter	Type	Default	Input Unit	Bounds	Description
optics.focal_length_m	float	required	m	(0.0001, 100.0)	Effective focal length of the telescope [m].
optics.n_spiders	int	0	—	(0, 12)	Number of secondary-support spider arms (radial struts). Default 0 (no struts). A 4-arm spider produces the familiar four-point diffraction spike. See RADIANT_Optics.md §3.3.
optics.nearfield_enabled	int	1	—	(0, 1)	Enable nearfield (warm-optics) emission calculation. Set to 0 to disable (int: 1=True, 0=False).
optics.obscuration_ratio	float	0.0	—	(0.0, 0.99)	Central obscuration ratio $D_{\text{secondary}} / D_{\text{primary}}$. Defaults to 0 (unobscured). Must satisfy $0 \leq \epsilon < 1$.
optics.psf_n_wavelengths	int	1	—	(1, 101)	Number of wavelengths for polychromatic PSF computation. 1 = monochromatic at band center (default). Values > 1 compute a photon-flux-weighted average of monochromatic PSFs across the spectral band.
optics.psf_oversample	int	8	—	(4, 16)	Focal-plane PSF samples per detector pixel. Sets the PSF grid spacing to $\text{pixel_pitch} / \text{psf_oversample}$; the padded FFT size grows to match. Larger values sharpen spatial-metric discretization (EE_box, RER, FWHM) at higher FFT cost. The schema floor is 4, above compute_sampling's Nyquist floor of 2: at $\text{oversample} \leq 3$ the padded grid can land at exactly $2\times$ the pupil width and the FFT-of-PSF path aliases at the grid edge, breaching the tolerance the two spatial paths must agree to (measured 0.032 against 0.02 on the reference MWIR config).
optics.pupil_npix	int	128	—	(32, 512)	Side length of the square pupil grid before FFT padding, in samples. Sets the resolution of the complex pupil that BOTH spatial paths derive from: the PSF (FT of the pupil) and the optical MTF (pupil autocorrelation). Larger values resolve finer aperture structure (thin spider vanes, small obscurations) at quadratically higher FFT cost — this is the dominant cost of a chain evaluation of a chain evaluation.
optics.scatter_halo_sigma_um	float	100.0	um	(0.1, 10000.0)	Focal-plane sigma of the Gaussian scatter halo [um] used by the TIS model. Sets where the scattered fraction lands; tune to a measured halo when available. Only meaningful when optics.surface_roughness_nm > 0.

Parameter	Type	Default	Input Unit	Bounds	Description
optics.spider_angle_deg	float	0.0	deg	(0.0, 360.0)	Orientation of the first spider arm about the optical axis [deg]; remaining arms equally spaced. Default 0 (first arm along +x).
optics.spider_width_m	float	0.0	m	(0.0, 1.0)	Width of each spider arm [m]. Converted to a fraction of the pupil diameter for the mask; also subtracted from the radiometric clear area (RADIANT_Optics.md §3.3). Default 0. Active only when n_spiders > 0.
optics.stray.absolute_irradiance_W_m2	float	0.0	W/m ²	(0.0, 1000000.0)	Absolute in-band stray irradiance at the FPA [W/m ²]. Distributed flat across the wavelength grid.
optics.stray.halo_sigma_um	float	50.0	um	(0.1, 1000.0)	Gaussian half-width of the veiling-glare halo on the focal plane [μm]. Must be small enough to fit the PSF grid for the kernel and analytic MTF term to stay exact Fourier pairs (the kernel is truncated at the grid edge).
optics.stray.includes_thermal	int	0	—	(0, 1)	If 1, the stray light measurement already includes warm-optics scatter; nearfield is suppressed to avoid double-counting. (int: 1=True, 0=False).
optics.stray.input_mode	str	veiling_glare	—	—	Stray light input mode: veiling_glare, absolute_irradiance, or spectral_file (curve injected via stage_outputs['optics_config']['stray_light_spectral']). pst_file is not offered: PST-based stray light needs a scene radiance distribution RADIANT v1 does not model (the mode is not offered rather than offered and always raising).
optics.stray.veiling_glare_fraction	float	0.0	—	(0.0, 1.0)	Veiling glare fraction: fraction of in-FOV scene irradiance that becomes stray light [0, 1].
optics.stray.veiling_glare_mtf	int	0	—	(0, 1)	Enable the SPATIAL veiling-glare model: the veiling-glare fraction is re-imaged as a Gaussian halo, entering the PSF path as a kernel $(1-vgf) \cdot \delta + vgf \cdot G(\sigma_{halo})$ and the MTF product path as its exact Fourier pair $(1-vgf) + vgf \cdot \exp(-2\pi^2 \sigma^2 f^2)$ — the low-frequency contrast-modulation loss the radiometric pedestal cannot express. 0 (default) = pedestal-only (historical behavior); 1 = halo model active when veiling_glare_fraction > 0. (int: 1=True, 0=False).

Parameter	Type	Default	Input Unit	Bounds	Description
optics.surface_roughness_nm	float	0.0	nm	(0.0, 10000.0)	Effective RMS surface micro-roughness of the optical train [nm] for the TIS scatter model: $TIS = 1 - \exp(-(4\pi\sigma/\lambda)^2)$ at band center. Zero (default) = no scatter. Smooth-surface limit — a warning fires when $TIS > 0.3$. Scattered energy lands in a Gaussian halo of width <code>optics.scatter_halo_sigma_um</code> , entering both spatial paths: a kernel on the PSF path and an analytic MTF term, an exact Fourier pair.
optics.transmission_input_mode	str	scalar	—	—	Which of the five transmission input modes to use: scalar, spectral_file, telescope_plus_filters, key_elements, full_prescription. Non-scalar modes read their curves/elements from pre-chain injections under <code>stage_outputs['optics_config']</code> (transmission_spectral; telescope_transmission + filter_specs; key_elements + residual_transmission; element_list) — e.g. via <code>Sensor.evaluate(extra_stage_outputs=...)</code> .
optics.transmission_scalar	float	0.7	—	(0.0, 1.0)	Flat broadband optical throughput τ_{opt} (Mode 1 of <code>RADIANT_Optics.md</code> §5.1). Dimensionless in [0, 1].
optics.wfe_mode	str	scalar_rms	—	—	Wavefront error input mode: scalar_rms (parameter-driven), or zernike / field_dependent (WavefrontError object injected via <code>stage_outputs['optics_config']['wavefront_error']</code>). <code>opd_map</code> is not offered: OPD maps have no pupil-phase representation in v1 (the mode is not offered rather than offered and always raising).
optics.wfe_reference_wavelength_um	float	0.633	um	(0.1, 30.0)	Reference wavelength at which the WFE is specified [um]. HeNe 0.633 um is the standard interferometry wavelength.
optics.wfe_rms_waves	float	0.0	waves	(0.0, 2.0)	RMS wavefront error in waves at the reference wavelength (scalar_rms mode).

Parameter	Type	Default	Input Unit	Bounds	Description
optics.zernike_file	str		—	—	Path to a Zemax ‘Zernike Standard Coefficients’ text export. When set, the API layer loads it pre-chain and injects the resulting ZERNIKE-mode WavefrontError via stage_outputs[‘optics_config’][‘wavefront_error’], which supersedes wfe_mode/wfe_rms_waves. The report’s own reference wavelength is honored; optics.wfe_reference_wavelength_um is the fallback when the export has no Wavelength header. Empty = disabled (scalar/parameter-driven WFE).

6.5 platform

Parameter	Type	Default	Input Unit	Bounds	Description
platform.ground_velocity_m_s	float	0.0	m/s	(0.0, 50000.0)	Platform along-track ground velocity [m/s]. For LEO at 600 km altitude: ~6900 m/s. Set to 0 to disable velocity-based smear.
platform.jitter_axes	str	isotropic	—	—	Jitter mode: ‘isotropic’ uses jitter_rms_urad for both axes, ‘anisotropic’ uses separate x/y values.
platform.jitter_rms_urad	float	0.0	urad	(0.0, 1000000.0)	Isotropic jitter RMS [μrad]. Applied equally to both axes. Set to 0 to disable jitter.
platform.jitter_rms_x_urad	float	0.0	urad	(0.0, 1000000.0)	Cross-track jitter RMS [μrad]. Only used when jitter_axes=‘anisotropic’.
platform.jitter_rms_y_urad	float	0.0	urad	(0.0, 1000000.0)	Along-track jitter RMS [μrad]. Only used when jitter_axes=‘anisotropic’.
platform.smear_length_um	float	0.0	um	(0.0, 1000.0)	Direct focal-plane smear input [μm]. Bypasses the velocity/altitude computation. If set (> 0), takes precedence over ground_velocity_m_s. Set to 0 to use velocity-based computation instead.

6.6 spectral_integration

Parameter	Type	Default	Input Unit	Bounds	Description
spectral_integration.filter_max_um	float	required	um	(0.1, 30.0)	Long-wavelength edge of the filter bandpass [μm].
spectral_integration.filter_min_um	float	required	um	(0.1, 30.0)	Short-wavelength edge of the filter bandpass [μm].
spectral_integration.integration_time_s	float	required	s	(1e-09, 100.0)	Detector integration time [s].

6.7 detector

Parameter	Type	Default	Input Unit	Bounds	Description
detector.charge_diffusion_length_m	float	0.0	m	(0.0, 0.001)	RMS charge diffusion length [m]. Zero disables diffusion MTF.
detector.clutter_sigma	float	0.0	—	(0.0, 1.0)	Scene clutter coefficient (fractional). Zero disables.
detector.dark_activation_energy_eV	float	0.0	eV	(0.0, 5.0)	Arrhenius activation energy for dark-rate temperature scaling. Zero disables scaling.
detector.dark_rate_e_per_s	float	100.0	1/s	(0.0, 1000000000.0)	Dark current generation rate per pixel [e-/s].
detector.dark_reference_temperature_K	float	77.0	K	(1.0, 500.0)	Temperature at which dark_rate_e_per_s is specified [K].
detector.detector_temperature_K	float	77.0	K	(1.0, 500.0)	Detector operating temperature [K].
detector.dsnu_e_rms	float	0.0	e-	(0.0, 1000000.0)	Dark-signal non-uniformity [e- RMS]. Zero disables.
detector.fill_factor	float	1.0	—	(0.0, 1.0)	Photosensitive fraction of the pixel cell.
detector.flicker_K	float	0.0	e ⁻²	(0.0, 1000000000000.0)	1/f flicker noise coefficient [e ⁻²]. Zero disables.
detector.flicker_f_high_hz	float	1000000.0	Hz	(0.001, 1000000000.0)	Upper frequency bound for 1/f integration [Hz].
detector.flicker_f_low_hz	float	0.01	Hz	(1e-06, 1000000.0)	Lower frequency bound for 1/f integration [Hz].
detector.glow_e_per_s	float	0.0	1/s	(0.0, 1000000.0)	Detector/ROIC glow rate [e-/s/pixel]. Zero disables.
detector.gr_factor	float	0.0	—	(0.0, 10.0)	G-R noise factor (0 = disabled, 1 = classic HgCdTe).
detector.ipc_coupling	float	0.0	—	(0.0, 0.25)	Inter-pixel capacitance coupling fraction α [0, 0.25).
detector.n_pixels_cross	int	0	—	(0, 1000000)	Number of detector pixels in the cross-track direction.
detector.noise_regime	str	imaging	—	—	Noise regime: ‘imaging’ (temporal only, FPN calibrated out) or ‘detection’ (temporal + spatial).
detector.persistence_fraction	float	0.0	—	(0.0, 1.0)	Fraction of prior-frame signal that persists. Zero disables.
detector.persistence_tau_s	float	1.0	s	(1e-06, 1000.0)	Persistence time constant [s].
detector.pixel_phase_mode	str	average	—	—	Pixel sampling phase (straddle) convention for point-source / sub-pixel EE_box: average (uniform over one pitch — expectation), centered (image on a pixel centre), worst_case (image on a four-pixel corner), specified (pixel_phase_x / pixel_phase_y).

Parameter	Type	Default	Input Unit	Bounds	Description
detector.pixel_phase_x	float	0.0	—	(-0.5, 0.5)	Cross-track offset of the geometric image point from the pixel centre, as a fraction of the pixel pitch (0 = centred, ± 0.5 = pixel edge). Used only when pixel_phase_mode = specified.
detector.pixel_phase_y	float	0.0	—	(-0.5, 0.5)	Along-track offset of the geometric image point from the pixel centre, as a fraction of the pixel pitch (0 = centred, ± 0.5 = pixel edge). Used only when pixel_phase_mode = specified.
detector.pixel_pitch_x_um	float	required	um	(0.1, 1000.0)	Pixel pitch along the cross-track (x) axis.
detector.pixel_pitch_y_um	float	required	um	(0.1, 1000.0)	Pixel pitch along the along-track (y) axis. Required — there is no ‘defaults to x pitch’ fallback; set it explicitly (equal to pixel_pitch_x_um for square pixels).
detector.prior_signal_e	float	0.0	e-	(0.0, 1000000000.0)	Signal electrons from prior frame (for persistence). Zero disables.
detector.pruno_pct	float	0.0	—	(0.0, 100.0)	Photo-response non-uniformity [%]. Zero disables.
detector.qe_material	str		—	—	Named bundled detector QE curve: a material in the bundled detector library (e.g. ‘insb’, ‘hgcdte_mwir’, ‘silicon’ — the API rejects unknown names with the legal vocabulary). Resolved pre-chain by the API layer onto the wavelength grid, QE = 0 past the data span. Precedence: qe_table_path (explicit file) > qe_material (library) > qe_value (scalar). Empty = disabled.
detector.qe_table_path	str		—	—	Path to a wavelength-vs-QE CSV. When set, RadiantSession loads it (io.qe_csv) onto the wavelength grid and applies it spectrally, superseding the scalar qe_value; past-cutoff QE is zero.
detector.qe_temperature_coeff_per_K	float	0.0	1/K	(-0.1, 0.1)	Linear QE temperature coefficient [1/K]. $QE(T) = QE_{base} \cdot (1 + coeff \cdot (detector_temperature_K - qe_temperature_ref_K))$, applied to the scalar qe_value or the qe_table_path curve. Default 0 (temperature-independent QE).
detector.qe_temperature_ref_K	float	300.0	K	(1.0, 1000.0)	Reference temperature [K] at which the QE (qe_value / qe_table_path) was characterised. Only used when qe_temperature_coeff_per_K $\neq$ 0.
detector.qe_value	float	required	—	(0.0, 1.0)	Wavelength-independent scalar quantum efficiency.
detector.r0a_ohm_cm2	float	0.0	ohm_cm2	(0.0, 1000000000000.0)	Detector R _{0A} product [$\Omega \cdot \text{cm}^2$]. Zero disables Johnson noise.

6.8 readout

Parameter	Type	Default	Input Unit	Bounds	Description
readout.adc_bits	int	16	—	(4, 32)	ADC bit depth.
readout.architecture	str	analog_well	—	—	Readout architecture: ‘analog_well’ (charge integration into a full well, existing path) or ‘digital_counting’ (in-pixel comparator + counter with charge-subtraction reset, DROIC/DFPA). Under ‘digital_counting’ the counting parameters below replace full_well_capacity_e; ReadoutStage rejects mixed specifications.
readout.binning_x_offchip	int	1	—	(1, 64)	Off-chip binning factor along x. 1 = no binning.
readout.binning_x_onchip	int	1	—	(1, 64)	On-chip binning factor along x. 1 = no binning.
readout.binning_y_offchip	int	1	—	(1, 64)	Off-chip binning factor along y. 1 = no binning.
readout.binning_y_onchip	int	1	—	(1, 64)	On-chip binning factor along y. 1 = no binning.
readout.cds_enabled	int	1	—	—	Correlated double sampling enabled (1=yes, 0=no).
readout.coadd_mode	str	sum	—	—	Coadd combination mode: ‘sum’, ‘average’, or ‘median’.
readout.count_packet_e	float	0.0	e-	(0.0, 10000000.0)	Charge packet per count [e-/count] for digital_counting: the charge-subtraction quantum removed from the well at each comparator trip. Default 0.0 means ‘unset’; the parameter is required (> 0) when architecture = ‘digital_counting’. Counting-only: rejected under analog_well.
readout.counter_bits	int	16	—	(1, 32)	In-pixel counter bit depth N for digital_counting. Effective well = $2^N \times \text{count_packet_e}$; counter rollover is treated as saturation (clip) in v1. Counting-only: rejected under analog_well.

Parameter	Type	Default	Input Unit	Bounds	Description
readout.counting_mode	str	up	—	—	Digital-counting accumulation mode (plan Phase 4, rulings D1/D6): ‘up’ — unsigned accumulation, counter rollover clips; ‘up_down’ — signed modulo accumulator that increments during the scene phase and decrements during a reference phase (in-pixel background subtraction for dim targets on a bright common background). Counting-only: rejected under analog_well; ‘up_down’ requires architecture = ‘digital_counting’.
readout.electronics_sigma_um	float	0.0	um	(0.0, 100.0)	Electronics MTF: equivalent Gaussian blur sigma on the focal plane [μm] from finite amplifier bandwidth at the pixel clock rate. Blurs the readout (cross-scan, x) axis only. Zero (default) = ideal electronics, no blur. Enters both the EffectivePSF (kernel) and the MTF product (analytic term) per spatial paths agree.
readout.frame_period_s	float	0.0	s	(0.0, 1000000.0)	Frame period [s]: the time between frame starts, stored independently of the integration time (spectral_integration.integration_time_s) per RADIANT_Conventions.md §4. Frame rate = 1/frame_period and duty cycle = t_int/frame_period are derived by radiant.readout.frame_timing and published in stage_outputs[‘readout’]. Default 0.0 means ‘unset’: the frame period defaults to the integration time (frame rate = 1/t_int, duty cycle = 1.0) with a logged warning. A duty cycle > 1 (integration longer than the frame period) is rejected.
readout.full_well_capacity_e	float	100000.0	e-	(100.0, 1000000000000.0)	Full well capacity per pixel [e-].
readout.gain_e_per_dn	float	1.0	e-/DN	(0.001, 1000000.0)	System conversion gain: electrons per digital number (LSB).
readout.max_count_rate_hz	float	0.0	Hz	(0.0, 1000000000000.0)	Comparator dead-time flux ceiling [Hz] for digital_counting: maximum in-pixel count rate. Gives a second saturation bound max_count_rate_hz x t_int x count_packet_e. Default 0.0 means ‘unset’ (no dead-time ceiling; counter rollover governs). Counting-only: rejected under analog_well.
readout.n_coadds	int	1	—	(1, 10000)	Number of coadded frames. 1 = no coadd.
readout.n_tdi	int	1	—	(1, 1000)	Number of TDI stages. 1 = no TDI.

Parameter	Type	Default	Input Unit	Bounds	Description
readout.node_capacitance_F	float	0.0	F	(0.0, 1e-09)	Sense-node capacitance [F]. Zero disables kTC noise.
readout.read_noise_e_rms	float	5.0	e-	(0.0, 10000.0)	Per-frame read noise delivered to the signal path [e- RMS].
readout.reference_integration_s	float	0.0	s	(0.0, 1000000.0)	Down-phase (reference) integration time [s] for counting_mode = 'up_down' (ruling D7: parameterized). Default 0.0 means 'unset': the down phase equals the scene integration time (spectral_integration.integration_time_s), the balanced case whose background and dark means cancel exactly. Meaningful only under 'up_down'.
readout.reference_rate_e_per_s	float	0.0	e-/s	(0.0, 1000000000000000.0)	User-specified down-phase reference charge rate [e-/s] for reference_source = 'user_level' (plan Phase 4). Default 0.0 means 'unset'; required (> 0) when 'user_level' is selected. Meaningful only under counting_mode = 'up_down'.
readout.reference_source	str	background_term	—	—	Down-phase reference flux model for counting_mode = 'up_down' (ruling D6): 'background_term' — the chain's own background radiometric term (valid only in sub-pixel / point-source regimes, where target and background are separate terms); 'user_level' — a user-specified reference charge rate (reference_rate_e_per_s), the extended-scene fallback. Meaningful only under 'up_down'.
readout.residue_readout	bool	True	—	—	Digital_counting only: read the analog residue (sub-packet charge) after the counter word. True: residue passes through the existing ADC model (adc_bits / gain scoped to a count_packet_e full scale) and DN is the combined word. False: DN is the bare counter and quantization noise is count_packet_e/sqrt(12). Counting-only: rejected under analog_well.
readout.tdi_misalign_pixels	float	0.0	—	(0.0, 10.0)	Cross-scan TDI misalignment in pixel units. Zero = perfect alignment.
readout.tdi_mode	str	analog	—	—	TDI readout mode: 'analog' (single readout after charge accumulation) or 'digital' (each stage read independently, summed digitally).

6.9 calibration

Parameter	Type	Default	Input Unit	Bounds	Description
calibration.band_center_uncertainty_um	float	0.0	um	(0.0, 1.0)	Band-center wavelength uncertainty (1-sigma) [um] — spectral-cal or filter drift as a rigid band shift. Becomes a scene-temperature-dependent radiance BIAS term: the calibration absorbs the scale error at its own temperature, so the residual is the scene-vs-cal difference of band-shift log-derivatives — accuracy budget only, never RSS'd into noise. Zero at $T_{\text{scene}} = T_{\text{cal}}$; Wien-side (short-wave) bands are most sensitive.
calibration.cal_flux_high	float	0.0	—	(0.0, 10.0)	Upper cal point as a fraction of the scene signal (cal_point_mode = 'flux_fraction'); 0.0 = unset.
calibration.cal_flux_low	float	0.0	—	(0.0, 10.0)	Lower cal point as a fraction of the scene signal (cal_point_mode = 'flux_fraction'); 0.0 = unset. Must be strictly below cal_flux_high (and cal_flux_mid, three_point).
calibration.cal_flux_mid	float	0.0	—	(0.0, 10.0)	Middle cal point as a fraction of the scene signal (three_point + flux_fraction only); 0.0 = unset.
calibration.cal_path	str	full_aperture	—	—	What the calibration view sees: 'full_aperture' (cal source fills the whole optical path — the NUC corrects every emission source; v1 behavior) or 'internal_shutter' (a flag/shutter inside the train — the NUC never sees the emission of elements in FRONT of it, which returns in operation as an uncorrected offset bias plus narcissus-pattern FPN). Requires an active scheme and shutter_after_element >= 1.

Parameter	Type	Default	Input Unit	Bounds	Description
calibration.cal_point_mode	str	temperature	—	—	How the NUC cal points are declared: ‘temperature’ ($\text{cal_temp_} * K$, mapped to cal signals through the band Planck photon-radiance ratio — the νI form) or ‘flux_fraction’ ($\text{cal_flux} *$ as fractions of the scene signal — the integrating-sphere / flat-field form, the flux-ratio door). Under flux_fraction the temperature-anchored inputs ($\text{cal_temp_} * K$, $\text{source_temp_uncertainty_K}$, $\text{source_uniformity_K}$, $\text{band_center_uncertainty_um}$, $\text{source_emissivity_uncertainty}$) have no anchor and are rejected as over-specification; drift, gain, and the internal-shutter path are temperature-free and flow unchanged.
calibration.cal_temp_high_K	float	0.0	K	(0.0, 3000.0)	Calibration-source temperature of the upper cal point [K]. Required when scheme is ‘two_point’ (must exceed cal_temp_low_K); 0.0 = unset.
calibration.cal_temp_low_K	float	0.0	K	(0.0, 3000.0)	Calibration-source temperature of the (lower) cal point [K]. Required when scheme is ‘one_point’ or ‘two_point’; 0.0 = unset. Cal-point fluxes are derived from cal temperatures through the band — never supplied directly (over-specification guard).
calibration.cal_temp_mid_K	float	0.0	K	(0.0, 1000.0)	Middle calibration-source temperature [K] for the ‘three_point’ scheme. Must satisfy $\text{cal_temp_low_K} < \text{cal_temp_mid_K} < \text{cal_temp_high_K}$; 0.0 = unset (evaluate-time validation when the scheme is active, matching the other cal points’ sentinel).
calibration.gain_drift_frac_per_s	float	0.0	%/hour	(0.0, 100.0)	Effective linear gain-drift rate since calibration (1-sigma fractional responsivity change per unit time; input as %/hour). Produces the signal-proportional ‘gain_drift’ residual term.
calibration.gain_uncertainty_pct	float	0.0	%	(0.0, 100.0)	Absolute radiometric gain (responsivity-scale) uncertainty (1-sigma, %). Direct BIAS input for systems whose absolute-cal budget is known as a number — accuracy budget only.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>calibration.narcissus_fpn_pct</code>	float	0.0	%	(0.0, 100.0)	Fraction of the uncorrected fore-optics offset appearing as spatial FPN (1-sigma, %) — the narcissus / vignetting pattern the internal cal imprints. Applies only under <code>cal_path = 'internal_shutter'</code> ; the mean offset itself is the <code>internal_cal_offset</code> BIAS term regardless of this value.
<code>calibration.nonlinearity_pct</code>	float	0.0	%	(0.0, 100.0)	Per-pixel quadratic-nonlinearity dispersion (1-sigma, % of the full-scale-referenced quadratic coefficient). Sets the post-NUC residual FPN amplitude under <code>'two_point'</code> — the parabolic residual vanishing at both cal points (plan §3.2, D1).
<code>calibration.offset_drift_e_per_s</code>	float	0.0	e-/hour	(0.0, 1000000000.0)	Effective linear offset-drift rate since calibration (1-sigma, electrons per unit time; input as e-/hour). Produces the signal-independent <code>'offset_drift'</code> residual term.
<code>calibration.scheme</code>	str	none	—	—	Radiometric calibration scheme: <code>'none'</code> (no calibration model — detector PRNU/DSNU act as static dispersions, today's behavior), <code>'one_point'</code> (offset corrected at one cal point), <code>'two_point'</code> (per-pixel gain and offset corrected at two cal points; residual set by nonlinearity dispersion and drift), or <code>'three_point'</code> (piecewise gain+offset through three cal points — each segment's nonlinearity parabola shrinks to that segment's span). Active schemes emit the post-NUC residual noise terms and calibration bias terms.
<code>calibration.shutter_after_element</code>	int	0	—	(0, 99)	Number of optical-train elements in FRONT of the internal cal shutter (scene side) — the flag sits behind the first N elements. Those N elements' near-field emission is what the internal cal cannot correct (the flag blocks them from the cal view; the NUC corrects only the elements behind it). Required when <code>cal_path = 'internal_shutter'</code> ; 0 = unset. N equal to the train length is legal — a flag at the detector leaves the entire train uncorrected, the maximum mismatch.

Parameter	Type	Default	Input Unit	Bounds	Description
calibration.source_emissivity	float	1.0	—	(0.0, 1.0)	Calibration-source emissivity (nominal). A scene-side material property of the cal source, so a legitimate independent input (Kirchhoff-derived emissivity is required of optical elements, not of sources). Must be > 0 when an emissivity uncertainty is set; validated at evaluate time.
calibration.source_emissivity_uncertainty	float	0.0	—	(0.0, 1.0)	Calibration-source emissivity uncertainty (1-sigma, absolute). Becomes a radiance-scale BIAS term ($\Delta\epsilon / \epsilon_{src}$) — accuracy budget only.
calibration.source_temp_uncertainty_K	float	0.0	K	(0.0, 100.0)	Calibration-source temperature uncertainty (1-sigma) [K]. Becomes a radiance-scale BIAS term via the band-integrated Planck derivative at the cal temperature — accuracy budget only, never RSS'd into noise (plan §3.4/§3.5).
calibration.source_uniformity_K	float	0.0	K	(0.0, 10.0)	Calibration-source spatial non-uniformity (1-sigma) across the aperture [K]. Imprinted into the correction at cal time: residual FPN = $dT_{unif} \times dS/dT$ at the cal temperature(s), non-zero even AT the cal points (unlike the NUC nonlinearity parabola). Cavity blackbodies typically hold 0.01-0.05 K. 0.0 (default) = perfectly uniform source, term off.
calibration.time_since_cal_s	float	0.0	hour	(0.0, 8760.0)	Elapsed time since the last calibration event. Drift terms grow linearly with this interval (time-linear v1, D4).

6.10 performance

Parameter	Type	Default	Input Unit	Bounds	Description
performance.detection_snr_threshold	float	5.0	—	(0.5, 100.0)	SNR at which a point target counts as detected — the threshold the in-chain detection-range solver bisects to. 5.0 is the classic Rose-criterion / SNR=5 detection threshold.
performance.metrics_interpretability	bool	True	—	—	Surface the Interpretability metric group: niirs, niirs_extrapolated, mrt_at_nyquist_K. Off stops the GIQE/IIRS and MRT computation.

Parameter	Type	Default	Input Unit	Bounds	Description
<code>performance.metrics.radiometric</code>	bool	True	—	—	Surface the Radiometric metric group: <code>snr</code> , <code>contrast_snr</code> , <code>scnr</code> , <code>detection_range_m</code> , <code>nedt_K</code> . Off stops their computation and any warnings they emit.
<code>performance.metrics.sampling</code>	bool	True	—	—	Surface the Sampling/geometry metric group: <code>gsd_*</code> , <code>ground_range_m</code> , <code>swath_width_m</code> , <code>access_rate_m2_s</code> , <code>q_*</code> , <code>sampling_regime_code</code> , <code>diffraction_limit_*</code> , <code>max_integration_time_s</code> . Off stops their computation.
<code>performance.metrics.saturation</code>	bool	True	—	—	Surface the Saturation metric group: <code>well_margin_dB</code> , <code>adc_margin_dB</code> , <code>dynamic_range_dB</code> . Off stops their computation.
<code>performance.metrics.spatial_mtf</code>	bool	True	—	—	Surface the Spatial-MTF metric group: <code>fwhm</code> , <code>rer</code> , <code>ee_1x1/3x3</code> , <code>mtf_at_nyquist</code> , <code>strehl(+marechal)</code> , <code>mtf_system/folded/alias</code> at Nyquist. Off stops the PSF/MTF spatial path, and with it the consistency check that compares the two spatial paths, unless an enabled metric needs a spatial input.
<code>performance.niirs.allow_extrapolated</code>	bool	False	—	—	Report a NIIRS/IIRS value even when a GIQE-5 input (GSD, RER, or SNR) is outside the published calibration ranges. Default False: an out-of-envelope configuration gets NIIRS as N/A (a result-typed <code>failure_reason</code> on <code>niirs_result</code> , no <code>niirs</code> metric) because the fitted formula is unreliable there — a strict refusal rather than a flagged extrapolation. True restores the extrapolated value, still flagged via <code>niirs_extrapolated</code> .

6.11 Consistency Groups

See `optics.f_number = optics.focal_length_m / optics.aperture_diameter_m`.

Error Taxonomy

Every exception RADIANT raises on purpose derives from `RadiantError`. That single base class is what lets user code separate “the framework rejected my input” from “an unrelated bug crashed the run”:

```
from radiant import RadiantError, Sensor

try:
    result = Sensor.from_yaml("my_config.yaml").evaluate()
except RadiantError as exc:
    # Every framework-defined rejection lands here.
    print(f"RADIANT rejected the run: {exc}")
```

`RadiantError` lives at `radiant.core.exceptions.RadiantError` and is re-exported at the top level as `radiant.RadiantError`. It is one of the two names in the package’s stability-guaranteed surface.

7.1 The actionability contract

An exception that says only “invalid parameter” is a bug. Every raise site carries three things:

Field	Question it answers
what	What specifically is wrong — the parameter, the value, the file, the line
why	The physics or logic reason it is wrong
action	What the user should do to fix it

Some classes formalize these as constructor fields, with an optional fourth context dict of diagnostic values that tooling can render field by field instead of parsing a message string:

```
from radiant.core.parameters import ParameterBoundsError

raise ParameterBoundsError(
    what=f"detector.detector_temperature_K = {T} K is out of bounds",
    why="HgCdTe operates at cryogenic temperature (1-300 K)",
    action="Set detector.detector_temperature_K to 77-120 K for HgCdTe detectors",
    context={
        "param": "detector.detector_temperature_K",
        "value": T,
        "bounds": (1, 300),
    },
)
```

Note the dot-path: parameter names have no `sensor.root` anywhere in RADIANT, in an error message no more than in a config file or a `Sensor.set()` call.

Rendered, the three fields flatten into one message (shown wrapped; the exception carries it on one line):

```
detector.detector_temperature_K = 320.0 K is out of bounds
| Why: HgCdTe operates at cryogenic temperature (1-300 K)
| Action: Set detector.detector_temperature_K to 77-120 K for HgCdTe detectors
```

The bounds check the framework performs for itself renders the same way, with the domain filled in from the schema:

```
Parameter 'source.target.emissivity' = 1.5 out of bounds [0.0, 1.0] (dimensionless)
| Why: 'source.target.emissivity' must lie within its declared valid domain
      [0.0, 1.0] dimensionless
| Action: Set 'source.target.emissivity' to a value in [0.0, 1.0] dimensionless
```

Classes that have not adopted the structured constructor carry the same three pieces of information in the message string. The classes with structured `what/why/action/` context fields are `ParameterBoundsError`, `ParameterEnumError`, `AtmosphereCapabilityError`, `TurbulenceSpecificationError`, `FPAPresetError`, `GeometrySpecificationError`, `ConfigSetError`, `ConfigurationScopeError`, `GuiUnavailableError`, and `ZemaxParseError`.

7.1.1 Built-in co-inheritance

Many classes co-inherit a built-in exception type — `ValueError`, `RuntimeError`, `KeyError`, `NotImplementedError`. That is a deliberate back-compat carve-out from the migration to the RADIANT hierarchy: sites that historically raised a bare `ValueError` are caught as such throughout the test suite and in user code, and those except `ValueError` blocks keep working. `RadiantError` remains the canonical base. **New** RADIANT exception classes should inherit from `RadiantError` only.

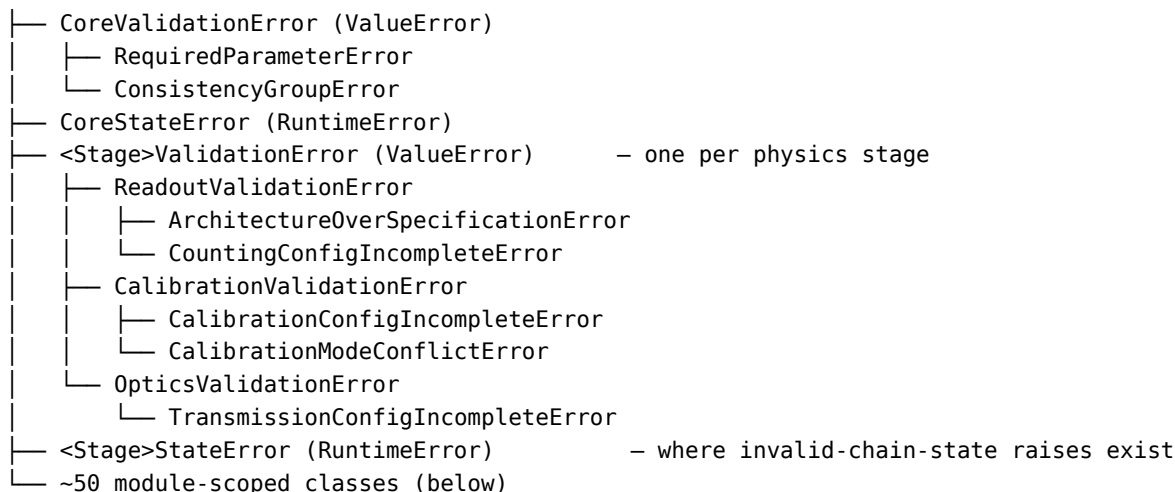
7.1.2 What is never used for errors

- `assert` is for developer invariants only, never for validating user input.
- No physics-layer function returns `NaN` or `inf` silently — it raises.
- No except `Exception`: `pass`, no silent default substitution, no clipping a value into range without at minimum a `UserWarning`.

The one carve-out is the **metric layer**: computations under `radiant.performance/` (`snr.py`, `nedt.py`, `niirs.py`) may return a result-typed failure carrying a structured `failure_reason` field instead of raising, because a metric that cannot be computed for this scene is a reportable outcome rather than a rejected input. The failure must be named in the result object the caller already inspects; silent `NaN` propagation stays forbidden. Physics-layer modules — source through readout — keep the universal raise rule.

7.2 The tree

```
Exception
└─ RadiantError
```



Most stage packages carry a stage-scoped `<Stage>ValidationError(RadiantError, ValueError)` in their `errors.py`, used by the generic input and argument guards, plus a `...StateError(RadiantError, RuntimeError)` where an invalid-chain-state raise exists.

Geometry is the exception. It raises `GeometrySpecificationError(RadiantError)` — `RadiantError` only, no `ValueError` co-inheritance, no `GeometryValidationError`. The “a `<Stage>ValidationError` in every stage” pattern is near-universal, not literally universal.

7.3 Core — `radiant.core`

Class	Bases	Module	What raises it, with an example message
<code>CoreValidationError</code>	<code>RadiantError</code> , <code>ValueError</code>	<code>core.exceptions</code>	Generic <code>radiant.core</code> input rejection. <i>“RadiometricFrame ‘at_aperture’: wavelength_um must be a 1-D array with at least 2 samples, got shape (1,)”</i>
<code>CoreStateError</code>	<code>RadiantError</code> , <code>RuntimeError</code>	<code>core.exceptions</code>	A core object used before it is ready (e.g. reading a <code>ParameterSet</code> before <code>resolve()</code>). <i>“radiant.core.solar: calibration integral is non-positive. Check the solar model constants.”</i>
<code>UnknownParameterError</code>	<code>RadiantError</code> , <code>KeyError</code>	<code>core.parameters</code>	A dot-path that is not in the schema; the message suggests near matches. <i>“Unknown parameter: ‘optics.nonexistent_param’. Did you mean: ‘optics.focal_length_m’, ‘optics.n_spiders’, ‘optics.defocus_um’?”</i>
<code>ConsistencyGroupError</code>	<code>CoreValidationError</code>	<code>core.parameters</code>	A consistency group whose members are all set and disagree; carries group and <code>parameters</code> structurally so the GUI routes it as an advisory. <i>“Consistency group ‘fnumber’ is over-constrained: ... Fix: either remove ‘optics. aperture_diameter_m’ from inputs and let it be derived, or correct the inconsistent value.”</i>

Class	Bases	Module	What raises it, with an example message
RequiredParameterError	CoreValidationError	core.parameters	Resolution with a required parameter unset. <i>“Required parameter ‘optics.aperture_diameter_m’ is not set. ... Action: set ‘optics.aperture_diameter_m’ — it has no default”</i>
ParameterBoundsError	RadiantError, ValueError	core.parameters	A value outside its declared physical domain. <i>“Parameter ‘source.target.emissivity’ = 1.5 out of bounds [0.0, 1.0] (dimensionless) Why: ... Action: Set it to a value in [0.0, 1.0]”</i>
ParameterEnumError	RadiantError, ValueError	core.parameters	A fixed-choice parameter given a value outside its closed set. <i>“Parameter ‘atmosphere.model’ = ‘fancy’; must be one of [‘simple’, ‘exo’, ‘tabulated’, ‘modtran’, ‘interpolated’] ... Action: Set it to one of [...]”</i>
OrbitError	RadiantError	core.orbit	Invalid orbital input. <i>“altitude_m must be positive (a LEO altitude), got -100.0.”</i>
RepeatGroundTrackError	RadiantError	core.repeat_ground_track	Invalid repeat-ground-track input. <i>“altitude_m must be positive, got 0.0.”</i>
SolarGeometryError	RadiantError	core.solar_geometry	Invalid solar-geometry input. <i>“day_of_year must be in [1, 366], got 400.”</i>

ParameterBoundsError is by far the most frequently raised class in the framework (279 raise sites), because every bounded parameter and every geometric guard funnels through it. `core.viewing_triangle` uses it for geometry-consistency violations, for example:

```
solve_viewing_triangle: h_sensor_m = 500.0 m is not above h_target_m =
8000.0 m, but      theta_o_rad = 0.2 rad (11.459°) places the sensor in the
above-the-horizon hemisphere seen from the target
```

7.4 Physics stages

Class	Bases	Module	What raises it, with an example message
GeometrySpecificationError	RadiantError	geometry.errors	An over- or under-specified geometry, or a scene-class assertion that contradicts the altitudes. <i>“geometry.scene_class asserts ‘air_to_ground’, but the altitudes derive ‘space_to_ground’ (sensor 600000 m ⇒ space, target 0 m ⇒ ground)” — the classic 600 m / 600 km magnitude slip.</i>
SourceValidationError	RadiantError, ValueError	source.errors	Source/target input guards. <i>“PhongBRDF: reflectance must be in [0, 1], got min=-0.2, max=1.4”</i>

Class	Bases	Module	What raises it, with an example message
AtmosphereValidationError	RadiantError, ValueError	atmosphere.errors	Atmosphere input and table guards. <i>“transmittance has negative values (min=-0.02), which have no logarithm. Transmittance is a probability and must be ≥ 0 — check the source table or tape7 columns.”</i>
AtmosphereStateError	RadiantError, RuntimeError	atmosphere.errors	An atmosphere backend failed at run time. <i>“MODTRAN exited with code 2. stderr: ...”</i>
AtmosphereCapabilityError	RadiantError, NotImplementedError	atmosphere.errors	The selected backend cannot serve this path topology (e.g. an up-looking scene on a down-looking family) — structured what/why/action.
TurbulenceSpecificationError	RadiantError	atmosphere.errors	An over- or under-specified turbulence input (r_0 vs. C_n^2 profile) — structured fields.
ModtranUnavailableError	RadiantError, RuntimeError	atmosphere.modtran	The MODTRAN binary is absent, the cache misses, and fallback is disabled. The message names the binary path that was tried.
Tape7ParseError	RadiantError, ValueError	atmosphere.modtran	A tape7 file that cannot be parsed. <i>“MODTRAN tape7 /path/tape7: fewer than 2 spectral data rows parsed. The output may be incomplete.”</i>
TransmissionConfigIncompleteError	OpticsValidationError	optics.errors	A transmission input mode selected without its inputs (key_elements with no element, spectral_file with no curve, ...) — a mid-switch state the GUI routes as an advisory. <i>“resolve_transmission: the key_elements transmission mode needs at least one optical element, and this configuration carries none. Add the elements on Optics ► Transmission ...”</i>
OpticsValidationError	RadiantError, ValueError	optics.errors	Optics input guards. <i>“scalar_rms_zernike_coeffs: rms_waves must be ≥ 0, got -0.05. An RMS wavefront error is a magnitude; set optics.wfe_rms_waves ≥ 0.”</i>
KirchhoffViolationError	RadiantError, ValueError	optics.element	An optical surface whose R and T violate energy conservation, or one given an independent emissivity. <i>“CavityModel: energy violation — $T_{\text{sys}} + R_{\text{sys}} = 1.04 > 1$. Check surface coating values.”</i>
PlatformValidationError	RadiantError, ValueError	platform.errors	Platform input guards. <i>“smear_width_m must be non-negative, got -1e-05”</i>
SpectralIntegrationValidationError	RadiantError, ValueError	spectral_integration.errors	Missing or malformed spectral input at the collapse. <i>“SpectralIntegrationStage: ‘post_optics’ frame has no spectral_radiance.”</i>

Class	Bases	Module	What raises it, with an example message
SpectralIntegrationStateError	RadiantError, RuntimeError	spectral_ integration.errors	The ensquared-energy invariant violated upstream. <i>“EE_box != 1.0 but regime is ‘extended’ (EE_box=0.82). In extended-scene mode, EE_box must not be applied. This is a programming error in PlatformStage.”</i>
DetectorValidationError	RadiantError, ValueError	detector.errors	Detector input guards. <i>“diffusion_length_m must be non-negative, got -2e-06”</i>
PersistenceSequenceError	RadiantError	detector. persistence_ sequence	Invalid persistence-sequence input. <i>“prior_signal_e must be ≥ 0, got -5.0.”</i>
ReadoutValidationError	RadiantError, ValueError	readout.errors	Readout input guards. <i>“check_well_saturation: full_well_capacity_e = 0.0 must be > 0.”</i>
ArchitectureOverSpecificReadoutValidationError	ReadoutValidationError	readout.errors	A readout architecture given a parameter its model derives. <i>“readout.full_well_capacity_e = 2e+06 e- is explicitly set while readout.architecture = ‘digital_counting’. Under counting the effective well is 2^counter_bits × count_packet_e.”</i>
CountingConfigIncompleteReadoutValidationError	ReadoutValidationError	readout.errors	A digital-counting readout missing the parameter that sizes the count. <i>“readout.count_packet_e is required when readout.architecture = ‘digital_counting’.”</i>
CalibrationValidationError	RadiantError, ValueError	calibration.errors	Calibration input guards. <i>“signal_e = -3.0 must be finite and non-negative. Why: the gain-drift residual is proportional to signal. Action: supply the post-integration signal in electrons.”</i>
CalibrationModeConflictError	CalibrationValidationError	calibration.errors	A cal-point mode mixed with the other mode’s inputs (flux_fraction with a cal temperature still set, or temperature with a flux point set) — a mid-switch state the GUI routes as an advisory. <i>“calibration.cal_temp_low_K is set, but calibration.cal_point_mode = ‘flux_fraction’. ... Action: unset calibration.cal_temp_low_K, or use cal_point_mode = ‘temperature’.”</i>
CalibrationConfigIncompleteError	CalibrationValidationError	calibration.errors	An active NUC scheme with no cal point. <i>“calibration.scheme = ‘two_point’ needs a cal point, but calibration.cal_temp_low_K is unset. Why: an active NUC scheme corrects at known cal-source temperatures; without them there is nothing to correct against.”</i>

PerformanceValidationError	RadiantError, ValueError	performance.errors	Metric input guards. “GSD must be positive, got along=0.0, cross=0.35”
----------------------------	--------------------------	--------------------	--

7.4.1 Metric-specific classes

Each independently testable metric under `radiant.performance/` owns its own error class, following the one-computation-one-module rule. All inherit `RadiantError` directly, and all guard the same way: a domain violation on a metric input.

Class	Module	Example message
<code>BlipRateError</code>	<code>performance.blip_rate</code>	“ <i>photon_signal_e must be ≥ 0, got -12.0.</i> ”
<code>DarkCrossoverError</code>	<code>performance.dark_crossover_rate</code>	“ <i>read_noise_e must be ≥ 0, got -1.0.</i> ”
<code>DetectivityError</code>	<code>performance.detectivity</code>	“ <i>area_cm2 must be positive, got 0.0.</i> ”
<code>DiffractionLimitError</code>	<code>performance.diffraction_limit</code>	“ <i>wavelength_m must be positive, got 0.0.</i> ”
<code>FrequencyUnitError</code>	<code>performance.frequency_units</code>	“ <i>convert_spatial_frequency: unknown unit ‘cycles/pixel’. Supported: cycles/mm, cycles/mrad, ...</i> ”
<code>GIQESensitivityError</code>	<code>performance.giqe_sensitivity</code>	“ <i>giqe5_sensitivity: rer = 0.0 must be positive — the GIQE-5 log terms are undefined at or below zero.</i> ”
<code>JohnsonCriteriaError</code>	<code>performance.johnson_criteria</code>	“ <i>critical_dimension_m must be positive, got 0.0.</i> ”
<code>MinimumResolvableError</code>	<code>performance.minimum_resolvable</code>	“ <i>noise_metric must be ≥ 0, got -0.1.</i> ”
<code>NepElectronsError</code>	<code>performance.nep_electrons</code>	“ <i>qe must be in (0, 1], got 1.4.</i> ”
<code>NepNetdError</code>	<code>performance.nep_netd</code>	“ <i>nep_w must be ≥ 0, got -1e-12.</i> ”
<code>NoiseEquivalentIrradianceError</code>	<code>performance.noise_equivalent_irradiance</code>	“ <i>total_noise_e must be ≥ 0, got -4.0.</i> ”
<code>RocError</code>	<code>performance.roc</code>	“ <i>snr must be ≥ 0, got -2.0.</i> ”
<code>SamplingRegimeError</code>	<code>performance.sampling_regime</code>	“ <i>Q must be positive, got 0.0.</i> ”
<code>TemperatureRetrievalError</code>	<code>performance.temperature_retrieval</code>	“ <i>wavelength_um_band must be a 1-D array with ≥ 2 points.</i> ”

7.5 I/O and data — `radiant.io`, `radiant.data`

Class	Bases	Module	What raises it, with an example message
<code>ConfigError</code>	<code>RadiantError</code>	<code>io.config</code>	A YAML config that cannot be loaded: malformed document, a reserved key, or a structured section this loader cannot attach. “ <i>Top-level YAML must be a mapping, got list.</i> ”
<code>ElementConfigError</code>	<code>RadiantError</code> , <code>ValueError</code>	<code>io.element_config</code>	A malformed <code>optical_elements</code> entry, Kirchhoff violations included. “ <i>Element ‘M1’: transfer_mode must be ‘REFLECTIVE’ or ‘REFRACTIVE’, got ‘MIRROR’.</i> ”

Class	Bases	Module	What raises it, with an example message
MeasurementParseError	RadiantError	io.measurement	A measured-curve CSV that will not parse. <i>“line 14: row has 1 column(s) but x_column=0, y_column=1 require at least 2. Check the delimiter (currently splitting into ['0.5;0.83']) and the column indices.”</i>
QeCsvParseError	RadiantError	io.qe_csv	A vendor QE CSV whose units cannot be resolved. <i>“cannot infer the wavelength unit from header column ‘lambda’. Pass wavelength_unit=“nm” or “um” explicitly.”</i>
DarkCurrentCsvParseError	RadiantError	io.dark_current_csv	A vendor dark-current CSV problem, including out-of-range temperature queries. <i>“file /path/jdark.csv does not exist or is not a file. Check the path; vendor dark-current data is the CSV export of the J_dark(T) datasheet plot.”</i>
ZemaxParseError	RadiantError	io.zemax_zernike	A Zernike Standard Coefficients export that will not parse — wrong file kind, unknown encoding, or repeated Noll indices from a multi-field export. Structured what/why/action/context.
AsterLibraryError	RadiantError	io.aster_library	An ASTER spectral-library file problem. <i>“file /path/spectrum.txt does not exist or is not a file. Check the path to the ASTER library spectrum (speclib.jpl.nasa.gov text format).”</i>
TargetLibraryError	RadiantError	io.target_library	A target-list workbook problem, including the missing openpyxl extra. <i>“file /path/targets.xlsx does not exist or is not a file. Check the path to the target-library workbook.”</i>
ResultArchiveError	RadiantError	io.serialization	A saved-result archive that cannot be read. <i>“Result archive not found: /path/run.zip. Check the path, or produce one with ChainResult.save(path).”</i>
FPAPresetError	RadiantError	data.fpa	A malformed or unknown FPA preset. Structured fields: <i>what</i> “Preset ‘x.yaml’ field ‘vendor’ is missing or not a non-empty string”, <i>why</i> “Every preset names its part identity so provenance is auditable”, <i>action</i> “Add ‘vendor: <text>’ to ...”.

7.6 API and front ends

Class	Bases	Module	What raises it, with an example message
ApiValidationError	RadiantError, ValueError	api.errors	A radiant.api call given a bad argument. <i>“Sensor.load: ‘_radiant.wavelength_points’ must be an integer >= 2, got 1 in my_config.yaml.”</i>

Class	Bases	Module	What raises it, with an example message
SpectralBandError	ApiValidationError	api.errors	The filter band edges are inverted or coincide, checked before any stage runs; structured what/why/action naming both edges, routed by the GUI as an advisory (CU-373 F-21). <i>"spectral_integration.filter_min_um (8 μm) is not below spectral_integration.filter_max_um (5 μm)"</i>
OperationCancelledError	RadiantError	api._progress	A long-running operation aborted through its cancel() callback. Carries operation, done, total. <i>"sweep canceled after 12/51 evaluations. No result is returned for a canceled operation; re-run, or sweep in smaller chunks if partial results are needed."</i>
SolveBracketError	RadiantError	api.solve	solve_for given a bracket that does not contain the target; carries both endpoint metric values. <i>"solve_for('optics.aperture_diameter_m'): bounds must satisfy lo < hi, got (1.0, 0.05)."</i>
BatchRunnerError	RadiantError	api.batch	Invalid batch construction or an invalid pivot query. <i>"BatchRunner needs at least one axis; got an empty sequence."</i>
ComparisonError	RadiantError	api.compare	An invalid compare_configs request. <i>"compare_configs needs at least 2 configurations, got 1. Pass two or more (label, ChainResult) pairs."</i>
MtfComparisonError	RadiantError	api.compare	An invalid compare_mtf request. <i>"compare_mtf: axis must be 'x' or 'y', got 'z'. 'x' is cross-track, 'y' is along-track."</i>
ConfigSetError	RadiantError	api.config_set	An invalid configuration-set operation: a duplicate name, a value-list length mismatch, a parameter both shared and configured, more than MAX_CONFIGS names. Structured fields.
ErrorBudgetError	RadiantError	api.error_budget	A malformed error-budget contributor. <i>"BudgetContributor: name must be non-empty."</i>
CalibrationAnalysisError	RadiantError	api.calibration_analysis	An invalid calibration-analysis input. <i>"wavelength_um must be a 1-D array with ≥ 2 points."</i>
GuiUnavailableError	RadiantError	cli.gui	radiant gui without the GUI extra installed. Structured fields naming pip install "radiant[gui]".
GuiValidationError	RadiantError, ValueError	gui.errors	A GUI-layer input rejection. <i>"RADIANT_GUI_DEBOUNCE_MS=-50 ms is negative — a debounce window cannot run backwards in time."</i>
ConfigurationScopeError	RadiantError	gui.config_scope	An edit applied at the wrong configuration scope (shared vs. per-configuration). Structured fields.

7.7 Catching selectively

Catch `RadiantError` when you want “the framework said no” and nothing else. Catch a specific class when you intend to recover from that one condition:

```
from radiant import RadiantError, Sensor
from radiant.api import OperationCancelledError
from radiant.core.parameters import ParameterBoundsError

sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")

try:
    sensor.set("source.target.emissivity", 1.5)
except ParameterBoundsError as exc:
    print(exc.what)           # the specific violation
    print(exc.why)           # the physics reason
    print(exc.action)        # the fix
    print(exc.context["bounds"]) # (0.0, 1.0) – structured, no string parsing
except RadiantError:
    raise
```

Because of the co-inheritance carve-out, `except ValueError` still catches the bounds and enum errors, and `except KeyError` still catches `UnknownParameterError`. Code that wants to be explicit about intent should name `RadiantError` or the concrete class instead.

A `BatchRunner` cell is the one place where a `RadiantError` is deliberately *recorded* rather than propagated: the cell becomes a row with a populated error column. Any other exception type is treated as a programming bug and propagates out of the batch.

Data Libraries

RADIANT ships three reference-data libraries inside the package, so a useful evaluation needs no external downloads and no MODTRAN license:

Library	Accessor	Contents
Spectral library	<code>radiant.data.SpectralLibrary</code>	19 material emissivity spectra, 6 detector QE curves, AM0 solar irradiance
FPA preset library	<code>radiant.data.FPALibrary</code>	21 focal-plane / ROIC presets with per-value datasheet attribution
Atmosphere library	selected through <code>atmosphere.model</code>	MODTRAN-derived transmittance and path radiance, as profiles and interpolable families

All three live under `src/radiant/data/tables/` in a source checkout and ship inside the wheel at `radiant/data/tables/`. Each directory carries a `MANIFEST.md` naming its generator, its inputs, and its provenance, per the regenerable-artifact rule.

8.1 SpectralLibrary

```
from radiant.data import SpectralLibrary
```

```
lib = SpectralLibrary()
```

```
lib.materials()                # 19 names, sorted
emissivity = lib.material("vegetation_green")
lib.material_info("vegetation_green")
# {'default_temperature_K': 300,
#  'description': 'Healthy green vegetation (deciduous canopy)',
#  'filename': 'vegetation_green.csv',
#  'source_citation': "Salisbury & D'Aria, JGR, 1992; ASTER Spectral Library"}
```

```
lib.detectors()                # 6 names, sorted
qe = lib.detector_qe("hgcdte_mwir")
```

```
solar = lib.solar()            # AM0 exo-atmospheric spectral irradiance
```

Every accessor returns a `SpectralData` object carrying `.wavelength_um` and `.values` arrays plus `.unit`, `.source`, and `.name` metadata — so a curve never travels without its units or its citation.

8.1.1 Materials — 19 emissivity spectra

aluminum, asphalt, blackbody, concrete, copper, glass, gold, ice, paint_black, paint_grey, paint_white, sand, snow, soil_dry, soil_wet, steel, vegetation_dry, vegetation_green, water_calm.

Values are dimensionless emissivity, $0 \leq \varepsilon(\lambda) \leq 1$. Per-material description, default temperature [K], and source citation live in `tables/emissivity/manifest.yaml` and come back from `material_info()`.

This is scene-material emissivity — a legitimate independent property. It is *not* the route for optical-element emissivity, which is always Kirchhoff-derived from R and T.

These curves are reachable from a config without any Python:

```
source:
  background:
    material: vegetation_green # resolved against SpectralLibrary before the chain
```

8.1.2 Detector QE — 6 curves

hgcdte_lwir, hgcdte_mwir, ingaas, inp_ingaasp, silicon, type2_sls.

Values are QE as a fraction. A curve selected through `detector.qe_material` supersedes the scalar `detector.qe_value`; a user-supplied CSV via `detector.qe_table_path` supersedes both. All three are resolved by the API layer *before* the chain, because stages do not read files.

8.1.3 Solar

`lib.solar()` returns AM0 exo-atmospheric spectral irradiance in W/m²/μm, from the RADIANT solar data library (a Planck fit to the Kopp & Lean 2011 total solar irradiance). It is the illumination source for every reflective-regime scene.

8.2 FPALibrary — 21 presets

An FPA preset is a curated set of `detector.*` and `readout.*` values taken from one vendor datasheet or paper, with **per-value attribution**: every number records the unit it was quoted in, the document it came from, and the exact location in that document.

```
from radiant.data import FPALibrary

lib = FPALibrary()
lib.names() # 21 part slugs, sorted
preset = lib.part("teledyne-h2rg-2p5")

preset.vendor # 'Teledyne Imaging Sensors'
preset.part_class, preset.part_kind
preset.material
preset.band.label, preset.band.cut_on_um, preset.band.cut_off_um
preset.parameters["detector.pixel_pitch_x_um"]
# FPAParameterEntry(value=18.0, unit='um', basis='datasheet', source='ds2022',
```

```
#                               location="'Array Format 2048 x 2048 pixel, 18 um pitch'", note=None)
preset.sources                  # source keys -> FPASource records (title, URL, file)
preset.qe_table                 # a bundled QE curve name, or None
```

Applying one is a single call (or a single config key):

```
from radiant.api import Sensor
```

```
sensor = Sensor.from_yaml("examples/mwir_leo_minimal.yaml")
report = sensor.apply_fpa("scd-blackbird-1920")
```

Values are stored in **the cited document's native unit** and converted to canonical units at apply time through the unit-aware parameter boundary — the one conversion point. An entry whose basis is assumed rather than datasheet carries a justification note instead of a document location, so an inferred value is never mistaken for a quoted one.

8.2.1 The roster

Part	Vendor	Class	Detector material	Band [μm]
dfpa-generic	MIT Lincoln Laboratory (research lineage)	cooled IR DROIC	detector-agnostic	1.0–14.0
e2v-ccd42-40	Teledyne e2v	scientific visible	back-illuminated Si CCD (AIMO)	0.35–1.06
flir-boson-plus-640	Teledyne FLIR	uncooled bolometer	microbolometer (VOx family)	8.0–14.0
flir-lepton-35	Teledyne FLIR	uncooled bolometer	VOx microbolometer	8.0–14.0
flir-neutrino-lc	Teledyne FLIR	cooled IR	HOT MWIR MCT-class	3.4–4.9
geosnap-10	Teledyne Space Imaging	cooled IR	HgCdTe, customer cutoff	0.4–14.0
geosnap-18	Teledyne Space Imaging	cooled IR	HgCdTe, customer cutoff	0.4–15.0
gpixel-gsense400bsi	Gpixel	scientific visible	BSI Si sCMOS	0.35–1.0
lynred-atto640	Lynred	uncooled bolometer	microbolometer (a-Si family)	8.0–14.0
lynred-daphnis-hd-mw	Lynred	cooled IR	HgCdTe MWIR, HOT to 110 K	3.7–4.8
rvs-miri-si-as	Raytheon Vision Systems	cooled IR	Si:As IBC	5.0–28.0
rvs-virgo-2k	Raytheon Vision Systems	cooled IR	HgCdTe on CdZnTe	0.75–2.45
scd-blackbird-1920	SCD	cooled IR	InSb	1.0–5.4
senseker-calcium-rp0033	Senseker	cooled IR DROIC	bare DPROIC	1.0–14.0
senseker-magnesium-rp0092	Senseker	cooled IR DROIC	bare DPROIC	2.0–14.0
sony-imx250	Sony Semiconductor Solutions	scientific visible	FSI Si CMOS (Pregius)	0.32–1.1
sony-imx455	Sony Semiconductor Solutions	scientific visible	BSI Si CMOS	0.35–1.0
sony-imx990-senswir	Sony Semiconductor Solutions	SWIR	InGaAs on Si (SenSWIR)	0.4–1.7
teledyne-e2v-ccd273	Teledyne e2v	scientific visible	back-illuminated full-frame Si CCD	0.3–1.0
teledyne-h2rg-2p5	Teledyne Imaging Sensors	scientific visible	substrate-removed HgCdTe hybrid	0.4–2.5

Part	Vendor	Class	Detector material	Band [μm]
teledyne-h4rg-10	Teledyne Imaging Sensors	scientific visible	HgCdTe hybrid, 2.5 μm cutoff	0.5–2.5

The roster deliberately spans five part classes — cooled IR, DROIC / digital counting, uncooled bolometer, scientific visible, and SWIR — so a trade study can move between detector technologies without hand-transcribing a datasheet.

8.2.2 Provenance chain

Each preset's sources block names the documents it cites. The documents themselves are committed at docs/validation/fpa_datasheets/ (22 PDFs — vendor datasheets, product leaflets, and peer-reviewed papers) with a MANIFEST.md giving, per file, the title, the acquisition URL (a Wayback URL where the live document is delisted), the retrieval date, and a SHA-256. A test asserts that every file a preset cites exists there and matches its hash.

Those PDFs are **repo-only** and excluded from the wheel — an installed RADIANT gives you each preset's citation URL or DOI instead of a redistributed copy. A vendor revision is handled as a new file plus a preset update, never an in-place replacement of a hashed document.

src/radiant/data/tables/fpa/configs/ holds one generated, loadable RADIANT config per preset, rendered by scripts/gen_fpa_configs.py and freshness-gated by a test.

8.3 Atmosphere library

The atmosphere tables are the largest shipped data product: NPZ spectra derived from a real MODTRAN 6 run matrix, packaged for the tabulated and interpolated models so a user without a MODTRAN license still gets real radiative-transfer atmospheres.

Family	Model	Coverage
profiles/	tabulated	Six standard atmospheres, nadir full column (us_standard, tropical, midlat_summer also carry real downwelling sky radiance)
us_standard_zenith_fan/	interpolated	Line-of-sight zenith fan, 0–60°
midlat_summer_ladders/	interpolated	Sensor altitude (35 km – GEO) × target altitude (0–29 km)
midlat_summer_boost_ladder/	interpolated	Space sensor × target altitude 0–100 km (missile boost)
midlat_summer_boost_offnadir/	interpolated	Space sensor × target 0–100 km × LOS zenith 0/45/60°
midlat_summer_sensor_ladder/	interpolated	Airborne → space sensor (3 km – GEO), ground target
midlat_summer_uplooking_*	interpolated	Up-looking families (ladders, sensor ladder, zenith fan)
midlat_summer_sst_column_fan*	interpolated	Up-looking whole-column to space, zenith 0–78.5°, sea level and 900 m site
midlat_summer_upwelling_offnadir/	interpolated	Down-looking upwelling grid, off-nadir
validation/	point data	Off-grid 45° and up-looking anchors

8.3.1 Provenance and treatment

The generator is `scripts/build_atmosphere_library.py`, reading a tracked MODTRAN 6 run set: a 39-run base, a 17-run boost-ladder expansion, and two geometry-flexibility batches. Every array is slit-degraded with a triangular FWHM = 5 cm^{-1} kernel on the native uniform 1 cm^{-1} wavenumber grid, then decimated to 2 cm^{-1} sampling and stored as float32 — 12,984 points spanning 0.375–14.29 μm . Band-mean τ shifts by $\lesssim 0.003$ (dimensionless) against the full-resolution tape7s.

Each NPZ carries `wavelength_um` [μm , ascending], `transmittance` [dimensionless], `path_radiance` [$\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$], an optional `atm_emission_down` [$\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$], and a `geometry` dict recording the full five-field run geometry: `sensor_altitude_m` [m], `target_altitude_m` [m], `path_zenith_rad` [rad], `solar_zenith_rad` [rad], and `solar_azimuth_rad` [rad]. All shipped down-looking runs used a solar zenith of 30° and an azimuth of 0° ; the interpolated model warns when a query departs from a recorded non-axis value.

8.3.2 Selection

With `atmosphere.model: interpolated` and no `atmosphere.interpolated_data_dir`, the loader picks the bundled family matching the scene’s line-of-sight direction and the configured `atmosphere.interpolation_axes`. The model **does not extrapolate**: a query outside a family’s node coverage is refused with an actionable error naming the coverage it does have, rather than silently producing an invented atmosphere.

Point `atmosphere.interpolated_data_dir` at your own directory of NPZ runs to use a private run matrix — the bundled catalog is a convenience, not a constraint.

Full detail, including the family tables, the run provenance, and the known limitations, is in the shipped library’s own manifest and README. Choosing between the five atmosphere backends for a given scene is the subject of the repository’s atmosphere-selection guide.

8.4 Mission templates

Not a data library in the same sense, but shipped in the same tree: nine complete, runnable mission configs at `radiant/data/templates/`, listed in the Command-Line Interface chapter under `radiant template`. They are the GUI welcome-screen set and the fastest way to a working config in a new band or regime.

External Data Interfaces

RADIANT reads several external file formats: MODTRAN output, vendor detector datasheet exports, Zemax wavefront reports, spectral-library files, and Excel workbooks. Every one of them follows the same two rules.

Conversion happens at the reader, once. A tape7 is in wavenumber and W/cm²; a vendor QE curve may be in nanometers and percent; a dark-current plot is in A/cm². Each reader converts to canonical units at the file boundary and hands the rest of the framework a clean object. No physics module ever sees a foreign unit.

Stages never read files. Every importer here is called *before* chain execution, by the API layer or by your own script, and its product is injected as a pre-chain stage output. This is what keeps stages pure functions.

This chapter documents what actually ships. Where a format is designed but not implemented, that is said plainly.

9.1 MODTRAN

Two ways in, and they are not equal.

9.1.1 Tape7 file import — the primary path

Set `atmosphere.modtran.tape7_path` to a tape7 produced anywhere — a colleague’s workstation, an archived run, a contractor deliverable. The loader parses it before the chain; the MODTRAN binary, the cache, and the fallback are never consulted.

```
atmosphere:
  model: modtran
  modtran:
    tape7_path: runs/midlat_summer_nadir/tape7
    tape7_sun_path: runs/midlat_summer_nadir/tape7_sun # optional solar leg
    tape7_up_path: runs/midlat_summer_nadir/tape7_up # optional up-looking leg
    flux_path: runs/midlat_summer_nadir/flux # optional flux output
```

Conversions applied exactly once, in `Tape7Reader.to_radiant_units`:

From	To	Relation
Wavenumber ν [cm ⁻¹]	Wavelength λ [μm]	$\lambda = 10000/\nu$
Radiance [W/cm ² /sr/cm ⁻¹]	[W/m ² /sr/μm]	$L(\lambda) = L(\nu) \nu^2$
Transmittance [-]	[-]	unchanged
Descending λ	Ascending λ	array reversed

The ν^2 factor is two conversions whose powers of ten cancel: the spectral-axis Jacobian $|d\nu/d\lambda| = 10^4/\lambda^2 = \nu^2/10^4$, times the 10^4 that carries $\text{cm}^{-2} \rightarrow \text{m}^{-2}$ on the area. A tape7 that yields fewer than two spectral rows, or whose columns cannot be located, raises `Tape7ParseError` naming the file.

A tape7 is **geometry-agnostic**: it is one path, already computed. The model does not respond to a change in altitude or look angle, and the chain says so in an advisory note rather than silently reusing the wrong path.

9.1.2 Binary invocation — the secondary path

With a MODTRAN executable installed, RADIANT can render a tape5 card deck, run the binary, parse the tape7, and cache the result keyed by an SHA-256 of the rendered deck. The relevant parameters are `atmosphere.modtran.binary_path`, `cache_dir`, `atmosphere_profile`, `aerosol_model`, `h2o_scale`, `o3_scale`, `spectral_resolution_cm1`, and `allow_fallback`.

When the binary is unavailable the behavior is explicit, never silent:

Condition	Behavior
Cache hit	Use the cached result
<code>allow_fallback: true</code>	Translate the parameters to the simple analytic model and warn
<code>allow_fallback: false</code>	Raise <code>ModtranUnavailableError</code> naming the binary path tried

This path has never been exercised against a licensed installation in the shipped test matrix; the tape7-import path is the supported one.

9.1.3 The shipped alternative

Most users need neither: the bundled atmosphere library is MODTRAN-derived data, available through `atmosphere.model: interpolated`. See the Data Libraries chapter.

9.2 Measured curves

`radiant.io.measurement.load_measured_curve` reads a two-column (x, y) curve from any delimited text file — measured MTF against spatial frequency, measured transmission against wavelength — for comparison against a RADIANT prediction.

```
from radiant.io.measurement import load_measured_curve
```

```
curve = load_measured_curve(
    "lab/mtf_measured.csv",
    x_column=0, y_column=1,
    delimiter=",",
    skip_header="auto",
    x_unit="cycles/mm",
)
curve.x, curve.y, curve.n_points, curve.x_unit, curve.source_file
```

Parsing rules, chosen so nothing is dropped quietly:

- `#` comment lines and blank lines are skipped.
- `skip_header="auto"` treats a first row that does not parse numerically as a header. An integer skips exactly that many data rows unconditionally.
- Any non-numeric value after the header is a hard `MeasurementParseError` — no silent row dropping.
- `x` must be strictly ascending. Out-of-order rows are sorted with a `UserWarning`; duplicated `x` values raise, because the intended ordering is then ambiguous.

Excel workbooks are out of scope for this reader — export the sheet to CSV first.

9.3 Vendor detector data

9.3.1 QE curves

`radiant.io.qe_csv.load_qe_csv` reads a vendor QE export into canonical units — μm and fraction — resolving the mixed conventions vendors ship (`wavelength_nm`, `QE_pct` and `lambda_um`, `quantum_efficiency` are both common):

```
import numpy as np
from radiant.io.qe_csv import load_qe_csv

qe = load_qe_csv("vendor/imx455_qe.csv")           # units inferred from the header
qe = load_qe_csv("vendor/qe.csv", wavelength_unit="nm", qe_unit="percent")

qe.evaluate(np.array([4.0, 4.2, 4.4]))             # QE at those wavelengths [μm], as fractions
qe.band_averaged_qe(3.5, 5.0)                     # band mean over 3.5-5.0 μm
```

Unit resolution in "auto" mode is header-driven, never magnitude-driven: a header token containing `nm` means nanometers, one containing `um` / μm / `micron` means micrometers, and a token containing `pct` / `percent` / `%` means percent. A header with no hint raises `QeCsvParseError` asking for an explicit `wavelength_unit` rather than guessing. A fraction-mode curve carrying values above 1.0 raises with a pointer at `qe_unit="percent"` instead of producing unphysical QE.

The same file can be reached from a config through `detector.qe_table_path`, in which case the API layer loads it before the chain and injects the curve onto the evaluation grid; past-cutoff QE is zero.

9.3.2 Dark current

`radiant.io.dark_current_csv.load_dark_current_csv` reads a temperature-vs-current-density export (`T_K`, `Jdark_A_cm2`) and converts to RADIANT's canonical dark rate for a given pixel pitch:

$$\text{rate} [\text{e}^-/\text{s}] = \frac{J [\text{A}/\text{cm}^2] \cdot (\text{pitch} \cdot 100)^2 [\text{cm}^2]}{q}$$

```
from radiant.io.dark_current_csv import load_dark_current_csv

dc = load_dark_current_csv("vendor/jdark.csv")
dc.j_dark_at(95.0)                                # A/cm² at 95 K
dc.dark_rate_e_per_s(95.0, pixel_pitch_m=18e-6)   # e-/s/pixel
dc.temperature_at_rate(100.0, pixel_pitch_m=18e-6) # K giving 100 e-/s/pixel
```

Interpolation is Arrhenius-faithful: $\ln J$ is interpolated linearly in $1/T$, which is exact for $J \propto \exp(-E_a/k_B T)$ between nodes and a far better model of diode dark current than linear-in- T . A query outside the measured temperature range **raises** — silently extrapolating an exponential is how dark-current budgets go wrong.

9.4 Zemax wavefront reports

`radiant.io.zemax_zernike.load_zemax_zernike` parses the text report Zemax (OpticStudio) writes from *Analyze* → *Wavefront* → *Zernike Standard Coefficients* → *Save As Text*. It does **not** read a .ZMX lens prescription.

```
from radiant.io.zemax_zernike import load_zemax_zernike

z = load_zemax_zernike("optics/lens_field0.txt")
z.zernike_coeffs          # {Noll index: coefficient [waves]}
z.reference_wavelength_um # μm, or None if the header line is absent
z.n_terms
wfe = z.to_wavefront_error() # feeds the ZERNIKE wavefront pipeline directly
```

Zemax “Standard” Zernikes are Noll-numbered and quoted in waves at the stated wavelength — exactly RADIANT’s own convention, so the parsed set feeds the wavefront pipeline with no renumbering. The parser tolerates both Z 1 and Z1 index styles, an optional trailing polynomial formula, header wording differences, and UTF-16 (LE/BE, with or without BOM), UTF-8, or latin-1 encodings — Zemax on Windows usually writes UTF-16-LE.

The report is single-field. A multi-field export concatenated into one file repeats Noll indices and is rejected with an actionable error telling you to export each field separately.

From a config, `optics.zernike_file` names the same report and the API layer loads it pre-chain, superseding the scalar `optics.wfe_rms_waves` path.

9.5 Spectral libraries

`radiant.io.aster_library.load_aster_spectrum` reads one file of the JPL/NASA ASTER spectral library — the Name: / Type: metadata header followed by two whitespace-separated columns:

```
from radiant.io.aster_library import load_aster_spectrum

spec = load_aster_spectrum("aster/conifer.txt")
spec.name, spec.wavelength_um, spec.reflectance
spec.emissivity()          # 1 - rho(lambda), Kirchhoff for an opaque material
spec.band_averaged_emissivity(8.0, 12.0)
```

ASTER files usually list wavelength in *descending* order; the reader sorts ascending and returns fractional reflectance. The Y unit is taken from the Y Units: header line (percent vs. fraction); a file with no recognizable unit line raises rather than guessing.

Emissivity for an opaque scene material is $\varepsilon(\lambda) = 1 - \rho(\lambda)$. This is the legitimate independent-emissivity case — Kirchhoff’s derived-only constraint binds optical elements, not scene targets.

RADIANT’s own bundled material and QE curves are covered in the Data Libraries chapter. A measured background emissivity can also be attached from a config through `source.background.emissivity_path` (a two-column `wavelength_um`, `emissivity` CSV), and a target reflectance or brightness temperature the same way.

9.6 Spreadsheets

Two Excel interfaces ship, and both depend on `openpyxl` from an optional extra. A third is designed but not implemented — see below.

9.6.1 Target-library import — [scenarios] extra

`radiant.io.target_library.load_target_library` reads a program-office target list from a workbook:

```
from radiant.io.target_library import load_target_library

targets = load_target_library("program/targets.xlsx", sheet="Targets")
for t in targets:
    t.target_name, t.length_m, t.width_m, t.height_m
    t.temperature_K, t.emissivity, t.material
    t.projected_area_m2      # length_m x width_m, the sub-pixel / point-source area term
```

Expected header columns, in any order: `target_name`, `length_m`, `width_m`, `height_m`, `temperature_K`, `emissivity`, `material`. `openpyxl` is imported lazily, so a missing extra raises an actionable `TargetLibraryError` naming the install rather than breaking `import radiant.io`:

```
pip install "radiant[scenarios]"
```

9.6.2 Workbook export — [gui] extra

`radiant.gui.xlsx_export.export_workbook` writes one workbook with up to three sheets, built purely from public API surfaces:

Sheet	Columns	Source
Config	parameter, value, unit	<code>Sensor.parameter_defs() + get_input()</code>
Metrics	name, value, unit, description	<code>ChainResult.to_records()</code>
Sweep	param + metric columns (1-D) or long form (2-D)	the last sweep, when one exists

```
from radiant.gui.xlsx_export import export_workbook

export_workbook("study.xlsx", sensor, result, sweep)
```

Numeric cells keep full precision — numbers, not formatted strings — and units get their own column, so the workbook is usable as data rather than only as a report.

9.6.3 Not implemented: radiant export / radiant import

The repository’s config-format document describes an XLSX *convenience view* of a config file — one sheet per namespace, editable value column, round-trippable back to YAML for reviewers who do not write YAML. **This is a design target.** There is no `radiant export` or `radiant import` command and no XLSX config code in `radiant.io`. (The existing `radiant convert` CLI is an unrelated scalar unit converter.) Use `Sensor.to_yaml()` or the GUI workbook export above.

9.7 Other file-valued parameters

Beyond the importers above, several schema parameters take a file path directly. The API layer loads each one before the chain and injects the product:

Parameter	File	Injected as
<code>atmosphere.modtran.tape7_path</code>	MODTRAN tape7	the atmosphere model
<code>atmosphere.tabulated_transmittance_file / ..._path_radiance_file / ..._downwelling_file</code>	CSV or NPZ, first column wavelength_um	the atmosphere model
<code>atmosphere.interpolated_data_dir</code>	directory of NPZ runs	the atmosphere model
<code>atmosphere.cn2_tabulated_file</code>	altitude-vs- C_n^2 CSV	<code>atmosphere_config.cn2_profile</code>
<code>detector.qe_table_path</code>	wavelength-vs-QE CSV	<code>spectral_integration.qe_curve</code>
<code>optics.zernike_file</code>	Zemax Zernike report	<code>optics_config.wavefront_error</code>
<code>source.background.emissivity_path</code>	wavelength-vs-emissivity CSV	<code>source_config.background_emissivity</code>
<code>source.target.emissivity_path / reflectance_path / brightness_temperature_path</code>	two-column CSV	the source descriptor
<code>element-train R/T references</code>	spectral CSV, or an inline table in the YAML	<code>optics_config.element_list</code>

Every one of these paths is resolved **relative to the config file’s own directory**, and `Sensor.save()` writes them back out relative to the destination directory — so a config and its data files move together as a portable unit.

Part 3 — Internals: A Note to the Reader

The three chapters that follow are not written for this manual. They are RADIANT’s living architecture specifications, bound here exactly as they exist in the repository:

Chapter	Source document	What it specifies
Signal-Chain Internals	docs/architecture/RADIANT_Signal_Chain_Architecture.md	The Stage protocol, ChainState and its with_* constructors, reference frames, regime dispatch, the runner
Parameter System Internals	docs/architecture/RADIANT_Parameter_System.md	ParameterDef, ParameterSet, resolution order, consistency groups, tolerances, provenance
Testing & Validation Framework	docs/architecture/RADIANT_Testing_Validation.md	The Level 0/1/2 test hierarchy, the reference cases, golden-result protocol, provenance tracking

Why bound as-is rather than rewritten. These three documents are under the project’s doc-and-code lock-step rule: any change to a stage protocol, a ChainState field, the parameter schema surface, or the test contract must update its specification in the same pull request that changes the code. That makes them the freshest description of the internals that exists. Distilling them into manual prose would create a second copy with no such guarantee — a copy that would start drifting the day it was written. Binding them directly means this volume inherits their accuracy for free.

The only editorial change is mechanical: the build strips each document’s Date / Status / Depends-on / Scope metadata block, which is repository bookkeeping rather than manual content. The files on disk are never modified by the build.

What that means for you as a reader. These chapters address a contributor working in the source tree, so their voice is more prescriptive than the rest of this volume and their cross-references cite repository paths — src/radiant/core/chain.py, docs/adr/ADR-0006-geometry-stage.md, tests/integration/ — rather than chapters of this manual. A path of that form refers to the RADIANT repository. Where a chapter names a rule by number (“Rule 9”, “C6”), it means the numbered constraints in CLAUDE.md and docs/architecture/RADIANT_Master_Architecture.md; the ones that matter to a user of the framework are summarized in the System Overview chapter of this volume.

When to read them. Part 1 and Part 2 of this volume are sufficient for driving RADIANT — writing scripts, writing configs, reading results, interpreting errors. Read Part 3 when you intend to change the framework: add a stage, add a parameter, add a physics term, or write a test that the project will accept. The final chapter, “Extending RADIANT”, is the practical walkthrough those three specifications support.

RADIANT Signal Chain Architecture

11.1 What Is a Chain?

11.1.1 Choice: Ordered pipeline of stages with an immutable, accumulating trace.

Not a free-form DAG. Not bare function composition. A Chain is an ordered list of Stage instances; each stage receives a ChainState and returns a new ChainState. The state is immutable per stage — every stage produces a fresh state that includes all prior state plus its own contributions.

11.1.2 Justification

Why not a DAG:

- A DAG sounds general, but the radiometric signal chain is intrinsically sequential. Source → atmosphere → optics → detector → readout has only one valid topology. The MTF chain, the noise budget, and the spectral propagation all flow in the same direction.
- DAGs require a topological sort, dependency tracking, and a scheduler. None of that earns its keep when the topology is fixed.
- The places where the chain *appears* to branch (multiple source terms summed in MWIR; multiple noise terms in quadrature) are not graph branches — they’re internal sums computed inside one stage.

Why not bare function composition:

- `chain = optics ◦ atmosphere ◦ source` gives no introspection. You can’t ask “what was the at-aperture radiance?” once it’s been pipelined into electrons.
- We need to record every intermediate quantity for the noise budget, MTF budget, and provenance chain. That requires a state object, not just function output.

Why an immutable accumulating trace:

- Every stage adds to the state but never overwrites prior fields. After the chain runs, the state contains a full history: source radiance, at-aperture radiance, post-optics radiance, photoelectrons, noise terms.
- Backward propagation is trivial: just look up the field from the relevant reference frame.
- Reproducibility and explainability are automatic — the state IS the audit record.

11.1.3 Stages in v1

0. GeometryStage – resolves the scene-geometry input mode (ADR-0006); publishes LOS, slant/ground range, solar geometry via `stage_outputs["geometry"]`
1. SourceStage – assembles target/background emission and reflection; classifies regime
2. AtmosphereStage – applies $\tau(\lambda)$, $L_{\text{path}}(\lambda)$, $L_{\text{atm}}(\lambda)$; publishes r_{θ_m} (Fried parameter). It does NOT add a turbulence MTF: the turbulence PSF kernel is applied

- in PlatformStage and the turbulence MTF term is written by PerformanceStage, both when $r0_m > 0$ (see D6 fix, 2026-07)
3. OpticsStage – applies A, Ω , $\tau_{opt}(\lambda)$, warm-optics emission, cold stop, narcissus; adds diffraction MTF, WFE/Strehl, defocus MTF, vignetting, encircled energy
 4. PlatformStage – adds smear MTF and jitter MTF; degrades the EffectivePSF with the jitter, smear, and turbulence kernels; publishes EE_box from the degraded PSF
 5. SpectralIntegrationStage – collapses spectral radiance to in-band photoelectrons per pixel; applies regime-dependent spatial factors (Ω_{pixel} or EE_box)
 6. DetectorStage – applies QE, dark current, full well, generates noise terms; adds pixel aperture MTF, IPC MTF, charge-diffusion MTF
 7. ReadoutStage – applies TDI signal gain, binning, coadds, gain, ADC; adds quantization noise; writes TDI-misregistration MTF and electronics MTF; finalizes noise budget
 8. CalibrationStage – terms-only (Gap 120, ADR-0012): appends post-NUC residual-FPN noise terms AFTER readout's TDI/coadd scaling (structurally exempt from $\sqrt{N}$ averaging) and calibration-scale bias terms to the accuracy budget; no frame, no MTF. scheme="none" (default) is a recorded no-op
 9. PerformanceStage – composes system MTF from all accumulated MTF terms; computes SNR, NEDT, RER, NIIRS, detection range

Spatial and radiometric effects are interleaved through the chain, not separated into parallel tracks. Each stage that has a spatial effect (diffraction, pixel aperture, jitter, IPC, etc.) writes its MTF contribution into `state.mtf_terms` at the same time it writes its radiometric contribution into a reference frame. PerformanceStage reads the accumulated MTF terms at the end and forms the system MTF as their product.

This interleaving is physically required because:

- **Encircled energy couples spatial and radiometric.** In point and sub-pixel regimes, the signal depends on how much PSF energy falls inside the pixel footprint (EE_box). OpticsStage computes the PSF; PlatformStage degrades it (jitter, smear, turbulence) and publishes EE_box from the fully degraded PSF (`stage_outputs["platform"]["EE_box"]`) — evaluated at the pixel sampling phase `detector.pixel_phase_mode` selects (Gap 129; default average), with EE_box_centered and straddle_factor beside it — before SpectralIntegrationStage can compute the photoelectron rate.
- **Regime dispatch depends on PSF size.** The extended/point/sub-pixel classification compares target angular extent to the diffraction PSF diameter — which doesn't exist until OpticsStage has run. SourceStage can tentatively classify based on IFOV alone, but the final regime is confirmed in OpticsStage.
- **Detector spatial response is part of the detector stage.** Pixel aperture MTF, IPC MTF, and charge-diffusion MTF are detector physics, not a separate spatial pass. They belong in DetectorStage.
- **Smear/jitter need integration time.** These MTF terms depend on t_{int} and platform motion and are naturally computed in PlatformStage, which is positioned after OpticsStage but before SpectralIntegration.

11.2 Module Interfaces

11.2.1 The Stage protocol

Every stage implements a single method:

```

@runtime_checkable
class Stage(Protocol):
    @property
    def name(self) -> str: ...

    def run(self, state: ChainState, params: ParameterSet) -> ChainState:
        """Apply this stage to the chain state and return a new state.

        - state: the accumulated chain state from prior stages (may be empty
          for the first stage). Stages MUST NOT mutate the input state.
        - params: the resolved parameter set (configuration; same for all stages).
        - Returns: a new ChainState with this stage's contributions added.
        """
        ...

```

name is exposed as a read-only property rather than a class attribute so concrete stages can satisfy the protocol with either a class-level constant or an instance-resolved value, and @runtime_checkable enables isinstance(obj, Stage) checks in tests and tooling.

11.2.2 What each stage produces and consumes

Stage	Consumes	Produces (radiometric)	Produces (spatial)
GeometryStage	params (geometry.* — altitudes, one viewing-mode entry, one solar-mode entry)	stage_outputs["geometry"]: — los_geometry, θ_o , η , slant/ground range, $\theta_s/\Delta\phi$, ground speed, mode labels (no frames — scene geometry, not radiometry)	
SourceStage	params (target T, ϵ , ρ , area; background; geometry)	$L_{\text{source}}(\lambda)$, target solid angle, tentative regime	—
AtmosphereStage	$L_{\text{source}}(\lambda)$, params (atmosphere model, geometry)	$L_{\text{at_aperture}}(\lambda)$, $\tau_{\text{atm}}(\lambda)$, $L_{\text{path}}(\lambda)$, $L_{\text{atm}}(\lambda)$, $r\theta_m$ (Fried parameter)	— (no MTF written; turbulence PSF kernel is applied in PlatformStage and the turbulence MTF term by PerformanceStage, when $r\theta_m > 0$)
OpticsStage	$L_{\text{at_aperture}}(\lambda)$, params (D, f, ϵ_{obs} , T_{opt} , ϵ_{opt} , η_{cold} , WFE, defocus, vignetting)	$L_{\text{post_optics}}(\lambda)$, A_{collect} , Ω_{pixel} , effective_psf, reference_psf (diffraction-limited reference with the same detector kernels, for PSF-derived Strehl), final regime	MTF_optics(f) — single term from the pupil autocorrelation; WFE and defocus enter via the pupil (Rule 4)
PlatformStage	effective_psf, regime, params (smear velocity, jitter, t_int)	EE_box (ensquared energy in pixel footprint, from the fully degraded PSF, at the selected pixel sampling phase — Gap 129), EE_box_centered, straddle_factor	MTF_smear(f), MTF_jitter(f)
SpectralIntegrationStage	$L_{\text{post_optics}}(\lambda)$, EE_box (from PlatformStage), regime, params (filter, QE, λ -grid, t_int)	In-band photoelectrons per pixel per integration (regime-dependent)	—

Stage	Consumes	Produces (radiometric)	Produces (spatial)
DetectorStage	photoelectrons, params (J_dark, FWC, IPC, glow, L_d, nonlinearity, etc.)	Signal S [e-], noise terms {shot, dark, read, 1/f, kTC, DSNU, PRNU, NUC, glow, persistence, ...}	MTF_pixel(f), MTF_IPC(f), MTF_diffusion(f)
ReadoutStage	signal, noise terms, params (TDI, binning, coadds, gain, ADC)	Signal in DN, full noise budget; quantization noise added	MTF_TDI(f) (TDI mis-registration), MTF_electronics(f)
CalibrationStage	signal level, params (calibration.* scheme, drift, source uncertainty)	Post-NUC residual noise terms (nuc_residual, gain_drift, offset_drift — post-scaling, sqrt(N)-exempt); bias terms (accuracy budget); stage_outputs["calibration"]	— (no MTF: residual FPN is spatial noise, not a spatial degradation — Rule 4 unaffected)
PerformanceStage	full state (all frames, all noise terms, all MTF terms)	SNR, NEDT, RER, NIIRS, detection range; turbulence MTF term (when r0_m > 0)	System MTF = $\prod \text{MTF}_i$

11.2.3 State transformation rules

1. **Stages MUST NOT mutate the input state.** They construct a new state object via `state.with_(. . .)`, which returns a copy with new fields added.
2. **Stages MUST NOT remove fields.** Every field added by an earlier stage remains accessible to later stages and to the user.
3. **Stages MAY add fields under their stage namespace** (e.g., `state.optics.photon_rate`) and **MUST** add radiometric quantities to the appropriate reference frame (`state.frames["at_aperture"]`).
4. **Stages MUST register at least one reference frame** if they transform radiometric quantities (see §5).

11.3 Data Flow: ChainState

11.3.1 Structure

```
@dataclass(frozen=True)
class ChainState:
    # Spectral grid (set once at chain start)
    wavelength_um: np.ndarray # the common grid

    # Reference frames – radiometric quantities at each propagation point.
    # Key is a frame name; value is a RadiometricFrame.
    frames: dict[str, "RadiometricFrame"]

    # Per-stage outputs (namespaced by stage name)
    stage_outputs: dict[str, dict[str, Any]]

    # Noise terms accumulated across stages (each carries its origin frame)
    noise_terms: tuple["NoiseTerm", ...]
```

```

# Bias terms – the accuracy budget (Gap 120, ADR-0012). Consumed ONLY by
# the radiometric-accuracy metric; never RSS'd into sigma_total. The
# noise/bias type separation is what enforces that rule.
bias_terms: tuple["BiasTerm", ...]

# MTF terms accumulated across stages (each stage writes its own terms)
mtf_terms: dict[str, np.ndarray] # term name -> MTF(f) array
spatial_freq_cycles_per_mrad: np.ndarray | None

# Computed performance metrics (populated by PerformanceStage)
metrics: dict[str, float]

# Trace metadata
history: tuple[str, ...] # ordered list of stage names that have run

# Per-run identifier (UUID4); minted by ChainRunner if the caller
# doesn't supply one. Required for the §C13 provenance record.
run_id: str | None

def with_frame(self, frame: "RadiometricFrame") -> "ChainState": ...
def with_stage_output(self, stage: str, key: str, value: Any) -> "ChainState": ...
def with_noise(self, term: "NoiseTerm") -> "ChainState": ...
def with_bias(self, term: "BiasTerm") -> "ChainState": ...
def with_mtf(self, term_name: str, mtf: np.ndarray) -> "ChainState": ...
def with_metric(self, key: str, value: float) -> "ChainState": ...

```

11.3.2 RadiometricFrame

A RadiometricFrame is a snapshot of the radiometric state at one reference point in the chain. It carries the spectral radiance/irradiance at that point plus relevant context (transmission accumulated so far, area, solid angle).

```

@dataclass(frozen=True)
class RadiometricFrame:
    name: str # registered snapshots, e.g. "at_aperture",
              # "post_optics", "photoelectrons"

    wavelength_um: np.ndarray
    spectral_radiance: np.ndarray | None # W/m²/sr/μm – when meaningful
    spectral_irradiance: np.ndarray | None # W/m²/μm – when meaningful
    photon_rate: np.ndarray | None # photons/s/m²/sr/μm or photons/s/pixel/μm
    in_band_value: float | None # post-spectral-integration scalar
    in_band_unit: str
    notes: str = ""

```

Not every frame uses every field — frames upstream of the optics carry radiance; frames at/after the FPA carry photon or electron rates.

11.3.3 Spectral arrays vs. scalars

- **Spectral arrays** (np.ndarray, length = N_wavelengths) flow through the radiance frames. They are stored in the RadiometricFrame and can be inspected at any reference point.

- **Scalars** (signal in electrons, noise in e- RMS, MTF at Nyquist) flow into stage outputs and metrics. The `SpectralIntegrationStage` is the explicit transition point — before it, everything spectral; after it, everything per-pixel scalar.
- Both kinds of data persist for the rest of the chain. A user can call `result.frames["at_aperture"].spectral_radiance` even after the detector stage has run.

11.4 Three Radiometric Regimes

11.4.1 Regimes

Regime	Condition	Signal equation
Extended	Target angular extent $\gg$ IFOV (target fills the pixel and overflows)	$S = L_{\text{target}} \times A_{\text{collect}} \times \Omega_{\text{pixel}} \times \tau_{\text{atm}} \times \tau_{\text{opt}} \times QE \times t_{\text{int}}$ — EE_box does not appear because every photon lost to a neighbor pixel is replaced by an identical photon from the neighboring scene, so the per-pixel radiance is preserved.
Point	Target angular extent $\ll$ PSF (point source unresolved)	$S = (I_{\text{target}} / R^2) \times A_{\text{collect}} \times \tau_{\text{atm}} \times \tau_{\text{opt}} \times QE \times \text{EE_box} \times t_{\text{int}}$ where $I = L \times A_{\text{target}}$. EE_box is a direct multiplicative factor on the signal: only the fraction of PSF energy inside the pixel footprint contributes. The compact target sits on a full-pixel background pedestal (<code>background_e</code> , computed over Ω_{pixel} from the at-aperture background frame — Gap 73): this is <i>not</i> part of the target signal (<code>contrast_e = signal_e</code>) but is real photo-charge that contributes background shot noise and fills the well. It is the same full-pixel formula as the sub-pixel/extended background reference, so the noise budget is continuous across the sub-pixel→point-source boundary. When no background frame is configured, <code>background_e = 0</code> (Decision #13).
Sub-pixel	Target angular extent $\sim$ IFOV (target smaller than pixel but not point-like)	Per-pixel signal = (target contribution) + (background contribution), where the target contribution scales with fill fraction $\eta \times \text{EE_box}$ and the background contribution is the extended-scene formula over $(1 - \eta)$ of the pixel. EE_box applies to the target term because the target is sub-PSF; it does not apply to the background because the background is locally extended.

11.4.2 Dispatch: auto-detect with override

The framework auto-detects the regime in **two steps** (§ “Architectural placement” below). Both steps compare the target angular extent $\sqrt{A_{\text{target}}}/R$ against a reference scale, but the reference differs:

Step 1 — SourceStage (tentative), against the IFOV (`core/regime.py` constants):

```
angular_extent_rad = sqrt(A_target) / R          # small-angle approximation
ifov_rad = pixel_pitch / focal_length
```

```
# REGIME_EXTENDED_IFOV_MULTIPLE = 2.0, REGIME_POINT_SOURCE_IFOV_MULTIPLE = 0.25
if angular_extent_rad >= 2.0 * ifov_rad: regime = EXTENDED
elif angular_extent_rad <= 0.25 * ifov_rad: regime = POINT_SOURCE
else: regime = SUB_PIXEL
```

Step 2 — OpticsStage (final), against the diffraction PSF FWHM (optics/stage.py constants). After the PSF is computed, `psf_fwhm_rad = fwhm_m / focal_length`:

```
# _EXTENDED_PSF_FWHM_MULTIPLE = 2.0, _POINT_SOURCE_PSF_FWHM_MULTIPLE = 0.5
if angular_extent_rad >= 2.0 * psf_fwhm_rad: regime = EXTENDED
elif angular_extent_rad <= 0.5 * psf_fwhm_rad: regime = POINT_SOURCE
else: regime = SUB_PIXEL
```

The final classifier uses the **PSF FWHM**, not the Airy first-dark-ring diameter $2.44 \lambda/D$. When in doubt, the framework picks SUB_PIXEL — the most general regime, which reduces correctly to the other two at the limits.

The classifier (optics/stage.py::`_finalize_regime`) is the **only** mechanism that sets the regime, and it never warns or raises. It must not be confused with `optics/stage.py::_validate_psf_regime_consistency`, the matrix §7 validity guard, which reads the *declared* `scene_type` against the separate `0.1 · PSF_FWHM / 0.01 · PSF_FWHM` thresholds, raises `ParameterBoundsError`, and never touches the regime (CU-298; thresholds and rationale in RADIANT_Use_Case_Matrix.md §1.1).

11.4.3 User override

The user may force a regime via the parameter `source.regime_override` (default "auto"):

Value	Meaning
"auto" (default)	Use the two-step detection rule above
"extended"	Force extended-scene equation
"point_source"	Force point-source equation
"sub_pixel"	Force sub-pixel equation

An override that conflicts with the geometry (e.g. forcing `point_source` on a target several PSF-FWHM across) logs a `UserWarning` from `OpticsStage` but is honored.

11.4.4 Architectural placement

Regime classification happens in **two steps**:

1. **SourceStage** makes a *tentative* classification based on target angular extent vs. IFOV only. This is enough to route atmosphere-side choices. Stored in `state.stage_outputs["source"]["regime_tentative"]`.
2. **OpticsStage** finalizes the regime after computing the diffraction PSF. It compares target angular extent against the actual PSF FWHM and writes `state.stage_outputs["optics"]["regime"]`. This is the authoritative regime for all downstream stages.

SpectralIntegrationStage reads the final regime and the PSF-derived **EE_box** from PlatformStage (`stage_outputs["platform"]["EE_box"]`), computed from the fully degraded PSF — jitter, smear, and turbulence

included), then applies the regime-appropriate formula. This is the single point where EE_box enters the radiometric calculation. No other stage multiplies by EE_box.

The three regime equations differ only in **which spatial factors multiply the photoelectron integral**. The atmospheric transmission and optical throughput are identical across regimes. Encircled energy EE_box is the critical spatial→radiometric coupling and is applied exactly once, at SpectralIntegrationStage.

11.4.5 Sub-pixel as the general case

The sub-pixel equation degenerates correctly:

- $\eta = 1 \rightarrow$ extended-scene equation
- $A_{\text{target}} \rightarrow 0$, $\eta \rightarrow 0$, but the *contrast* term (target – background) is preserved by switching to the point-source intensity formulation

For the implementation, we keep two code paths:

1. **Resolved/extended path** ($\eta \geq$ some threshold): area-weighted radiance integral
2. **Point-source path** (η below threshold): intensity / $R^2 \times$ encircled energy

The “subpixel” auto-classification routes to whichever path is more numerically stable for the actual target size.

11.5 Backward Propagation: Reference Frames

11.5.1 The reference frame registry

A user must be able to ask: “*What is the noise referred to the aperture?*” or “*What is the signal in DN?*” The architecture answers this via the ReferenceFrame enum (core/quantity.py), whose members are the canonical **query positions** for signal_at / noise_at:

Frame (ReferenceFrame)	Position	Units
at_target	Source emission, before atmosphere	W/m ² /sr/μm
at_aperture	After atmosphere, at the entrance pupil	W/m ² /sr/μm
post_optics	After optical throughput, before detector	W/m ² /sr/μm
photoelectrons	After QE and integration time, before readout	e- (per pixel per integration)
post_readout	After TDI/coadds/gain	e- (per output sample)
dn	Digitized output	DN

These six enum positions are the *query* frames of the conversion machinery, and are **not** identical to the RadiometricFrame snapshots actually stored in state.frames (the atmosphere registers at_source_target / at_source_target_reflected / at_aperture_target / at_aperture / at_aperture_background / at_source_background, optics registers post_optics, spectral integration registers photoelectrons; source/detector/readout register none. The at_source_* frames (Gap 91) are the pre-atmosphere source emission L_source — the emitted+reflected radiance leaving the target/background before the up-leg, satisfying $at_aperture_target \approx \tau_{up} \cdot at_source_target + L_{path_up}$; they

are additive diagnostics that feed no downstream metric. `at_source_target_reflected` is likewise additive: the ρ -proportional part of `at_source_target` — direct solar plus diffuse sky, without the self-emission — published so the reflective GUI view can pair the reflectance with the radiance it produces). In particular there is **no `at_fpa` frame** — the focal-plane irradiance lives only as the optics stage-outputs `nearfield_irradiance_at_fpa` / `stray_light_irradiance_at_fpa` — and `at_target` is a query position reachable through the τ_{atm} transfer factor, not a stored snapshot. The full mechanism (transfer-factor extraction, backward propagation, the saturated-well 1/gain fallback) is specified in **RADIANT_Reference_Frames.md**.

11.5.2 Forward propagation (signal)

The conversion factors between adjacent frames are extracted from stage_outputs **at query time** by `_compute_transfer_factors` (`core/quantity.py`) — recomputed on each `ChainQuantity.to()` call from the stored `tau_atm`, `tau_opt`, `signal_e`, `signal_e_final`, `signal_dn_final`, and `gain_e_per_dn` outputs, not cached under a dedicated `forward_factor` key. The full forward chain for an **extended scene** is:

<code>L_at_target(λ)</code>	[W/m ² /sr/ μ m]
× $\tau_{\text{atm}}(\lambda)$ (+ <code>L_path</code> , <code>L_atm</code> added)	→ <code>at_aperture</code>
× <code>A_collect</code> × Q_{pixel} × $\tau_{\text{opt}}(\lambda)$	→ <code>post_optics</code> (now W/pixel/ μ m)
× (λ / <code>hc</code>)	→ photons/s/pixel/ μ m
× <code>QE</code> (λ)	→ e-/s/pixel/ μ m
$\int d\lambda$ over filter bandpass	→ e-/s/pixel (<code>in_band</code>)
× <code>t_int</code>	→ photoelectrons (per integration)
× <code>N_TDI</code> × <code>N_coadds</code>	→ <code>post_readout</code>
÷ <code>gain</code>	→ DN

For a **point source**, insert the regime-specific spatial factor at `SpectralIntegrationStage` (`EE_box` supplied by `PlatformStage`):

<code>I_target / R²</code>	[W/m ² / μ m, per wavelength]
× $\tau_{\text{atm}}(\lambda)$ × <code>A_collect</code> × $\tau_{\text{opt}}(\lambda)$	→ photon flux at FPA
× <code>EE_box</code>	→ fraction landing in target pixel
× (λ /hc) × <code>QE</code> (λ), $\int d\lambda$, × <code>t_int</code>	→ electrons in target pixel

EE_box enters exactly once, applied to the target photon flux in the point-source and sub-pixel-target branches. It does not apply to the background term in the sub-pixel case. It does not apply at all in the extended-scene case (each photon displaced out of pixel (i,j) is replaced by a statistically identical photon from pixel (i±1, j±1)).

The full path is the product of all factors. The conversion from any frame A to any frame B is the product (or inverse product) of the factors between them.

11.5.3 Backward propagation (noise)

Noise is generated at specific points in the chain (shot noise after spectral integration, dark noise at the detector, read noise at the readout, quantization at the ADC) and must be referable to any frame.

Each noise term carries an **origin frame** — the frame in which it was generated. To express it in another frame, multiply (forward) or divide (backward) by the appropriate forward factors.

```
def noise_at(
```

```

    state: ChainState,
    target_frame: ReferenceFrame,
    term_name: str | None = None,
) -> ChainQuantity:
    """Get noise (total or a specific term) at a target reference frame.

    If ``term_name`` is None, the total noise is the RSS of all terms whose
    origin_frame matches; mixed-origin RSS raises ValueError. Conversion
    between frames is handled by ChainQuantity.to(target_frame, state).
    """
    ...

```

11.5.4 NoiseTerm

```

@dataclass(frozen=True)
class NoiseTerm:
    name: str          # "shot", "dark", "read", "1/f", "quantization", ...
    value_e: float     #  $\sigma$  in electrons RMS at origin_frame
    origin_frame: str   # one of the registered frame names
    physical_basis: str # "Poisson", "thermal", "1/f", "ADC LSB/sqrt(12)", ...
    contributes_to: tuple[str, ...] = ("total",) # which budgets to sum into

```

11.5.5 Public query API

```

result = chain.run(params)

# Forward propagation
result.signal_at("photoelectrons") # 12,450 e- (valid frame; "electrons" is not one)
result.signal_at("dn")              # 124.5 DN
result.signal_at("at_aperture")     # spectral radiance returned

# Backward propagation
result.noise_at("photoelectrons")  # 263 e- RMS total
result.noise_at("dn")              # 2.63 DN RMS total
result.noise_at("photoelectrons", term_name="dark") # 89.2 e- (just one term)
# result.noise_budget() is NOT a method (Appendix A) – use result.noise_terms
# (tuple of NoiseTerm) or inspect_result(result) for a formatted breakdown.

# Spatial – NOTE: these convenience methods are NOT on ChainResult (see
# RADIANT_Scripting_API.md Appendix A). Use the shipped equivalents:
result.metrics["mtf_at_nyquist"]   # 0.42
result.state.mtf_terms["mtf_optics_x"] # individual MTF(f) arrays
result.stage_outputs["performance"]["mtf_budget"] # MTF budget

# Performance – snr()/nedt()/niirs() ARE real methods on ChainResult
result.snr()                       # 47.3
result.nedt()                      # 23 mK
result.niirs()                     # 5.4

```

11.6 Non-Radiometric Information

Geometry, timing, and configuration do **not** flow through the chain state. They live in the `ParameterSet` and are passed alongside the state to every stage:

```
def run(self, state: ChainState, params: ParameterSet) -> ChainState:
    aperture = params.get("optics.aperture_diameter_m") # m
    ...
```

This is a deliberate separation:

- `ChainState` carries **physical quantities flowing through the chain** (spectral radiance, photoelectrons, MTF curves).
- `ParameterSet` carries **configuration that drives those physics** (aperture diameter, integration time, look angle).

Stages may read from both, but only write to a new `ChainState`. They never modify the parameter set.

11.6.1 Why not put geometry in the state?

It was tempting. But geometry is constant for the duration of one chain run — it’s a configuration input, not a propagating quantity. Putting it in `ChainState` would blur the line between “what is flowing” and “what is configured.” Worse, it would invite stages to derive geometry on the fly, which violates the “compute once, in one place” rule.

11.6.2 Inter-stage non-radiometric communication

Some stages need to publish metadata that downstream stages consume but that isn’t a radiometric quantity. Examples:

- `SourceStage` publishes the **regime** (“extended” / “point” / “subpixel”)
- `SourceStage` publishes the **target solid angle**
- `OpticsStage` publishes **A_collect**, **Ω_{pixel}** , the **effective_psf** and the diffraction-limited **reference_psf**
- `PlatformStage` publishes **EE_box** (ensquared energy in the pixel box, from the fully degraded PSF)
- `PerformanceStage` publishes the **system MTF** used in its NIIRS computation

These go into `state.stage_outputs[stage_name]` as a namespaced dict. Downstream stages access them via `state.stage_outputs["source"]["regime"]`. This is read-only by convention.

11.7 Concrete Python Interfaces

```
# src/radiant/core/chain.py – Stage protocol
from typing import Protocol, runtime_checkable

@runtime_checkable
class Stage(Protocol):
    @property
    def name(self) -> str: ...
    def run(self, state: "ChainState", params: "ParameterSet") -> "ChainState": ...
```

```

# src/radiant/core/radiometry.py - RadiometricFrame, NoiseTerm
from dataclasses import dataclass, field, replace
from typing import Any
import numpy as np

@dataclass(frozen=True)
class RadiometricFrame:
    name: str
    wavelength_um: np.ndarray
    spectral_radiance: np.ndarray | None = None
    spectral_irradiance: np.ndarray | None = None
    photon_rate: np.ndarray | None = None
    in_band_value: float | None = None
    in_band_unit: str = ""
    notes: str = ""

@dataclass(frozen=True)
class NoiseTerm:
    name: str
    value_e: float
    origin_frame: str
    physical_basis: str
    contributes_to: tuple[str, ...] = ("total",)

# src/radiant/core/chain.py - ChainState
# All Mapping fields are wrapped in types.MappingProxyType at construction
# (__post_init__), so direct mutation raises TypeError. Stages only ever
# call the with_* helpers, which return a NEW state.
@dataclass(frozen=True)
class ChainState:
    wavelength_um: np.ndarray
    frames: Mapping[str, RadiometricFrame] = field(default_factory=dict)
    stage_outputs: Mapping[str, Mapping[str, Any]] = field(default_factory=dict)
    noise_terms: tuple[NoiseTerm, ...] = ()
    bias_terms: tuple[BiasTerm, ...] = () # accuracy budget (Gap 120, ADR-0012)
    mtf_terms: Mapping[str, np.ndarray] = field(default_factory=dict)
    spatial_freq_cycles_per_mrad: np.ndarray | None = None
    metrics: Mapping[str, float] = field(default_factory=dict)
    history: tuple[str, ...] = ()
    run_id: str | None = None # UUID4; minted by ChainRunner per §C13

    def with_frame(self, frame: RadiometricFrame) -> "ChainState":
        new = dict(self.frames)
        new[frame.name] = frame
        return replace(self, frames=new)

    def with_stage_output(self, stage: str, key: str, value: Any) -> "ChainState":
        new = {k: dict(v) for k, v in self.stage_outputs.items()}

```

```

        new.setdefault(stage, {})[key] = value
        return replace(self, stage_outputs=new)

def with_noise(self, term: NoiseTerm) -> "ChainState":
    return replace(self, noise_terms=self.noise_terms + (term,))

def with_bias(self, term: BiasTerm) -> "ChainState":
    # Accuracy budget (Gap 120): consumed only by the radiometric-accuracy
    # metric – never by SNR/NEDT.
    return replace(self, bias_terms=self.bias_terms + (term,))

def with_mtf(self, term_name: str, mtf: np.ndarray) -> "ChainState":
    new = dict(self.mtf_terms)
    new[term_name] = mtf
    return replace(self, mtf_terms=new)

def with_spatial_freq(self, freq_cycles_per_mrad: np.ndarray) -> "ChainState":
    return replace(self, spatial_freq_cycles_per_mrad=freq_cycles_per_mrad)

def with_metric(self, key: str, value: float) -> "ChainState":
    new = dict(self.metrics)
    new[key] = value
    return replace(self, metrics=new)

def with_history(self, stage_name: str) -> "ChainState":
    return replace(self, history=self.history + (stage_name,))

# src/radiant/core/chain.py – ChainRunner
class ChainRunner:
    def __init__(self, stages: list[Stage]) -> None:
        # Validates stage-name uniqueness at construction.
        self._stages = list(stages)

    def run(
        self,
        params: ParameterSet,
        wavelength_um: np.ndarray,
        *,
        run_id: str | None = None,
        initial_stage_outputs: Mapping[str, Mapping[str, Any]] | None = None,
    ) -> ChainState:
        state = ChainState(wavelength_um=wavelength_um, run_id=run_id or new_run_id())
        # Rule 6 hook: pre-chain injection of file-derived objects. The
        # IO/API layer loads files and seeds them into stage_outputs
        # before the first stage runs – e.g. the atmosphere model under
        # "atmosphere_config", the optical element list under
        # "optics_config" – so stages never read files themselves.
        if initial_stage_outputs is not None:
            for namespace, outputs in initial_stage_outputs.items():
                for key, value in outputs.items():
                    state = state.with_stage_output(namespace, key, value)

```

```

    for stage in self._stages:
        state = stage.run(state, params)
        # History auto-recorded if the stage didn't self-record.
    return state

# src/radiant/io/results.py - ChainResult
# ChainRunner.run returns the raw ChainState; the API layer
# (api/session.py) wraps it in a ChainResult for user-facing queries.
class ChainResult:
    def __init__(self, state: ChainState, params: ParameterSet | None = None) -> None:
        self._state = state
        self._params = params

    # --- Raw state access ---
    @property
    def state(self) -> ChainState: ...
    @property
    def frames(self) -> Mapping[str, RadiometricFrame]: ...
    @property
    def noise_terms(self) -> tuple[NoiseTerm, ...]: ...
    @property
    def stage_outputs(self) -> Mapping[str, Mapping[str, Any]]: ...
    @property
    def history(self) -> tuple[str, ...]: ...
    @property
    def metrics(self) -> Mapping[str, float]: ...

    # --- Forward / backward queries (ChainQuantity carries value + unit + frame) ---
    def signal_at(self, frame: ReferenceFrame | str) -> ChainQuantity: ...
    def noise_at(
        self,
        frame: ReferenceFrame | str,
        term_name: str | None = None,
    ) -> ChainQuantity: ...

    # --- Performance metrics ---
    def snr(self) -> float: ...
    def nedt(self) -> float: ...
    def niirs(self) -> float: ...

    # --- Full-well saturation status (CU-101; WellStatus carries the
    # clip state + supporting well-charge numbers, each with units) ---
    def well_status(self) -> WellStatus: ...

    # --- Provenance ---
    def to_provenance_record(self) -> dict: ...

```

11.8 Worked Example: Full Forward Pass

A staring MWIR sensor observing an extended thermal scene.

11.8.1 Inputs (resolved ParameterSet)

- $D = 0.30$ m, $f = 1.20$ m, $\epsilon_{\text{obs}} = 0.33$
- pixel pitch = $18\text{ }\mu\text{m}$, $\lambda_c = 4.2\text{ }\mu\text{m}$, $\Delta\lambda = 0.5\text{ }\mu\text{m}$
- $T_{\text{target}} = 300$ K, $\epsilon_{\text{target}} = 0.95$, A_{target} large (extended)
- $T_{\text{optics}} = 280$ K, $\eta_{\text{cold}} = 0.9$
- $t_{\text{int}} = 5$ ms, $QE_{\text{peak}} = 0.75$
- MODTRAN file loaded $\rightarrow \tau_{\text{atm}}(\lambda)$, $L_{\text{path}}(\lambda)$, $L_{\text{atm}}(\lambda)$

11.8.2 Stage execution

1. **SourceStage** computes $L_{\text{source}}(\lambda) = \epsilon \times \text{Planck}(T, \lambda)$. Registers no RadiometricFrame (the pre-atmosphere `at_source_*` snapshots are added by AtmosphereStage — see §5). Tentative regime = extended (target angular size $\gg$ IFOV). Stores tentative regime + target solid angle in `state.stage_outputs["source"]`.
2. **AtmosphereStage** computes $L_{\text{at_aperture}}(\lambda) = L_{\text{source}}(\lambda) \times \tau_{\text{atm}}(\lambda) + L_{\text{path}}(\lambda) + L_{\text{atm}}(\lambda)$. Registers the `at_source_*` / `at_aperture_*` / `at_aperture` frames. Publishes `r0_m`; for this space scenario `r0_m` is unset (vacuum path), so no turbulence PSF kernel or MTF term is produced downstream.
3. **OpticsStage** computes $L_{\text{post_optics}}(\lambda) = L_{\text{at_aperture}}(\lambda) \times \tau_{\text{opt}}(\lambda) + L_{\text{warm_optics}}(\lambda) \times (1 - \eta_{\text{cold}})$. Adds `post_optics` frame. Stores `A_collect`, `$\Omega_{\text{pixel}}$` , computes the `effective_psf` and `diffraction-limited reference_psf`, finalizes regime = extended, writes `MTF_optics` (pupil autocorrelation) into `state.mtf_terms`.
4. **PlatformStage** writes `MTF_smear` (sinc from $v_{\text{img}} \times t_{\text{int}}$), `MTF_jitter` (Gaussian from σ_{jitter}) into `state.mtf_terms`, degrades the PSF accordingly, and publishes `EE_box` from the fully degraded PSF (`stage_outputs["platform"]`["EE_box"]; 1.0 in the extended regime).
5. **SpectralIntegrationStage** reads regime = extended $\rightarrow$ does not apply `EE_box`. Computes per-pixel photon rate by integrating $L \times A \times \Omega \times \tau \times QE \times \lambda/hc$ over the filter bandpass. Adds the photoelectrons frame (after $\times t_{\text{int}}$). There is no `at_fpa` frame — the focal-plane irradiance lives only as the optics stage-outputs `nearfield_irradiance_at_fpa` / `stray_light_irradiance_at_fpa` (§5).
6. **DetectorStage** generates noise terms: shot, dark shot, read, $1/f$, kTC (off because CDS=on), DSNU residual, PRNU residual, NUC residual, IPC, glow, persistence. Writes `MTF_pixel`, `MTF_IPC`, `MTF_diffusion` into `state.mtf_terms`.
7. **ReadoutStage** applies gain, ADC; adds quantization noise. Updates noise terms for TDI/coadds (each multiplied by $\sqrt{N_{\text{TDI}}} \cdot \sqrt{N_{\text{coadd}}}$ in the appropriate direction).
8. **PerformanceStage** forms system $MTF = \prod MTF_i$ over all accumulated terms. Computes $SNR = \text{signal} / \sigma_{\text{total}}$, NEDT from dS/dT , RER from system MTF integrated over the edge spread function, NIIRS from GIQE/IIRS using $RER + SNR + GSD + Q$.

11.8.3 Backward query

A user wants to know the noise referred to the aperture in $W/m^2/sr/\mu m$:

- `result.noise_at("at_aperture")`
 - The framework looks up the total noise (σ_{total} in electrons), traces backward through the forward factors stored in stage outputs, and returns the equivalent radiance noise.
-

11.9 Architectural Invariants

These hold for every chain run:

1. **Stages are pure functions of (state, params).** No I/O, no global state, no mutation.
2. **Reference frames accumulate.** Once added, a frame is never removed or modified.
3. **Noise terms accumulate.** Each carries its origin frame; conversion between frames is computed at query time, not stored.
4. **Spectral integration happens exactly once,** in `SpectralIntegrationStage`. Before it: spectral arrays. After it: scalars.
5. **The MTF chain and the radiometric chain are interleaved, not independent.** Spatial effects (diffraction, pixel MTF, IPC, jitter, smear) are computed in the same stages that produce their radiometric counterparts. The two chains are coupled through `EE_box` (ensquared energy in the pixel footprint), a spatial quantity computed in `PlatformStage` from the fully degraded PSF (jitter, smear, turbulence included) that multiplies the radiometric signal in the point-source and sub-pixel regimes. `PerformanceStage` combines the accumulated MTF terms into the system MTF but does not “join” two separate chains.
6. **Regime is tentatively classified in `SourceStage` and finalized in `OpticsStage`** after the diffraction PSF is computed. The final regime lives in `stage_outputs["optics"]["regime"]`.
7. **`EE_box` is applied exactly once,** in `SpectralIntegrationStage`, and only in point-source and sub-pixel regimes. It is the single architectural coupling between the spatial PSF and the radiometric signal.
8. **Geometry and configuration are read from `ParameterSet`, never written.** Stages may read; only the resolver writes.

RADIANT Parameter System Design

12.1 Critical Decision: Spectral Arrays

12.1.1 Choice: (c) Hybrid

Scalar parameters go through the full parameter system (named, typed, validated, toleranced, dependency-tracked, provenanced). Spectral arrays — wavelength-dependent quantities like $QE(\lambda)$, $\tau_{atm}(\lambda)$, $T_{filter}(\lambda)$ — live in a parallel **SpectralDataStore**, not in the scalar parameter resolver.

12.1.2 Justification

Why not (a) — treat spectral arrays as parameters:

- A $QE(\lambda)$ curve has 500–5000 points. Putting each through the resolver with validation, tolerance, provenance tracking is absurd overhead.
- Tolerance on a spectral curve is ambiguous: do you perturb each point independently? Shift the curve? Scale it? The answer depends on the physics (QE tolerance is a peak-QE uncertainty + cutoff-wavelength uncertainty — both scalar).
- Users don't "set" $QE(\lambda)$ point by point. They select a detector material or load a file. The scalar parameter is the material name or file path; the array is derived.

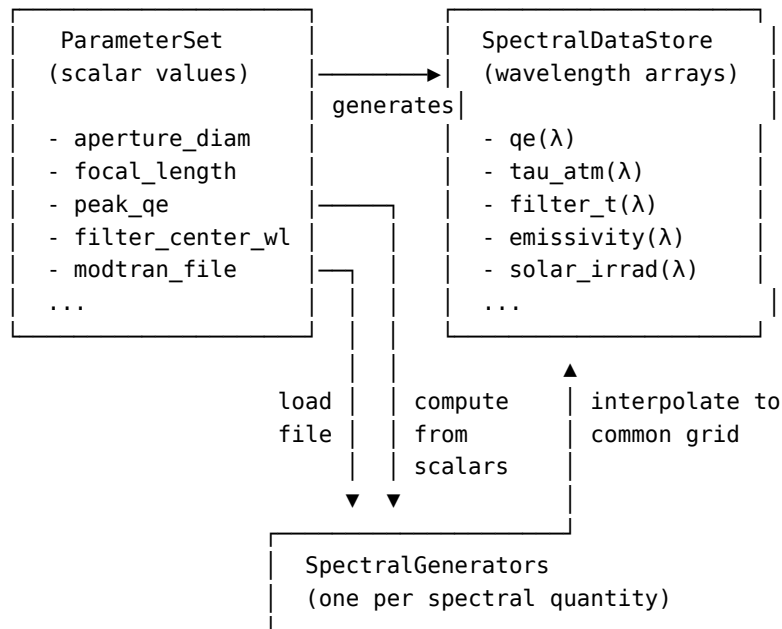
Why not (b) — scalars only, arrays in computation layer:

- Some spectral data IS user input (a MODTRAN tape7 file, a measured QE curve). It needs to be loaded, validated, and tracked — it can't just appear inside a computation module with no provenance.
- Filter transmission is defined by scalar parameters (λ_c , $\Delta\lambda$, T_{peak} , OOB_rejection) but consumed as an array $T_{filter}(\lambda)$. There must be a defined place where this computation happens.

Why (c) — hybrid:

- Clean separation of concerns: scalar parameters are *configuration*, spectral data is *physical state*.
- Many spectral arrays are computed from scalar parameters: QE from (material, cutoff_wavelength, peak_qe); filter from (λ_c , $\Delta\lambda$, T_{peak} , shape); emissivity from (material_name). The scalars go through the resolver; the arrays are generated downstream.
- Some spectral arrays are loaded from files (MODTRAN output, measured data). The file path is a parameter; the loaded data goes into the SpectralDataStore.
- All spectral data is interpolated onto a common wavelength grid before any physics computation. The grid definition (λ_{min} , λ_{max} , N_{points}) is itself a set of scalar parameters.
- Tolerances are clean: perturb the scalar parameters (peak_QE $\pm 5\%$, cutoff_wavelength $\pm 0.1 \mu m$), regenerate the spectral curve. No ambiguity.

12.1.3 Architecture



12.2 Parameter Class Design

12.2.1 ParameterDef — Schema Definition

Every parameter in RADIANT is defined once in a schema. The schema is the single source of truth for what parameters exist, what they mean, and what values are legal.

```

@dataclass(frozen=True)
class ParameterDef:
    name: str # Dot-path: "optics.aperture_diameter_m"
    description: str # Dense, specific: "Entrance pupil diameter"
    dtype: type # float, int, str, bool
    canonical_unit: str # Internal unit per Conventions: "m", "rad", "s", "e-/s"
    input_unit: str # User-facing unit: "m", "deg", "urad", "ms"
    default: Any | None = None # Default value in input_unit; None means required
    bounds: tuple[float, float] | None = None # (min, max) in input_unit
    enum_values: tuple[str, ...] | None = None # For categorical (dtype=str only)
    group: str | None = None # Consistency group name: "fnumber"
    tags: frozenset[str] = frozenset() # Metadata: {"detector", "noise", "mwir"}
    default_justification: str = "" # One-line rationale for a non-obvious default
    deprecated_aliases: frozenset[str] = frozenset() # Old names (renames), warn +
    # redirect

    required_unless: str | None = None # Alternative param that supersedes this one
    # (Gap 66)

    is_file_path: bool = False # str value names a data file – stored portably
    # (CU-177)
  
```

Key properties:

- `frozen=True`: definitions are immutable after creation.
- bounds are in `input_unit`, not `canonical_unit`. The user thinks in input units; validation should too.
- `default=None` means the parameter is **required** — the resolver will error if it's not provided.
- tags enable filtering (“show me all detector parameters”, “what parameters matter in LWIR?”).
- `deprecated_aliases` (Gap 12): old dot-paths for renamed parameters. `set/get/set_tolerance/clear_input` resolve an alias to the canonical name with a `DeprecationWarning`. Aliases may not collide with defined names and are validated at `ParameterSet` construction. Current aliases: the `source.target.*` → `geometry.target.*` `set` and `platform.h_sensor` → `geometry.sensor_altitude_m` (ADR-0006). Distinct from **removed** parameters: `radiant.core.parameters.REMOVED_PARAMETERS` maps a dot-path a model change *deleted* to the guidance an operator needs, and `ParameterSet._suggest` answers with that text instead of a did-you-mean list — a deleted name is a harder error than a typo, because the config is not misspelled, it encodes a model RADIANT no longer has. Current entries: `optics.nearfield_fraction`, `optics.cold_stop_efficiency`, `optics.optics_distance_to_fpa_m` (all Gap 128).
- tags regime-relevance convention (Gap 85, 2026-07-16): a tag of the form `regime:<scene_type>` (`regime:extended` / `regime:sub_pixel` / `regime:point_source`) declares that the parameter matters only for those **declared** scene types (`source.scene_type`). A def with no `regime:` tag is regime-independent. Consumed by the GUI Source form, which disables (never hides) irrelevant rows with an explanatory tooltip when a scene type is declared; auto gates nothing. Currently authored on the source-stage background / contrast-reference / fill-fraction parameters; other stages are unauthored (relevance there is regime-independent until tagged).
- `required_unless` (Gap 66; comma-list since Gap 69): names one or more comma-separated alternative parameters, any of which supersedes this required one. When the alternative is explicitly set (non-empty, non-None input), the requirement is waived and the parameter is left **unresolved** — `get()` raises if any code path reads it anyway, so no phantom value is ever consumed. An explicitly-set empty string does not waive the requirement. Only valid on required (`default=None`) parameters. Current use: `detector.qe_value` is required unless **either** `detector.qe_table_path` **or** `detector.qe_material` is set (`required_unless="detector.qe_table_path,detector.qe_material", detector/_schema.py`) — a spectral QE curve (from file or the bundled library) supersedes the scalar.
- `is_file_path` (CU-177): marks a `dtype=str` parameter whose value names a data file on disk (a CSV of $\epsilon(\lambda)/\rho(\lambda)/L(\lambda)$, a QE table, a Zernike file, a tabulated-atmosphere file). **Serialization stores such a value relative to the output YAML's directory** (`save_config/Sensor.save/Sensor.to_yaml(relative_to=...)`), and **loading resolves it back to absolute against the source YAML's directory** (`load_config` when the source is a file). A config that references a repo-internal data file therefore stays portable across checkout locations, machines, and OSes (relative paths are written forward-slashed, Rule 30); configs written before CU-177 with absolute paths still load unchanged. Not set for system/environment paths that are **not** part of the portable config surface — `atmosphere.modtran.binary_path`, `atmosphere.modtran.cache_dir`, `atmosphere.interpolated_data_dir`, and the staged MODTRAN `tape7_*/flux_path` files, which stay verbatim. A relative path loaded from a bare dict (no file anchor) is left as-is.

12.2.2 Unit Conversion

Units are **not** runtime types (no `astropy.units`). The Conventions document locks all canonical units. Each `ParameterDef` carries a conversion factor computed once at schema load:

```
# Precomputed conversion table
UNIT_CONVERSIONS = {
    ("deg", "rad"): math.pi / 180,
    ("urad", "rad"): 1e-6,
    ("ms", "s"): 1e-3,
    ("us", "s"): 1e-6,
    ("cm", "m"): 1e-2,
    ("km", "m"): 1e3,
    ("mm", "m"): 1e-3,
    ("K", "K"): 1.0,          # identity
    ("m", "m"): 1.0,
    ("s", "s"): 1.0,
    ("rad", "rad"): 1.0,
    ("e-", "e-"): 1.0,
    ("e-/s", "e-/s"): 1.0,
    # ...
}

def convert(value, from_unit, to_unit):
    if from_unit == to_unit:
        return value
    return value * UNIT_CONVERSIONS[(from_unit, to_unit)]
```

Conversion happens exactly once: on `set()`. Internally, everything is in canonical units. On `get()` for display, convert back to input units.

Unit-aware input (Gap 6). `ParameterSet.set(name, value, unit=...)` (and `Sensor.set(dotpath, value, unit=...)`) accepts the caller's native unit and converts at the set boundary — the only place user-unit conversion happens (Rule 2). The value is converted `unit` → `canonical` → `input_unit` through the registered conversion table, then follows the normal input-unit validation path (bounds are checked after conversion, so `set("detector.qe_value", 150, unit="%")` fails the `[0, 1]` bounds check as 1.5). Percent ("% → fraction) and `min` → `s` are registered for this purpose. Unknown units raise an actionable `ValueError` naming the parameter's native unit. The original value and unit are recorded in the resolved provenance source string. Without `unit=`, values are taken in `input_unit` (historical behavior, unchanged).

12.3 Parameter Naming Convention

12.3.1 Dot-path namespaces

Parameters are organized hierarchically by dot-separated namespaces:

```
optics.aperture_diameter_m      # m
optics.focal_length_m          # m
optics.f_number                 # dimensionless
```

```

optics.obscuration_ratio      # dimensionless (0–1)
optics.wfe_rms_waves          # waves (at optics.wfe_reference_wavelength_um)
optics.zernike_file            # path – Zemax 'Zernike Standard Coefficients'
                                # export; loaded pre-chain (Rule 6), injects a
                                # ZERNIKE WavefrontError that supersedes the
                                # scalar WFE; report wavelength honored,
                                # wfe_reference_wavelength_um is the no-header
                                # fallback
optics.transmission_scalar    # dimensionless (0–1); the scalar lump never emits
                                # (Gap 127)
optics.defocus_um             # μm
optics.surface_roughness_nm    # nm, effective train RMS roughness (TIS scatter;
                                # 0 = off)
optics.scatter_halo_sigma_um  # μm, Gaussian scatter-halo width on the focal
                                # plane
optics.cold_stop_undersize_frac # dimensionless (0–0.49); fractional
                                # pupil-DIAMETER reduction the cold stop
                                # imposes:  $D_{\text{eff}} = (1 - u) \cdot D$ . The cold stop is
                                # the aperture stop (Gap 128); the effective
                                # pupil feeds A_collect, f/#, the PSF/MTF, and
                                # Ω_cone together
optics.cold_stop_obscuration_ratio # dimensionless (0–0.99); obscuration the cold
                                # shield imposes; effective obscuration =
                                # max(this, optics.obscuration_ratio)
optics.stray.veiling_glare_fraction # dimensionless (0–1)
optics.stray.veiling_glare_mtf      # int 0/1; 1 = spatial halo model (kernel + MTF
                                # pair), 0 = pedestal-only (default)
optics.stray.halo_sigma_um          # μm, Gaussian veiling-glare halo width on the
                                # focal plane

readout.electronics_sigma_um        # μm, equivalent Gaussian blur from amplifier
                                # bandwidth (x-axis only)

detector.pixel_pitch_x_um           # μm
detector.pixel_pitch_y_um           # μm
detector.fill_factor                 # dimensionless (0–1)
detector.qe_value                    # dimensionless (0–1)
detector.dark_rate_e_per_s           # e-/s/pixel (at reference temp)
detector.detector_temperature_K      # K
detector.ipc_coupling                # dimensionless (0–1)
detector.charge_diffusion_length_m   # m
detector.n_pixels_cross              # int (cross-track)

readout.read_noise_e_rms             # e- RMS
readout.gain_e_per_dn                # e-/DN
readout.adc_bits                     # int
readout.full_well_capacity_e          # e- (analog_well only – rejected if explicitly
                                # set under digital_counting)
readout.architecture                 # enum: "analog_well", "digital_counting" – DR0IC
                                # dispatch (Gap 117; digital_counting is
                                # schema-only until Digital_Pixel_Readout_Plan
                                # Phase 1 lands: ReadoutStage validates the

```

```

# parameters, then raises an actionable
# not-implemented error before any physics)
readout.counter_bits # int – in-pixel counter depth N; effective well =
#  $2^N \times \text{count\_packet\_e}$  (counting-only: rejected
# if explicitly set under analog_well)
readout.count_packet_e # e- per count – charge-subtraction quantum
# (counting-only; REQUIRED > 0 when architecture
# = digital_counting; schema default 0.0 = unset
# sentinel)
readout.residue_readout # bool – read the analog residue through the
# existing ADC model (counting-only)
readout.max_count_rate_hz # Hz – comparator dead-time flux ceiling
# (counting-only; 0.0 = unset  $\Rightarrow$  no ceiling,
# counter rollover governs)
readout.counting_mode # enum: "up", "up_down" – signed-differential
# background subtraction (plan Phase 4, D1/D6;
# counting-only; reference params rejected under
# "up")
readout.reference_source # enum: "background_term" (sub-pixel/point-source
# only, D6), "user_level" (extended-scene
# fallback); up_down-only
readout.reference_rate_e_per_s # e-/s – down-phase reference charge rate
# (REQUIRED > 0 under "user_level"; 0.0 = unset
# sentinel); up_down-only
readout.reference_integration_s # s – down-phase duration (0.0 = unset  $\Rightarrow$  equal to
# the scene integration time, D7); up_down-only
readout.cds_enabled # int (1 = yes, 0 = no; dtype=int, default 1)
readout.n_tdi # int
readout.n_coadds # int
readout.binning_x_onchip # int
readout.binning_y_onchip # int

calibration.scheme # enum: "none" (default – model off, today's
# PRNU/DSNU behavior), "one_point", "two_point",
# "three_point" (Gap 120, ADR-0012)
calibration.cal_temp_low_K # K – (lower) cal-point source temperature (0.0 =
# unset sentinel; required at evaluate time when
# a scheme is active)
calibration.cal_temp_high_K # K – upper cal point (two_point only; must exceed
# low; 0.0 = unset)
calibration.nonlinearity_pct # %  $\rightarrow$  fraction – per-pixel quadratic-nonlinearity
# dispersion (1 $\sigma$ ); sets the two_point post-NUC
# residual amplitude
calibration.time_since_cal_s # s (input: hour) – drift terms grow linearly with
# this (D4)
calibration.gain_drift_frac_per_s # 1/s (input: %/hour) – gain-drift rate  $\rightarrow$ 
# "gain_drift" term
calibration.offset_drift_e_per_s # e-/s (input: e-/hour) – offset-drift rate  $\rightarrow$ 
# "offset_drift" term
calibration.source_temp_uncertainty_K # K – cal-source  $\Delta T$  (1 $\sigma$ )  $\rightarrow$  BIAS term
# (accuracy budget only)
calibration.source_emissivity_uncertainty # fraction – cal-source  $\Delta \epsilon$  (1 $\sigma$ )  $\rightarrow$  BIAS term

```

```

calibration.source_emissivity      # fraction – nominal cal-source emissivity
                                   # (independent input: a source material
                                   # property, not a Rule-5 optical element)
calibration.gain_uncertainty_pct   # % → fraction – absolute gain uncertainty (1σ) →
                                   # BIAS term

spectral_integration.filter_min_um  # μm
spectral_integration.filter_max_um  # μm
spectral_integration.integration_time_s # s

geometry.sensor_altitude_m          # m
geometry.target_altitude_m          # m
geometry.site_elevation_m           # m – terrain elevation under the LOS (default 0 =
                                   # MSL); references the Hufnagel-Valley Cn2
                                   # SURFACE term only (CU-262,
                                   # RADIANT_Atmosphere.md §7.1). Not derived from
                                   # the LOS lower endpoint – see that section.
geometry.target.shape               # enum: none/sphere/cylinder/flat_plate/box/cone –
                                   # target spatial extent (ADR-0008; was
                                   # source.target.shape, now a deprecated alias)
geometry.target.shape_radius_m      # m (+
                                   # shape_length_m/width_m/height_m/base_radius_m)
geometry.target.shape_yaw_rad       # rad (+ shape_pitch_rad/shape_roll_rad) – body
                                   # ZYX Euler
geometry.target.projected_area_m2   # m2 – projected area facing observer (0.0 =
                                   # extended default)
geometry.path_zenith_rad             # rad (input: deg)
geometry.solar_zenith_rad           # rad (input: deg)
geometry.solar_azimuth_rad          # rad (input: deg)
geometry.ground_speed_m_s           # m/s
# Consumed by AtmosphereStage, PlatformStage, PerformanceStage,
# and (post-CU-009) by SourceStage's `_infer_los` for
# `LineOfSightGeometry` construction. See RADIANT_Atmosphere.md §6.5.

atmosphere.model                   # enum: "simple", "exo", "tabulated", "modtran",
                                   # "interpolated"
atmosphere.visibility_km            # km
atmosphere.precipitable_water_cm    # cm
atmosphere.standard_atmosphere      # enum: "tropical", "midlat_summer",
                                   # "midlat_winter", "subarctic_summer",
                                   # "subarctic_winter", "us_standard"
atmosphere.modtran.binary_path      # str (file path)
atmosphere.modtran.h2o_scale        # dimensionless
atmosphere.r0_m                    # m (Fried parameter)

source.target.temperature            # K
source.target.emissivity             # dimensionless (0–1)
source.target.emissivity_path        # str – 2-col CSV  $\epsilon(\lambda)$ ; spectral thermal target
                                   #  $\epsilon(\lambda) \cdot B(\lambda, T)$  (Gap 47); mutually exclusive with
                                   # scalar  $\epsilon$  / reflective / radiance /
                                   # brightness-temp surfaces
source.target.is_hot_target          # bool – MWIR routing opt-out (CU-007); see

```

```

# source._inferred matrix §3.2
source.target.reflectance      # dimensionless (0–1), Lambertian
# Target spatial extent (shape/dims/orientation/projected_area) moved to the
# geometry.target.* namespace (ADR-0008); source.target.shape*/projected_area_m2
# remain as deprecated aliases. Spectral/material params above stay in source.
source.target.range_m          # m – deprecated alias of geometry.target_range_m
                                # (ADR-0006)
source.target.fill_fraction    # dimensionless (0–1) – sub-pixel, stays in source
source.background.temperature   # K
source.background.emissivity    # dimensionless (0–1)

platform.jitter_rms_urad       # rad (input: μrad)
platform.ground_velocity_m_s    # m/s
platform.smear_length_um        # μm (image-plane smear)

performance.detection_snr_threshold # dimensionless; SNR at which a point
                                     # target counts as detected (Gap 77).
                                     # Default 5.0 (Rose criterion). The
                                     # in-chain detection-range solver bisects
                                     # to this threshold.

```

The ten parameter namespaces are geometry, source, atmosphere, optics, platform, spectral_integration, detector, readout, calibration, and performance. Every first segment is an owning stage — since ADR-0006 geometry.* is owned by GeometryStage (stage 0), which resolves the scene-geometry input modes and publishes the derived quantities; it is no longer a stage-less shared block. performance holds only analyst-tuned metric thresholds — most performance metrics are derived from upstream chain quantities and take no parameters.

Renames use ParameterDef.deprecated_aliases (warn-and-redirect at set()/get()): source.target.range_m → geometry.target_range_m and platform.h_sensor → geometry.sensor_altitude_m (CU-090 fold) — both ADR-0006, 2026-07-12.

12.3.2 Naming rules (per ADR-D, docs/adr/ADR-D-parameter-naming.md, 2026-07-06)

1. All lowercase, underscores separating words: aperture_diameter_m, not ApertureDiameter or aperture-diameter.
2. Every dimensioned parameter carries a unit suffix: _m, _um, _urad, _K, _rad, _s, _hz, _km, _cm, _e, _e_per_s, _e_rms, _m_s, _W_m2, _eV, _pct, _waves, ... (e.g. optics.aperture_diameter_m, detector.pixel_pitch_x_um). The suffix names the **input unit** — the unit a user supplies to params.set(), exactly as declared in the schema's input_unit. It usually coincides with the canonical unit; where they differ, the suffix follows the input unit (pixel_pitch_x_um accepts μm, stores meters canonically; jitter_rms_urad accepts μrad, stores rad). Dimensionless parameters (ratios, counts, flags, enums) have no suffix (optics.f_number, detector.fill_factor). The suffix is a human affordance; the ParameterDef remains the single source of truth for conversion.
3. Namespace depth is 2 or 3: stage.parameter_name or stage.group.parameter_name, where the group names a cohesive sub-model (source.target.*, source.background.*, optics.stray.*, atmosphere.modtran.*). The first segment is the owning stage; there is no sensor. super-prefix.
4. Boolean parameters are named as adjectives or states: cds_enabled, not use_cds or cds.
5. Count parameters are prefixed with n_: n_tdi, n_coadds, n_pixels_cross.

12.4 Defaults

Each parameter definition includes a default value or None (required).

12.4.1 Default categories

1. **Required (no default):** Parameters that fundamentally define the scenario. There is no sensible default for aperture diameter — the user must choose a sensor.
 - `optics.aperture_diameter_m`: None (required)
 - `optics.focal_length_m`: None (required, unless derived via the `fnumber` group)
 - `detector.pixel_pitch_x_um`: None (required)
 - `detector.pixel_pitch_y_um`: None (required)
2. **Defaulted to common value:** Parameters where a reasonable assumption covers 80% of use cases.
 - `readout.cds_enabled`: 1 (dtype=int, 1=yes/0=no; most modern ROICs use CDS)
 - `readout.n_coadds`: 1
 - `readout.n_tdi`: 1 (no TDI)
 - `readout.binning_x_onchip`: 1
 - `readout.binning_y_onchip`: 1
 - `optics.obscuration_ratio`: 0.0 (unobscured by default)
 - `detector.fill_factor`: 1.0
 - `source.target.temperature`: 300.0 K
 - `source.background.temperature`: 290.0 K
3. **Defaulted to “off” for optional effects:** Optional parameters default to the value that disables the effect.
 - `platform.jitter_rms_urad`: 0.0 (no jitter)
 - `detector.ipc_coupling`: 0.0 (no inter-pixel capacitance)
 - `detector.clutter_sigma`: 0.0 (no clutter)
 - `optics.defocus_um`: 0.0 (in focus)

12.4.2 Default documentation

Every default value includes a one-line justification in the schema:

```
ParameterDef(
    name="readout.cds_enabled",
    description="Correlated double sampling active",
    dtype=bool,
    canonical_unit="",
    input_unit="",
    default=True,
    bounds=None,
    default_justification="CDS is standard on modern HgCdTe and CMOS ROICs; "
                          "eliminates kTC noise"
```

)

12.5 Tolerances

12.5.1 Specification

Any scalar parameter can carry a tolerance specification for Monte Carlo / sensitivity analysis:

```
@dataclass
class Tolerance:
    distribution: str          # "gaussian", "uniform", "truncated_gaussian",
                             # "log_normal"
    params: dict              # distribution-specific parameters, in input_unit

# Examples (all in input units):
# ±5% QE:
Tolerance("gaussian", {"std_fraction": 0.05}) # std = 5% of nominal value

# Absolute jitter uncertainty:
Tolerance("gaussian", {"std": 0.5})           # ±0.5 μrad (in input_unit)

# Operating temperature range:
Tolerance("uniform", {"low": 75, "high": 85}) # 75–85 K

# WFE with hard limits:
Tolerance("truncated_gaussian", {"std": 0.02, "low": 0.0, "high": 0.2}) # waves
```

12.5.2 Monte Carlo execution

1. For each trial, the resolver samples each toleranced parameter from its distribution.
2. The full dependency graph is re-resolved with the sampled values.
3. Spectral data is regenerated from the sampled scalars (e.g., sampled cutoff_wavelength → new QE(λ)).
4. The signal chain runs with the sampled parameter set.
5. Results are collected across N trials for statistical analysis.

12.5.3 Sensitivity analysis

Single-parameter sensitivity: perturb one parameter by $\pm 1\sigma$, hold all others at nominal, record output change. This is a $2N+1$ evaluation (N parameters $\times$ 2 perturbations + 1 nominal). Cheaper than full Monte Carlo for identifying dominant sensitivities.

12.6 Consistency Groups

12.6.1 Problem

Some parameters are linked by physics constraints: $f/\# = f / D$. Given any 2 of {f, D, f/#}, the third is derived. The parameter system must handle this without requiring the user to know which one to leave unspecified.

12.6.2 Design

A consistency group defines:

- A set of N linked parameters
- N derivation rules (one for each parameter that could be the free variable)
- The constraint equation for validation

```
@dataclass(frozen=True)
class ConsistencyGroup:
    name: str
    parameters: tuple[str, ...]
    constraint: str
    derivations: dict[str, Callable]
    tolerance: float = 1e-9

    # human-readable: "f_number =
    #   focal_length_m / aperture_diameter_m"
    # {free_param: function(known_values) ->
    #   value}
    # relative tolerance for the
    # over-specification check
```

12.6.3 Resolution algorithm

1. Count how many parameters in the group are user-specified.
2. If exactly N-1 are specified: derive the Nth using the appropriate rule. Set provenance = DERIVED.
3. If all N are specified: validate consistency. If $|\text{computed} - \text{specified}| > \text{tolerance}$, raise ConsistencyGroupError (a CoreValidationError carrying group and parameters structurally, CU-373 F-09) with a diagnostic message showing the inconsistency.
4. If fewer than N-1 are specified: check if any have defaults. Apply defaults, then re-evaluate. If still underdetermined, raise an error listing what's missing.

12.6.4 v1 consistency groups

Groups are assembled in `radiant/api/_param_registry.py` (there is no module-level constant in `radiant.core.parameters`) and passed to the ParameterSet constructor: `ParameterSet(schema, groups)`. Two groups are defined:

```
# radiant/api/_param_registry.py
_FNUMBER_GROUP = ConsistencyGroup(
    name="fnumber",
    parameters=(
        "optics.aperture_diameter_m",
        "optics.focal_length_m",
        "optics.f_number",
    ),
    constraint="f_number = focal_length_m / aperture_diameter_m",
```

```

derivations={
    "optics.f_number": lambda kv: (
        kv["optics.focal_length_m"] / kv["optics.aperture_diameter_m"]
    ),
    "optics.focal_length_m": lambda kv: (
        kv["optics.aperture_diameter_m"] * kv["optics.f_number"]
    ),
    "optics.aperture_diameter_m": lambda kv: (
        kv["optics.focal_length_m"] / kv["optics.f_number"]
    ),
},
tolerance=1e-3,
)

def build_parameter_set() -> ParameterSet:
    schema = list(SRC_PARAMS + ATMO_PARAMS + OPT_PARAMS + ...)
    return ParameterSet(schema, [_FNUMBER_GROUP, _GROUND_SPEED_GROUP])

```

The second group **collapses a duplicate parameter** (Gap 75, 2026-07-11). `platform.ground_velocity_m_s` (consumed by `smear`) and `geometry.ground_speed_m_s` (consumed by the access-rate metric) are the same physical quantity — the along-track ground velocity. Linking them as an identity consistency group means setting either derives the other (one number feeds both consumers) and setting both to disagreeing values raises the over-specification error instead of silently using two different velocities:

```

_GROUND_SPEED_GROUP = ConsistencyGroup(
    name="ground_speed",
    parameters=("platform.ground_velocity_m_s", "geometry.ground_speed_m_s"),
    constraint="platform.ground_velocity_m_s == geometry.ground_speed_m_s",
    derivations={
        "platform.ground_velocity_m_s": lambda kv: kv["geometry.ground_speed_m_s"],
        "geometry.ground_speed_m_s": lambda kv: kv["platform.ground_velocity_m_s"],
    },
    tolerance=1e-6,
)

```

Both parameters default to 0.0, so an unset pair resolves to 0 (no motion). The `Sensor.set_ground_velocity_from_orbit()` helper (Gap 75) derives the value from `geometry.sensor_altitude_m` via `radiant.core.orbit.ground_track_speed_m_s` for orbital platforms. The analogous altitude duplicate has been collapsed: `platform.h_sensor` is now a **deprecated alias** of `geometry.sensor_altitude_m` (warn-and-redirect at `set()/get()`, CU-090 fold / ADR-0006, 2026-07-12) — see the `deprecated_aliases` note earlier in this section.

12.6.5 Target-spec exclusivity — the resolve-time seam (CU-244)

The `source.target.*` user-entry surfaces (Target Definition Matrix §1 spec forms) carry a family of **cross-parameter mutual-exclusivity rules** that are not consistency groups — there is no derivation between the members, only the constraint “set at most one door”: the reflectance/albedo aliases (scalar and `_path`), ρ vs the legacy (ϵ , T) thermal pair, ρ vs the S8/S10 absolute radiance/intensity paths, ρ vs the S11/S12 brightness/radiance-temperature forms, and the S11/S12 internal pairings.

These guards live in `radiant.source.target_spec` (one door-check function per spec door, run in the inferrer's dispatch order $S11 \rightarrow S12 \rightarrow S4/S5/S6 \rightarrow S8 \rightarrow S10b \rightarrow S10 \rightarrow S1\text{-with-}\epsilon(\lambda)$) and are called from **two** places:

1. **Evaluate time** — the source inferrer's door builders call the door checks at the same points the historical inline blocks occupied, so `evaluate()` behaviour is unchanged (defence in depth). The message text is unchanged too, with one deliberate exception: CU-295 retargeted the `what=` prefix from `"source._inferred: "` to `"source.target_spec: "`, because the named module no longer holds the check.
2. **Resolve time** — `Sensor.validate_target_spec()` runs the same functions with no physics, file I/O, or resolve required. Both views apply the same provenance rule — a resolved set reads `get_resolved(...).provenance`, an unresolved set reads `input_provenances()`, and `DEFAULT` and `DERIVED` both read as *not* user-set (CU-300: an over-specification is a statement about user input, so a value RADIANT computed must never trigger one; `SAMPLED` does count, a sweep axis being user intent). The GUI's clone-validate edit discipline calls this after each candidate set and rejects a conflict *the edit introduces* at the door — with the identical `what/why/action` `ParameterBoundsError` the evaluate-time path produces, by construction (same code).

Completeness checks ("S12 also needs its band edges") are deliberately **not** part of the seam: a half-entered spec is a legitimate intermediate state while a user types, and `evaluate()` remains the surface that reports what is missing.

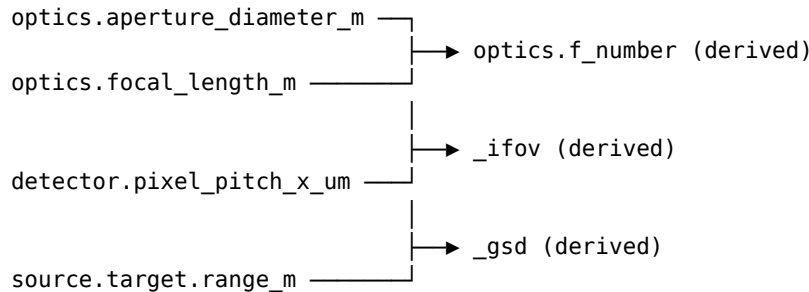
The two entry points are symmetric (CU-293, owner ruling 2026-08-01). They were not, on landing: CU-244 left the S8, S10 and S10b door guards inlined in the inferrer — so a conflict entered purely through those doors slipped past the GUI editor and failed only at `evaluate()` — and the S11 door had no guard against its S12 twin at all, so a `brightness_temperature_* + radiance_temperature_K` pair *raised at the seam but evaluated silently*, the S11 builder dispatching first and discarding the radiance-temperature surface (a Rule-17 violation). CU-293 moved the three remaining door blocks into `check_user_radiance_conflicts / check_point_intensity_conflicts / check_user_intensity_conflicts` and added the missing S11-vs-S12 guard to `check_brightness_temperature_conflicts`. CU-318 moved the last inlined door guard, the $\epsilon(\lambda)$ door's `check_emissivity_path_conflicts`. Every door guard is now registered at both entry points, so a pair either door knows about is refused with the same message at both.

The last asymmetry is closed (CU-323, owner ruling 2026-08-02). CU-318 measured one residual case: the $\epsilon(\lambda)$ door dispatches *after* every other door, and nine of the ten surfaces its guard lists open a door of their own, so at `evaluate()` those doors returned first, the $\epsilon(\lambda)$ guard was never reached, and `source.target.emissivity_path` was discarded in silence (a Rule-17 defect of the same class CU-293 closed for S11/S12) while the seam refused the same pairs. The owner extended the CU-293 ruling class: refusal is the correct behaviour, so `check_emissivity_path_conflicts` is no longer a per-door guard sitting last in dispatch order — it is a **pre-dispatch** check, run first in `validate_target_spec` and first in `_inferred._build_target_descriptor`. It is a no-op unless `emissivity_path` is user-set and non-empty, so no single-door spec can newly raise (measured at the ruling: none of the 67 shipped YAML configs sets the surface). Running it in the same position at both entry points also makes them report the same *first* error for a config that over-specifies in more than one way, not just for a two-surface pair.

12.7 Dependency Tracking

12.7.1 DAG structure

Parameters can depend on other parameters via derivation rules. The resolver maintains a directed acyclic graph (DAG):



12.7.2 Resolution algorithm

1. **Collect inputs:** User-set values + config file values + defaults.
2. **Validate inputs:** Type check, bounds check, enum check on all provided values.
3. **Convert units:** All input-unit values → canonical-unit values.
4. **Resolve consistency groups:** Process each group as described above.
5. **Topological sort:** Order the derivation rules so that dependencies are computed before dependents.
6. **Execute derivations:** Compute each derived parameter in topological order.
7. **Post-validation:** Check all derived values against bounds.
8. **Freeze:** The resolved parameter set is immutable. Changing a parameter requires creating a new resolution pass.

12.7.3 Cycle detection

The DAG is validated at schema registration time. If a cycle is detected (A depends on B depends on A), that's a schema bug, not a runtime error. Raise immediately with the cycle path.

12.7.4 Invalidation

When a parameter changes (e.g., in a parametric sweep):

1. Find all downstream dependents via the DAG.
2. Mark them as stale.
3. Re-resolve only the stale subgraph, not the entire parameter set.
4. Regenerate any spectral data that depends on changed parameters.

12.8 Provenance

12.8.1 Provenance tags

```
class Provenance(Enum):
```

```

USER_SET = "user_set"           # Explicitly provided by user
CONFIG_FILE = "config_file"     # Loaded from a sensor/scenario config file
DEFAULT = "default"             # From schema default
DERIVED = "derived"             # Computed from other parameters
SAMPLED = "sampled"             # Generated by Monte Carlo sampling
PRESET = "preset"               # Applied from a named FPA preset (Gap 119)

```

PRESET (added at Gap 119 Phase 0) labels values applied from a named FPA preset (`radiant.data.FPALibrary`); its source string is `fpa:<part-name>/<source-key>` so every preset value traces to the citation in the preset document. The apply path itself lands in the plan's Phase 2; user and config values set later override preset values through the ordinary last-write path, and the override is reported.

12.8.2 Tracking

Every resolved parameter carries:

```

@dataclass
class ResolvedValue:
    value: Any                # In canonical units
    input_value: Any          # In input units (as the user provided it)
    provenance: Provenance
    source: str               # "user", "/path/to/sensor_abc.yaml",
                             # "default", "derived:f/D"
    derived_from: dict[str, Any] | None # For DERIVED: {param_name: value_used}
    timestamp: str            # ISO 8601 when this value was set/computed

```

Source strings in use. `source` is a free-form human-readable origin, not an enum; the writers that populate it are:

source	Written by
"user", "Sensor.set", "Sensor.set_many"	interactive / scripting edits
fpa:<part>/<source-key>	<code>radiant.api.fpa_preset.apply_fpa_preset</code> (Gap 119 — traces to the citation in the preset document)
the YAML file path (or "dict")	<code>radiant.io.config.load_config</code>
"default: <justification>"	schema defaults applied during resolution
"derived: <constraint>"	consistency-group derivation
"config:<name>"	a configured parameter's value, materialized for the configuration named <name> by <code>ConfigurationSet.sensor_for</code> (ADR-0010 D-C)

The `config:<name>` form is what lets `Sensor.resolved()` / `Sensor.explain()` name the owning configuration of a multi-configuration study with **zero new provenance machinery**: a materialized configuration is an ordinary `Sensor` whose configured values were set with `Provenance.USER_SET` and that source string. Note the ADR-0010 D-10 disambiguation convention: <name> here is a **configuration** (a member of a configuration set), never a config *file* — file-sourced values carry the file path.

12.8.3 Reading inputs without resolving

`ParameterSet.peek_input(name)` (and `Sensor.peek_input`) returns the explicitly-set input value in input units, or `None`, **without** requiring or triggering resolution — unlike `get_input`, which raises while

any required parameter is unset. Added for incomplete-configuration display (Gap 119): a committed edit must be readable before the last required parameter lands. Bounds/enum validation still happens at resolve time; `peek_input` is a raw read of the input store (values already unit-converted at `set()`, Rule 2). Resolve-time required-parameter failures raise `RequiredParameterError` (carrying the dot-path as `.param`), a `CoreValidationError` subclass.

12.8.4 Provenance audit

The entire resolved parameter set can be serialized to a provenance record:

```
{
  "radiant_version": "0.1.0",
  "resolved_at": "2026-04-06T14:30:00Z",
  "parameters": {
    "optics.aperture_diameter_m": {
      "value": 0.3,
      "canonical_unit": "m",
      "input_value": 0.3,
      "input_unit": "m",
      "provenance": "user_set",
      "source": "user"
    },
    "optics.f_number": {
      "value": 4.0,
      "canonical_unit": "",
      "input_value": 4.0,
      "input_unit": "",
      "provenance": "derived",
      "source": "derived:focal_length_m/aperture_diameter_m",
      "derived_from": {
        "optics.focal_length_m": 1.2,
        "optics.aperture_diameter_m": 0.3
      }
    }
  }
}
```

This per-parameter dict is the `parameter_set` block of the §C13 provenance record returned by `ChainResult.to_provenance_record()`. The record also carries `input_file_hashes` — an ordered list of (path, sha256) pairs for every YAML consumed during the run. `radiant.io.config.load_config` populates this list by calling `ParameterSet.record_loaded_file(path, sha256)` after a successful YAML parse; in-memory dict sources record nothing (no path to hash).

Given the provenance record, any result is exactly reproducible.

12.9 Explainability

Every parameter can answer: “why does this have this value?”

```
params.explain("optics.f_number")
```

```
# Returns:
# "optics.f_number = 4.0 (dimensionless)
# Provenance: DERIVED
# Rule: f_number = focal_length_m / aperture_diameter_m
# From: optics.focal_length_m = 1.2 m (USER_SET)
#       optics.aperture_diameter_m = 0.3 m (USER_SET)
# Consistency group: fnumber"

params.explain("readout.cds_enabled")
# Returns:
# "readout.cds_enabled = 1
# Provenance: DEFAULT
# Justification: CDS is standard on modern HgCdTe and CMOS ROICs; eliminates kTC noise"
```

For downstream outputs, explainability is exposed by these **shipped** surfaces (there is no single `result.explain("snr")` method — it is a design target):

```
# Noise breakdown for a computed metric (shipped):
result.explain_noise("dark")      # ChainResult.explain_noise(term) – io/results.py
result.noise_terms                 # tuple[NoiseTerm] – the full per-term budget

# Parameter-level explanation (shipped):
sensor.explain("readout.cds_enabled") # Sensor.explain(dotpath) – api/sensor.py

# Sweeps and sensitivities are Sensor / radiant.api surfaces (NOT ParameterSet):
sensor.sweep("optics.aperture_diameter_m", [0.15, 0.30, 0.45, 0.60])
```

[DESIGN-TARGET] A single unified `result.explain("snr")` that folds the signal chain, the full noise budget, and the top-3 parameter sensitivities into one narrative does not exist yet; assemble it today from `explain_noise` + the sensitivity API.

12.10 Schema Introspection (Gap 70, 2026-07-11)

`ParameterSet` exposes a public introspection surface so GUIs, CLIs, and sweep tooling never touch private state. This is the enumeration contract the GUI parameter panel generates from.

```
ps.parameter_defs()      # Mapping[str, ParameterDef] – read-only live view of the
                        # full schema keyed by dot-path; each ParameterDef carries
                        # dtype, canonical/input units, bounds, enum_values,
                        # default, description, tags, group, deprecated_aliases.
ps.parameter_def(name)   # Single ParameterDef; alias-aware (DeprecationWarning);
                        # unknown names raise UnknownParameterError (a
                        # RadiantError co-inheriting KeyError) with a
                        # did-you-mean hint (CU-073).
ps.consistency_groups()  # tuple[ConsistencyGroup, ...] in registration order.
ps.tolerances()          # Mapping[str, Tolerance] – read-only view.
ps.inputs()              # Mapping[str, Any] – explicit inputs only (name → raw
                        # input-unit value); defaults/derived excluded. The
                        # persistence surface: re-setting exactly these on a
                        # fresh set reproduces this resolution (Gap 67).
```

```

ps.input_provenances()  # Mapping[str, Provenance] – the provenance companion of
                        # inputs() (Gap 93); explicit inputs only, no resolve.
ps.is_resolved          # bool property: resolve() has run and no input changed.
ps.copy()               # Unresolved deep-enough copy: schema, groups, inputs
                        # (with provenance), tolerances, loaded-file records.
                        # The supported way to build sweep/clone variants.

```

The read-only views are MappingProxyType wrappers: mutation raises TypeError, and the schema itself is fixed at ParameterSet construction. Framework consumers (cli/schema_cmd.py, api/sensitivity.py, api/sweep.py, api/tolerance.py, api/sensor.py) use only this surface — `_defs`, `_groups`, `_inputs`, `_tolerances`, `_resolved_flag` are private and carry no compatibility guarantee.

12.11 Spectral Data Store

12.11.1 Common wavelength grid

All spectral data is interpolated onto a single common wavelength grid before any physics computation. There is **no spectral.* parameter namespace**; the grid is built from the filter bandpass plus a point count:

```

grid = numpy.linspace(
    spectral_integration.filter_min_um,  # μm – lower filter edge (schema parameter)
    spectral_integration.filter_max_um,  # μm – upper filter edge (schema parameter)
    wavelength_points,                  # Sensor(...) constructor argument, NOT a
                                        # schema param
)

```

The grid is uniform in wavelength (`Sensor._wavelength_grid`, `api/sensor.py`). MODTRAN and every other spectral table are interpolated **onto** this grid; there is no `grid_type` enum and no “use the MODTRAN grid directly” mode at this layer.

12.11.2 SpectralData class

```

@dataclass
class SpectralData:
    name: str  # "qe", "tau_atm", "filter_transmission"
    wavelength_um: np.ndarray  # ascending, in μm
    values: np.ndarray  # same length, in canonical units
    unit: str  # "dimensionless", "W/m2/sr/um", etc.
    source: str  # "file:data/qe_hgcdte.csv", "computed:from
                # filter_center_wavelength, filter_bandwidth",
                # "modtran:tape7.out"
    source_parameters: dict[str, Any]  # scalar params used to generate this data

```

12.11.3 SpectralDataStore

```

class SpectralDataStore:
    def __init__(self, grid: np.ndarray):
        self.grid = grid  # common wavelength grid (μm, ascending)
        self._data: dict[str, np.ndarray] = {}  # name -> values on common grid

```

```

self._metadata: dict[str, SpectralData] = {} # name -> full SpectralData

def add(self, data: SpectralData) -> None:
    """Interpolate data onto common grid and store."""

def get(self, name: str) -> np.ndarray:
    """Return values on common grid."""

def get_metadata(self, name: str) -> SpectralData:
    """Return full metadata including source and provenance."""

```

12.11.4 Spectral generators

Each spectral quantity has a registered generator that either computes from scalar parameters or loads from a file:

Spectral Data	Generator Input	Source
qe	detector.qe_value (flat), detector.qe_table_path (file), or detector.qe_material (bundled library curve — Gap 69; path > material > scalar)	Constant array, loaded file, or library curve (0 past data span)
filter_transmission	spectral_integration.filter_min_um, spectral_integration.filter_max_um	Computed top-hat bandpass
tau_atm	atmosphere.model + model params (e.g. atmosphere.tabulated_transmittance_file)	Loaded from file or computed from simple model
path_radiance	atmosphere.model + model params	Loaded from file or computed
atm_emission	atmosphere.model + model params	Loaded from file or computed
solar_irradiance	None (reference spectrum)	Loaded from built-in data file
optical_transmission	optics.transmission_scalar or per-element data (optics.transmission_input_mode)	Computed or loaded
target_emissivity	source.target.emissivity (scalar) or source.target.reflectance_path	Scalar → constant array, or loaded from spectral library

12.12 Configuration File Format

Parameters are specified in YAML. The dot-path namespace maps to YAML nesting:

```

# scenario_example.yaml – top-level keys are the stage namespaces
source:
  target:
    temperature: 300.0          # K
    emissivity: 0.95
  background:
    temperature: 290.0         # K
    emissivity: 0.96

atmosphere:
  standard_atmosphere: midlat_summer

```

```

geometry:
  sensor_altitude_m: 8000.0      # m

optics:
  aperture_diameter_m: 0.30      # m
  focal_length_m: 1.20           # m (f/4.0)
  obscuration_ratio: 0.33
  wfe_rms_waves: 0.07           # waves
  transmission_scalar: 0.70

detector:
  pixel_pitch_x_um: 18.0         # μm
  pixel_pitch_y_um: 18.0         # μm
  qe_value: 0.70
  dark_rate_e_per_s: 100.0      # e-/s

spectral_integration:
  filter_min_um: 3.5             # μm
  filter_max_um: 5.0             # μm
  integration_time_s: 0.005      # s (5 ms)

readout:
  read_noise_e_rms: 5.0          # e- RMS
  gain_e_per_dn: 32.0            # e-/DN
  adc_bits: 16
  cds_enabled: 1
  n_tdi: 1

platform:
  jitter_rms_urad: 3.0           # μrad

```

12.12.1 Loading precedence

Five levels, lowest to highest, each with the provenance tag it records:

1. Schema defaults — DEFAULT
2. An applied FPA preset — PRESET
3. The config-file body — CONFIG_FILE
4. `Sensor.set()` / `set_many()` — USER_SET
5. CLI `--set` — USER_SET

An FPA preset sits below the config file deliberately: presets seed, explicit values win, in any key order.

Each layer records its provenance. If the same parameter appears in multiple layers, the highest-priority layer wins, and the provenance shows the winning source. Derived parameters carry `DERIVED` and a `derived_` from record naming the inputs and their values.

12.13 Design Implications from Personas

Persona	Parameter system requirement	How it's met
Sarah (P1)	Sweep one parameter, hold others constant	<code>Sensor.sweep("optics.aperture_diameter_m", [0.15, 0.20, ..., 0.60])</code> (api/sensor.py; sweeps live on <code>Sensor/radiant.api</code> , not <code>ParameterSet</code>)
Mike (P2)	Inspect every noise term independently	Provenance + explainability on all derived noise values
Raj (P3)	Load a sensor config and specify only the scenario	Config file layering: sensor config + scenario overrides
Lisa (P4)	Batch run across target × atmosphere × sensor	Cross-product of config files → list of <code>ParameterSets</code>
Tom (P5)	Override WFE, see MTF effect	Direct <code>params.set()</code> override with instant re-resolution
Dr. Chen (P6)	Full provenance for reproducibility	Provenance audit record attached to every output
Karen (P7)	Compare predicted vs. measured; adjust one param to close gap	Sensitivity analysis via <code>Sensor.sensitivity("snr", "detector.dark_rate_e_per_s")</code> (api/sensor.py; on <code>Sensor/radiant.api</code> , not <code>ParameterSet</code>)

RADIANT Testing and Validation Framework

13.1 Test Hierarchy

Four levels of tests. Every level must pass before the next level is meaningful. Level 0 failures are bugs in physics fundamentals — they invalidate every result the tool has ever produced.

Level	Name	pytest marker	Scope	Speed
0	Physics correctness	level0	Single equations and constants against known-good analytic results	< 1 s per test
1	Module-level	level1	Individual stage or module with controlled inputs	< 10 s per test
2	End-to-end chain	level2	Full signal chain against multi-stage reference scenarios	< 60 s per scenario
—	Regression (golden)	golden	Full chain against frozen golden output files	< 120 s per golden

The registered markers in `pyproject.toml` are exactly `level0`, `level1`, `level2`, and `golden` — golden regression is its own marker, **not** a “level3” (no `level3` marker exists; `--strict-markers` would reject it). All levels run in CI on every PR. Level 0–2 run in < 5 minutes total. Golden tests run in a separate CI job on main branch only (not on every PR, to avoid golden file drift blocking development).

13.2 Level 0: Physics Correctness Tests

These tests are **the ground truth for the entire tool**. They use no RADIANT infrastructure beyond `constants.py` and `units.py`. They test equations against literature values.

13.2.1 Planck and Stefan-Boltzmann

Test: Numerical integral of $\text{Planck}(T=300\text{K}, \lambda)$ over $0.1\text{--}200\ \mu\text{m}$ equals $\sigma \times T^4$.

```
def test_planck_stefan_boltzmann():
    """Integral of spectral radiance over all wavelengths =  $\sigma T^4 / \pi$ ."""
    from radiant.core.constants import h, c, k_B, sigma_sb
    import numpy as np
    from scipy.integrate import quad

    T = 300.0 # K
    def planck_w_m2_sr_um(lam_um):
        lam_m = lam_um * 1e-6
        return (2 * h * c**2 / lam_m**5) / (np.exp(h * c / (lam_m * k_B * T)) - 1) * 1e-6
```

```

integral, _ = quad(planck_w_m2_sr_um, 0.1, 200.0, limit=500)
expected = sigma_sb * T**4 / np.pi # W/m²/sr (Lambertian hemisphere)
assert abs(integral - expected) / expected < 0.001 # < 0.1%

```

Test: Wien's displacement law: $\lambda_{\text{max}} \times T = 2897.77 \text{ } \mu\text{m} \cdot \text{K}$.

```

@pytest.mark.parametrize("T,expected_lam_peak_um", [
    (300, 9.659), # LWIR
    (1000, 2.898), # MWIR
    (6000, 0.483), # solar
])
def test_wiens_displacement(T, expected_lam_peak_um):
    """ $\lambda_{\text{max}} \cdot T = 2897.77 \text{ } \mu\text{m} \cdot \text{K}$  (CODATA 2018 Wien constant)."""
    from radiant.core.blackbody import planck_spectral_radiance
    import numpy as np
    wl = np.linspace(0.2, 30.0, 5000)
    L = planck_spectral_radiance(wl, T)
    lam_peak = wl[np.argmax(L)]
    assert abs(lam_peak - expected_lam_peak_um) / expected_lam_peak_um < 0.005 # < 0.5%

```

13.2.2 Shot Noise

Test: Shot noise variance equals signal (Poisson statistics).

```

@pytest.mark.parametrize("n_signal", [100, 1000, 10000, 100000, 1_000_000])
def test_shot_noise_is_sqrt_signal(n_signal):
    """ $\sigma_{\text{shot}} = \sqrt{\text{signal}}$ . This must be exact – no approximations."""
    from radiant.detector.shot_noise import shot_noise_e
    sigma = shot_noise_e(n_signal)
    assert sigma == pytest.approx(np.sqrt(n_signal), rel=1e-10)

```

13.2.3 Quantization Noise

Test: ADC quantization noise = $\text{LSB} / \sqrt{12}$.

```

@pytest.mark.parametrize("lsb_e", [50, 100, 200, 500])
def test_quantization_noise(lsb_e):
    """ $\sigma_q = \text{LSB} / \sqrt{12}$ . Exact formula – no approximation."""
    from radiant.readout.adc import AnalogToDigital
    adc = AnalogToDigital(gain_e_per_dn=lsb_e, n_bits=16)
    assert adc.quantization_noise_e() == pytest.approx(lsb_e / np.sqrt(12), rel=1e-10)

```

13.2.4 Diffraction Limit

Test: Airy disk first dark ring at $r = 1.22 \lambda f/D$ on the focal plane. There is no closed-form `airy_first_dark_ring` helper — the PSF is computed via FFT of the complex pupil (`radiant.optics.psf_mono.compute_psf`), and the first minimum of its radial profile is located numerically (this is exactly how `optics/tests/test_diffraction.py::TestAiryFirstZero` does it):

```

def test_airy_first_zero_location(config, psf_unaberrated):
    """First zero of the Airy pattern at  $r = 1.22 \lambda f/D$ .
    Truth anchor: Born & Wolf, Principles of Optics, §8.5.2."""

```

```

c = config.padded_npix // 2
profile = psf_unaberrated[c, c:] # radial profile from center
r_zero_expected = 1.21966989 * lam_m * f_m / D_m
for i in range(1, len(profile) - 1): # first local minimum
    if profile[i] < profile[i - 1] and profile[i] <= profile[i + 1]:
        r_min = i * config.focal_spacing_m
        break
assert r_min == pytest.approx(r_zero_expected, rel=0.01)

```

Test: Optical MTF from the pupil autocorrelation (radiant.optics.pupil_mtf, Rule 4 — no separate circular_aperture_mtf exists) is 1.0 at zero frequency and 0.0 at and beyond cutoff $f_c = D / (\lambda f)$:

```

def test_pupil_mtf_normalization(amplitude, phase, config):
    from radiant.optics.pupil_mtf import (
        pupil_autocorrelation_mtf_1d,
        pupil_autocorrelation_mtf_2d,
    )
    mtf_2d = pupil_autocorrelation_mtf_2d(amplitude, phase, config.padded_npix)
    freq, mtf = pupil_autocorrelation_mtf_1d(mtf_2d, config.focal_spacing_m, axis="x")
    f_cutoff = D_m / (lam_m * f_m) # cycles/m (focal plane)
    assert mtf[0] == pytest.approx(1.0, abs=1e-8)
    assert np.all(mtf[freq >= f_cutoff] < 1e-6)

```

13.2.5 Pixel Aperture MTF

Test: $MTF(f) = |\text{sinc}(f \times p \times \sqrt{FF})|$ where p is pixel pitch and FF is the areal fill factor.

The pixel aperture MTF has no standalone helper — DetectorStage computes it inline as `np.abs(np.sinc(freq_m * pixel_pitch * sqrt_ff))`, where `sqrt_ff = math.sqrt(detector.fill_factor)` (the photosite linear width is `pitch * sqrt_ff` for areal fill factor FF , CU-074), and publishes it as `mtf_pixel_aperture_x / mtf_pixel_aperture_y` (`detector/stage.py`, ~lines 153–162). This width matches the PSF-path pixel-aperture kernel exactly, so the two Rule-4 paths agree at `fill_factor < 1`. At $FF = 1$ the $\sqrt{FF}$ factor is unity and the term reduces to the plain $|\text{sinc}(f \cdot p)|$. The Level 0 check reads the stage's published MTF term:

```

def test_pixel_aperture_mtf(state_after_detector, freq_m):
    """FF = 1: MTF at Nyquist = sinc(0.5) ≈ 0.6366. At Nyquist/2 = sinc(0.25) ≈ 0.9003."""
    import numpy as np
    p = 18e-6 # m
    f_ny = 1 / (2 * p) # Nyquist frequency (cycles/m); fill_factor = 1.0
    mtf = state_after_detector.mtf_terms["mtf_pixel_aperture_x"]
    assert np.interp(f_ny, freq_m, mtf) == pytest.approx(np.sinc(0.5), rel=1e-3)
    assert np.interp(f_ny / 2, freq_m, mtf) == pytest.approx(np.sinc(0.25), rel=1e-3)

```

13.2.6 kTC Noise and CDS Cancellation

Test: kTC noise magnitude and exact cancellation by CDS.

```

def test_ktc_cds_cancellation():
    """CDS completely cancels kTC noise. Without CDS, kTC = sqrt(k_B T C) / q."""
    from radiant.detector.noise.roic import ktc_reset_noise
    from radiant.core.constants import k_B, q
    T_roic = 290 # K

```

```

C_node = 50e-15 # F (50 fF)
expected = np.sqrt(k_B * T_roic * C_node) / q

# With CDS enabled: kTC contribution = 0
assert ktc_reset_noise(C_node, T_roic, cds_enabled=True) == 0.0
assert ktc_reset_noise(C_node, T_roic, cds_enabled=False) == pytest.approx(
    expected, rel=1e-6
)

```

13.2.7 TDI Signal and Noise Scaling

Test: TDI improves SNR by exactly $\sqrt{N_{\text{TDI}}}$ (in shot-noise-limited regime).

```

@pytest.mark.parametrize("n_tdi", [1, 2, 4, 8, 16])
def test_tdi_snr_improvement(n_tdi):
    """N-stage TDI: signal scales as N, shot noise as  $\sqrt{N}$ , SNR as  $\sqrt{N}$ ."""
    S_single = 10000.0 # e- per stage
    S_total = S_single * n_tdi
    sigma_shot = np.sqrt(S_total) # shot-noise dominated
    snr = S_total / sigma_shot
    snr_single = S_single / np.sqrt(S_single)
    expected_improvement = np.sqrt(n_tdi)
    assert snr / snr_single == pytest.approx(expected_improvement, rel=1e-8)

```

13.2.8 Beer-Lambert Transmittance

Test: Simple atmosphere transmittance follows Beer-Lambert law.

There is no standalone `beer_lambert_transmittance` helper — `SimpleAtmosphere` (`radiant/atmosphere/simple.py`) computes the column-integrated optical depth per species and applies $\tau = \exp(-OD)$ internally. The check verifies the model's transmittance against a hand-computed optical depth (this is `atmosphere/tests/test_simple.py::test_truth_anchor_1_rayleigh_only_matches_hand_calc`):

```

def test_beer_lambert_matches_hand_calc():
    """ $\tau(\lambda) = \exp(-OD)$  with OD hand-computed from the column integral."""
    state = simple_atmosphere.build_state(...) # Rayleigh-only configuration
    od = sigma_0 * col_mol # hand-computed column OD
    expected_tau = math.exp(-od)
    assert state.transmittance.values[i] == pytest.approx(
        expected_tau, rel=1e-12, abs=1e-12
    )

```

13.2.9 Encircled Energy — Airy Disk

Test: Airy disk EE at first dark ring = 83.8%.

```

def test_airy_encircled_energy_at_first_dark(psf_data):
    """~84% of Airy energy within the first zero (Born & Wolf: 83.8%
    within the first dark ring). PSFData.encyrcled_energy uses a square
    box, so the tolerance is loose (this is
    optics/tests/test_psf.py::test_airy_84_percent)."""
    D, f, lam_m = 0.30, 1.20, 4.2e-6

```

```

r_first_dark = 1.22 * lam_m * f / D  # radius of first dark ring (m, focal plane)
ee = psf_data.encircled_energy(r_first_dark)
assert ee == pytest.approx(0.84, abs=0.05)

```

(The pixel-box coupling quantity EE_box itself is computed by `radiant.optics.ee_box.compute_ee_box(psf, n_pixels)` from the EffectivePSF.)

13.2.10 Unit Conversion Roundtrip

Test: All defined unit conversions are invertible and consistent.

```

def test_unit_conversion_roundtrip():
    from radiant.core.units import convert
    pairs = [
        (1.0, "deg", "rad"),
        (1.0, "urad", "rad"),
        (10.0, "ms", "s"),
        (5.0, "um", "m"),
        (600.0, "km", "m"),
    ]
    for value, from_u, to_u in pairs:
        converted = convert(value, from_u, to_u)
        roundtripped = convert(converted, to_u, from_u)
        assert roundtripped == pytest.approx(value, rel=1e-12)

```

13.3 Level 1: Module-Level Tests

Each physics subpackage tests its own stage in isolation, with controlled inputs (no file I/O, no other stages).

13.3.1 Module Test Requirements

Every module-level test must:

1. Construct the minimum `ParameterSet` needed for the stage under test.
2. Construct a `ChainState` that represents the stage's expected input.
3. Call the stage's `.run()` method.
4. Assert on specific fields in the returned `ChainState`.
5. Not read any file from disk (use in-memory spectral arrays or fixtures loaded in `conftest.py`).

13.3.2 Coverage

Coverage is **not gated in CI today**. `pytest-cov` is available as a dev dependency, so a contributor can run `pytest --cov=radiant --cov-report=html` locally (see CLAUDE.md “Running Tests Locally”), but no CI job passes `--cov` or `--cov-fail-under`, and there is no per-subpackage coverage threshold enforced anywhere. The `ci.yml` test jobs run `pytest -m level0/level1/level2` with `--strict-markers` only (§9.3). Do not read a coverage floor into this doc that the pipeline does not enforce.

[DESIGN-TARGET] A future per-subpackage coverage gate (e.g. higher bars on `core/` and `api/`, lower on `cli/`) is a reasonable target, but until a CI job actually asserts it, treat coverage as an informational local metric, not a merge gate.

13.3.3 Key Module Tests (Non-Exhaustive)

SourceStage — thermal emission:

- $T=300K$, $\epsilon=0.95$: verify at_target frame spectral radiance matches $0.95 \times \text{Planck}(300K, \lambda)$
- $T=1000K$, $\epsilon=1.0$: verify spectral peak near $2.9 \mu m$ (Wien's law)
- Regime classification: $100 m^2$ target at $10 km$ range $\rightarrow$ extended; $0.1 m^2$ target at $10 km \rightarrow$ point

AtmosphereStage — MODTRAN tape7 reader:

- Load fixture tape7; verify $\tau_{atm}(4.2 \mu m) \in [0.6, 0.9]$ (plausible range)
- Verify wavelength array is ascending (MODTRAN is descending wavenumber $\rightarrow$ must flip)
- Verify Jacobian conversion: $W/cm^2/sr/\mu m \rightarrow W/m^2/sr/\mu m$ (factor of $1e4$)
- Simple model: Beer-Lambert at $10 km$, $23 km$ visibility $\rightarrow \tau \approx 0.65$

OpticsStage:

- $f/\#$ derivation: $D=0.30m$, $f=1.20m \rightarrow f/\#=4.0$
- `EE_box` for diffraction-limited unobscured aperture: verify > 0.8 for 1-pixel box
- MTF_diffraction at $f=0$: 1.0 ; at $f=f_{cutoff}$: 0.0

DetectorStage — noise terms:

- Signal = $10,000 e^-$, `dark_current` = $500 e^-/s$, `t_int` = $5 ms \rightarrow dark = 2.5 e^-$; $\sigma_{dark} = \sqrt{2.5}$
- CDS enabled $\rightarrow kTC = 0$; CDS disabled $\rightarrow kTC \neq 0$
- IPC $\alpha=0.0 \rightarrow MTF_{IPC} = 1.0$ everywhere

ReadoutStage:

- `gain` = $100 e^-/DN$, 14-bit ADC: FWC in DN = `full_well` / `gain`; saturation flag if signal $>$ FWC
- Fowler-N read noise reduction: $N=2$ samples $\rightarrow \sigma_{read_effective} = \sigma_{read} / \sqrt{2}$

13.4 Level 2: End-to-End Reference Scenarios

Ten reference scenarios with known-good results. These test the full chain but not frozen bit-for-bit outputs.

13.4.1 Scenario Table

#	Name	Configuration	Key check	Tolerance
E01	MWIR blackbody, no atmosphere	$T=500K$, $\epsilon=1.0$, no atmosphere, $0.3m$ aperture, $80mm$ focal length, $15\mu m$ pixel, $QE=0.70$, <code>t_int</code> = $1ms$	$SNR = \sqrt{S}$, where $S =$ computed photoelectrons (shot-noise limited)	SNR within 1% of $\sqrt{S}$

#	Name	Configuration	Key check	Tolerance
E02	Shot-noise limited SNR	Extended scene, increase aperture by $2\times \rightarrow$ SNR should increase by $2\times$ (signal $\propto A^2$, shot noise $\propto A$)	$SNR(2D) / SNR(D) = 2.0$	within 1%
E03	NEDT vs. analytic	$T_{target}=300K$, $T_{bg}=290K$; compute NEDT numerically; compare to $(\sigma_{total} / signal) \times (1 / (dL/dT \times A \times \Omega \times \tau \times QE \times t_{int} / L))$	NEDT within 2% of analytic	2% relative
E04	MODTRAN round-trip	Load midlat_summer_mwir. tape7; apply to 300K blackbody; verify at-aperture radiance is $\tau \times L_{source} + L_{path}$ (at $4.2 \mu m$)	$\Delta L < 0.5\%$ relative at $\lambda=4.2 \mu m$	0.5%
E05	MTF product	Compute system MTF as product of 5 terms; verify $MTF_{system} \leq \min(\text{individual terms})$ at all frequencies	Monotonic, bounded by minimum component	exact inequality
E06	Point source EE_box	Sub-resolution target ($A_{target} \rightarrow 0$); verify signal scales as EE_box; EE_box $\rightarrow 1$ as PSF $\rightarrow 0$	Signal = $S_{extended} \times EE_{box}$ within 1%	1%
E07	TDI gain end-to-end	$N_{TDI}=4$; verify total signal = $4 \times$ single-line signal; total noise = $2 \times$ single-line noise; $SNR = 2 \times$ single-line SNR	Exact TDI scaling	0.1%
E08	GIQE5 reference case	Published NGA GIQE5 test case: GSD=1.0m, SNR=30, RER=0.35 $\rightarrow$ NIIRS=6.0 (approx.)	NIIRS within ± 0.1 of reference	± 0.1
E09	NEDT vs. temperature sweep	NEDT as a function of T_{target} from 200–350K in LWIR; should have minimum near scene temperature for MWIR crossover	NEDT monotonically well-behaved; minimum at $\approx$ crossover temperature	qualitative check + < 5% vs. manual calculation at 300K
E10	Dark-current dominated regime	Very long integration time ($t_{int}=1.0$ s, dark= $1e6$ e-/s); verify σ_{total} dominated by $\sqrt{(\text{dark} \times t_{int})} = 1000$ e- (> read noise = 25 e-)	Dominant noise term = dark_current_shot; $\sigma_{dark_shot} / \sigma_{total} > 0.90$	exact fraction check

13.4.2 Additional Correctness Checks

Wavelength grid self-consistency: Run any scenario twice with $n_{points}=200$ and $n_{points}=1000$. Key metrics (SNR, NEDT) should agree within 0.5%. This validates numerical integration convergence.

Regime consistency: For an extended scene (large target), verify the regime is "extended" and EE_box does not appear in the signal computation. For a point target ($A_{target} < 0.01$ m² at 10 km range), verify regime = "point" and signal scales with EE_box.

Provenance completeness: After a run, call `ChainResult.to_provenance_record()` and verify (this mirrors what `tests/test_provenance.py` asserts):

- "radiant_version" key is present and non-empty
- "run_id" is a UUID4 string for a run that went through `ChainRunner.run` (None for a synthetic state)
- "parameter_set" has one entry per resolved parameter, each carrying value + unit + per-parameter provenance
- "input_file_hashes" is a list of {"path", "sha256"} records for every config file consumed

The exact key set is `run_id`, `radiant_version`, `git_commit`, `python_version`, `dependency_versions`, `parameter_set`, `input_file_hashes`, `active_models` — see §7.1.

13.5 Golden Regression Tests (@pytest.mark.golden)

Golden results are frozen JSON files representing the output of a full chain evaluation for a reference scenario. They are not updated automatically — updates require an intentional command (`scripts/update_golden.py`, §5.3). Today there is exactly one golden file: `tests/integration/golden/mwir_leo_minimal.json`, driven by `examples/mwir_leo_minimal.yaml`.

13.5.1 Golden File Structure

Each value in the real golden file is a {value, unit, provenance} object (not a bare number), and a top-level `_provenance` block records how the file was generated. The current shape (read `tests/integration/golden/mwir_leo_minimal.json`):

```
{
  "_provenance": {
    "config": "examples/mwir_leo_minimal.yaml",
    "wavelength_grid": "np.linspace(3.5, 5.0, 500)",
    "chain_stages": [
      "source", "atmosphere", "optics",
      "spectral_integration", "detector", "readout", "performance"
    ],
    "generated_by": "scripts/update_golden.py",
    "notes": "CU-155 refresh (2026-07-18). ... human-readable history of every value change ...",
    "last_updated": "2026-07-20T12:52:23.144171+00:00"
  },
  "signal_e": {"value": 956457.31579691, "unit": "e-", "provenance": "chain output, T3Mixed routing"},
  "e_rate_per_s": {"value": 191291463.159382, "unit": "e-/s", "provenance": "signal_e / t_int"},
  "A_collect": {"value": 0.07068583470577035, "unit": "m^2", "provenance": "pi/4 * 0.30^2"},
  "Omega_pixel": {"value": 2.25e-10, "unit": "sr", "provenance": "(18e-6)^2 / 1.20^2"},
  "noise_shot": {"value": 977.9863576742315, "unit": "e- RMS", "provenance": "sqrt(signal_e)"},
  "noise_dark_shot": {"value": 0.7071067811865476, "unit": "e- RMS",
```

```

        "provenance": "sqrt(dark_rate * t_int)"),
"noise_read":      {"value": 5.0,          "unit": "e- RMS",
                    "provenance": "readout.read_noise_e_rms"},
"noise_quantization": {"value": 9.237604307034013, "unit": "e- RMS",
                       "provenance": "gain / sqrt(12)"},
"noise_rss":       {"value": 978.0430200815521, "unit": "e- RMS",
                    "provenance": "RSS of the noise terms"},
"snr":             {"value": 977.9296985496178, "unit": "dimensionless",
                    "provenance": "signal_e_final / sigma_total_e"}
}

```

There is no `golden_version`, `config_hash`, `frozen_at`, `metrics`, `noise_budget`, or `signal` block — those were an earlier design that never shipped. The frozen fields are the scalar chain outputs (`signal_e`, `e_rate_per_s`, `A_collect`, `Omega_pixel`), the individual noise terms, their RSS, and `snr`.

13.5.2 Golden Test Protocol

There is no `radiant_golden` fixture and no `assert_within_tolerance()` helper. The golden test lives with the integration suite under `tests/integration/`, loads the JSON directly, and asserts each recorded value against a freshly-run chain with an explicit `pytest.approx` tolerance (§6). Shape:

```

@pytest.mark.golden
def test_golden_mwir_leo_minimal():
    """Full chain against the frozen golden result for the MWIR LEO minimal scenario."""
    golden = json.loads(
        (GOLDEN_DIR / "mwir_leo_minimal.json").read_text(encoding="utf-8")
    )
    # ... build params from examples/mwir_leo_minimal.yaml, run RadiantSession ...
    assert snr == pytest.approx(golden["snr"]["value"], rel=1e-6)

```

13.5.3 Intentional Golden Updates

When a physics improvement legitimately changes golden results (e.g. improved MODTRAN reader, a corrected routing rule such as CU-007/CU-155), the golden file is regenerated with the dedicated script — there is no `radiant freeze-golden` CLI:

```
python scripts/update_golden.py --i-know-what-im-doing --reason "CU-NNN: why"
```

The `--i-know-what-im-doing` flag and a `--reason` are both mandatory: without either the script prints an error and exits non-zero; the reason is appended to `_provenance.notes` with the update date. The script re-runs the MWIR LEO minimal chain, logs every value that changed (old → new with relative change), preserves the human-readable `_provenance.notes` history, and rewrites `_provenance.last_updated`. A golden update requires:

1. A PR describing **why** the physics changed (the reason is also appended to `_provenance.notes`).
2. Domain-expert review of the before/after values the script logs.

Golden files are committed to the repository. They are not `.gitignored`.

There are two golden families, with two regeneration scripts. The above covers the MWIR LEO minimal golden under `tests/integration/golden/`. The **scenario GUI baselines** — `scenarios/<persona>/`

<slug>/inputs/<slug>.gui.expected.json, gated by tests/integration/test_gui_baselines.py (CU-179) — regenerate with a different command:

```
python scenarios/tools/emit_gui_yaml.py          # all scenarios
python scenarios/tools/emit_gui_yaml.py 10.1     # one, by id
```

Four things about it are load-bearing:

- **Check the exit code — the emitter exits 1 if any scenario failed** (CU-319). It used to print [FAIL] <id> per broken scenario and still return 0, so a batch regeneration looked successful while leaving those baselines stale. `test_gui_baseline_is_rebuildable_from_its_runner` is the pytest-side guard: it builds each registry entry's Sensor straight from its runner, which the reload tests cannot do (they read the committed YAML, so they stay green while the path that *produces* it is broken — the failure mode CU-319 recorded).
- **Pass an id when only one scenario legitimately moved.** Regenerating all 38 hides which ones the change actually touched, which is the review signal.
- **The tools resolve their own tree, and refuse if they cannot** (CU-338). The emitter imports `radiant`, and the editable install's `.pth` pins that to whichever checkout ran `pip install -e .` — normally the primary tree. Run bare from a git worktree it therefore used to compose the *primary* tree's physics into this tree's baselines and print [ok]; that corrupted 15 baselines twice on 2026-08-30/31. Every scenarios/tools entry point now calls `scenarios/tools/_local_radiant.ensure_local_radiant()` before importing the registry, which prepends <repo root>/src and then raises `ForeignRadiantError` if `radiant.__file__` still lands outside the invoking tree. **No PYTHONPATH prefix is needed for these tools.** (pytest was always immune: `pythonpath = ["src", "."]` in `pyproject.toml` is rootdir-relative. `ruff` and `mypy` take explicit paths. `lint-imports` is **not** immune — it resolves `radiant` by import, so it still needs `PYTHONPATH=./src` in a worktree.)
- **The emitter repoints generated inputs at their committed counterparts** and raises `UnreloadableBaselineError` if none exists (CU-273). A baseline that references the scenario's gitignored `outputs/` tree reloads only in the tree that just ran the scenario; `test_gui_baseline_references_only_committed_files` is the static guard against it.

Three further sweep-protocol rules (October sweep, owner-ratified 2026-09-12):

- **Enumerate moved scenes from the RUNNERS, never from the GUI baselines.** Nine scenario folders have no baseline (1.6, 2.4, 4.2, 6.5, 8.3, 9.1–9.4), so a baseline-driven sweep cannot see them — exactly how CU-321's otherwise thorough refresh missed 1.6, the one moved scene without one. The CU-324 D-swap sweep's all-43-runners enumeration is the model.
- **A results-affecting sweep runs on the COMPOSED tree, after the last sibling merges.** A walk-through refreshed on a branch is measured against a tree without its siblings, so its digits go stale at the $\sim 1e-4$ level when they land, with every branch-side attribution individually correct and their composition not (measured on 1.3/3.2/10.2 after the CU-335 / CU-324-item-2 merges). One composed-tree pass replaces per-branch refreshes.
- **Scenario figures are committed, and the sweep regenerates the moved scenarios' figures on the machine running the sweep, in the same PR.** The runners are deterministic (double-run pixel-verified 2026-08-31): on a single machine a figure's bytes change only when its numbers did, so figure diffs are meaningful. A churn-only diff (bytes moved, stdout identical) signals environment drift and is worth investigating, not reverting. Rule 26(b) applies — walkthroughs reference the figures.

13.6 Numerical Tolerances

13.6.1 Per-Test Tolerances

Test type	Tolerance	Rationale
Physical constants	exact ($\leq 1e-12$ relative)	CODATA 2018 values are exact by definition
Pure physics formulas (Planck, sinc, Beer-Lambert)	$< 0.01\%$ relative	Floating-point rounding only
Spectral integrals ($\int L d\lambda$)	$< 0.1\%$ relative	Numerical quadrature error on 500-point grid
Full-chain SNR, NEDT	$< 1\%$ relative	Accumulated numerical errors through chain
NIIRS	± 0.05	GIQE5 is already empirical ± 0.3
MTF at specific frequency	$< 0.5\%$ relative	Interpolation + numerical PSF error
Noise terms individually	$< 1\%$ relative	Acceptable numerical precision
Golden regression (SNR, NEDT)	$< 0.1\%$ relative	Detect unexpected changes
Golden regression (NIIRS)	± 0.02	Detect rounding changes

13.6.2 When Tests May Legitimately Fail

A physics improvement that changes results beyond tolerance is **not a test failure** — it is a golden update. The protocol in §5.3 governs.

Scenarios that require intentional golden updates:

- MODTRAN reader bug fix (incorrect Jacobian)
- Improved IPC MTF model (from sinc-based to actual kernel)
- Marechal approximation replaced with full OTF integral
- Dependency version bump (numpy ≥ 2.0 changed default float precision in `trapz`)

Tests that fail for non-physics reasons (platform, random seed, file path):

- Are bugs to fix, not golden updates
- Must never be silenced with `pytest.mark.skip` without a linked issue

13.7 Provenance Tracking

Every RADIANT result carries a provenance record so a run's inputs and environment are recoverable. The record is produced by `ChainResult.to_provenance_record()` (`radiant/io/results.py`) and implements the contract in `RADIANT_Master_Architecture.md` §C13; `tests/test_provenance.py` pins the exact key set.

13.7.1 What the Provenance Record Contains

The real record has exactly these eight keys (copy the shape from `to_provenance_record()`):

```

{
  "run_id": "a4f7c2e1-8b3d-4e9a-a0b1-2c3d4e5f6789",
  "radiant_version": "0.1.0",
  "git_commit": "abc1234",
  "python_version": "3.11.9",
  "dependency_versions": {
    "numpy": "1.26.4",
    "scipy": "1.13.0",
    "pyyaml": "6.0.1",
    "click": "8.1.7"
  },
  "parameter_set": {
    "optics.aperture_diameter_m": {
      "value": 0.30,
      "unit": "m",
      "provenance": "config_file"
    },
    "optics.focal_length_m": {
      "value": 1.20,
      "unit": "m",
      "provenance": "config_file"
    }
  },
  "input_file_hashes": [
    {"path": "examples/mwir_leo_minimal.yaml", "sha256": "1a2b3c4d..."}
  ],
  "active_models": [
    "source", "atmosphere", "optics",
    "spectral_integration", "detector", "readout", "performance"
  ]
}

```

Field notes (per the `to_provenance_record` docstring):

- `run_id` — UUID4 assigned at chain-runtime (`radiant.core.provenance.new_run_id`); None if the result came from a synthetic state that never went through `ChainRunner.run`.
- `git_commit` — short SHA of the working-tree HEAD; "unknown" outside a git repo.
- `dependency_versions` — {name: version} for the declared runtime deps (numpy, scipy, pyyaml, click).
- `parameter_set` — {dotpath: ResolvedValue.to_dict()} for every resolved parameter (value + unit + per-parameter provenance); empty if no `ParameterSet` was attached.
- `input_file_hashes` — an ordered **list** of {"path", "sha256"} records for every config file consumed via `radiant.io.config.load_config`.
- `active_models` — an ordered **list of stage names** that ran (mirrors `ChainResult.history`), not a model-id dict.

There is no `git_tag`, `resolved_at`, `config_hash`, or `dependencies` key, and `parameters/active_models` are not the nested-dict shapes an earlier draft described.

13.7.2 Run ID

Every chain run generates a UUID4 run ID (`radiant.core.provenance.new_run_id`), surfaced as the `run_id` provenance field and preserved across `ChainResult.save()/load()`. It is **not** logged to the console today (`[RADIANT] Run ... started` is not implemented) — it is a provenance field only.

13.7.3 Reproducibility

Reproducibility today rests on determinism, not a dedicated tool:

- **Same inputs → same outputs.** The chain has no random component; re-running the same config on the same RADIANT version and inputs reproduces the result bit-for-bit (this is what the golden regression test asserts).
- To re-run a recorded scenario, load the same config (`examples/*.yaml`) through `RadiantSession/Sensor` and execute the chain again.
- The provenance record captures what an external observer needs to confirm the environment: `radiant_version`, `git_commit`, `dependency_versions`, and `input_file_hashes` (so a changed input file is detectable by hash comparison).

[DESIGN-TARGET] A `config_hash` (SHA256 of the fully-resolved parameter set) and a one-shot `radiant reproduce provenance.json` command that reconstructs the parameter set, re-verifies input hashes, and re-runs the chain are **not implemented** — no such CLI subcommand or hash field exists in the code. They remain a reasonable future target but must not be documented as shipped.

13.8 Error Handling Philosophy

13.8.1 Every Error Must Answer Three Questions

1. **What:** What went wrong, stated precisely. Not “parameter error” — `optics.aperture_diameter_m = -0.30` is invalid.
2. **Why:** Why it’s wrong. “Aperture diameter must be positive (it is a physical length).”
3. **What to do:** What the user should change. “Set `optics.aperture_diameter_m` to a positive value in meters.”

The base class `RadiantError` lives in `radiant.core.exceptions` (re-exported as `radiant.RadiantError`). It is currently a plain `Exception` subclass — the structured what / why / action / context payload is carried by `ParameterBoundsError`, the most user-facing subclass. The other concrete subclasses (`KirchhoffViolationError`, `ModtranUnavailableError`, `Tape7ParseError`, `ConfigError`, `ElementConfigError`) carry the same information in their message strings until the carve-out is generalized.

```
# radiant/core/exceptions.py
class RadiantError(Exception):
    """Base class for all RADIANT-defined exceptions."""

# radiant/core/parameters.py – the canonical structured form
class ParameterBoundsError(RadiantError, ValueError):
    def __init__(
```

```

    self,
    what: str,
    why: str = "",
    action: str = "",
    context: dict[str, Any] | None = None,
) -> None: ...

```

Concrete subclasses MAY co-inherit from a built-in exception (`ValueError`, `RuntimeError`) for back-compat with except `ValueError` / `pytest.raises(ValueError, ...)` patterns; new RADIANT exception classes SHOULD inherit from `RadiantError` only.

13.8.2 No Silent Failures

These behaviors are never acceptable:

- Returning NaN or inf without raising an exception
- Clipping a value to bounds without warning
- Defaulting to a model when the specified model fails to load
- Catching an exception and returning a placeholder value
- Logging a warning and continuing when the physics is undefined

The rule: If RADIANT cannot compute a physically meaningful result, it raises an exception. The user gets an error with context, not a wrong answer.

13.8.3 Validate Before Compute

The validation pipeline (type → bounds → enum → required → consistency → file) runs before any physics computation begins. A config that fails validation never reaches the chain runner. The user sees all validation errors in one pass (not fail-fast per error — collect all, report all).

Validation is cheap. Physics is expensive. Never skip validation to save time.

13.8.4 Progressive Disclosure

Error messages have two levels:

Summary (always shown):

```

ConfigError in configs/leo_mwir_clear.yaml: detector.detector_temperature_K = 400 K
(out of bounds: 1–300 K)

```

(The config loader raises `radiant.io.config.ConfigError`; a bad parameter value raises `ParameterBoundsError`. There is no `ConfigValidationError` class.)

Detail (shown with `-verbose` or when requested):

```

[1] ParameterBoundsError
  Parameter: detector.detector_temperature_K
  Value: 400 K
  Source: sensors/baseline_mwir.yaml, line 18
  Bounds: 1 K ≤ detector_temperature_K ≤ 300 K
  Why: HgCdTe and InSb detectors operate at cryogenic temperatures.
      300 K is the upper bound because above this, dark current is

```

astronomically high (Rule 07 activation).
 Fix: Typical operating temperatures: HgCdTe 77–120 K, InSb 77 K,
 InGaAs 220–300 K, Si CMOS 230–300 K.
 For uncooled microbolometer, use material=V0x and remove this parameter.

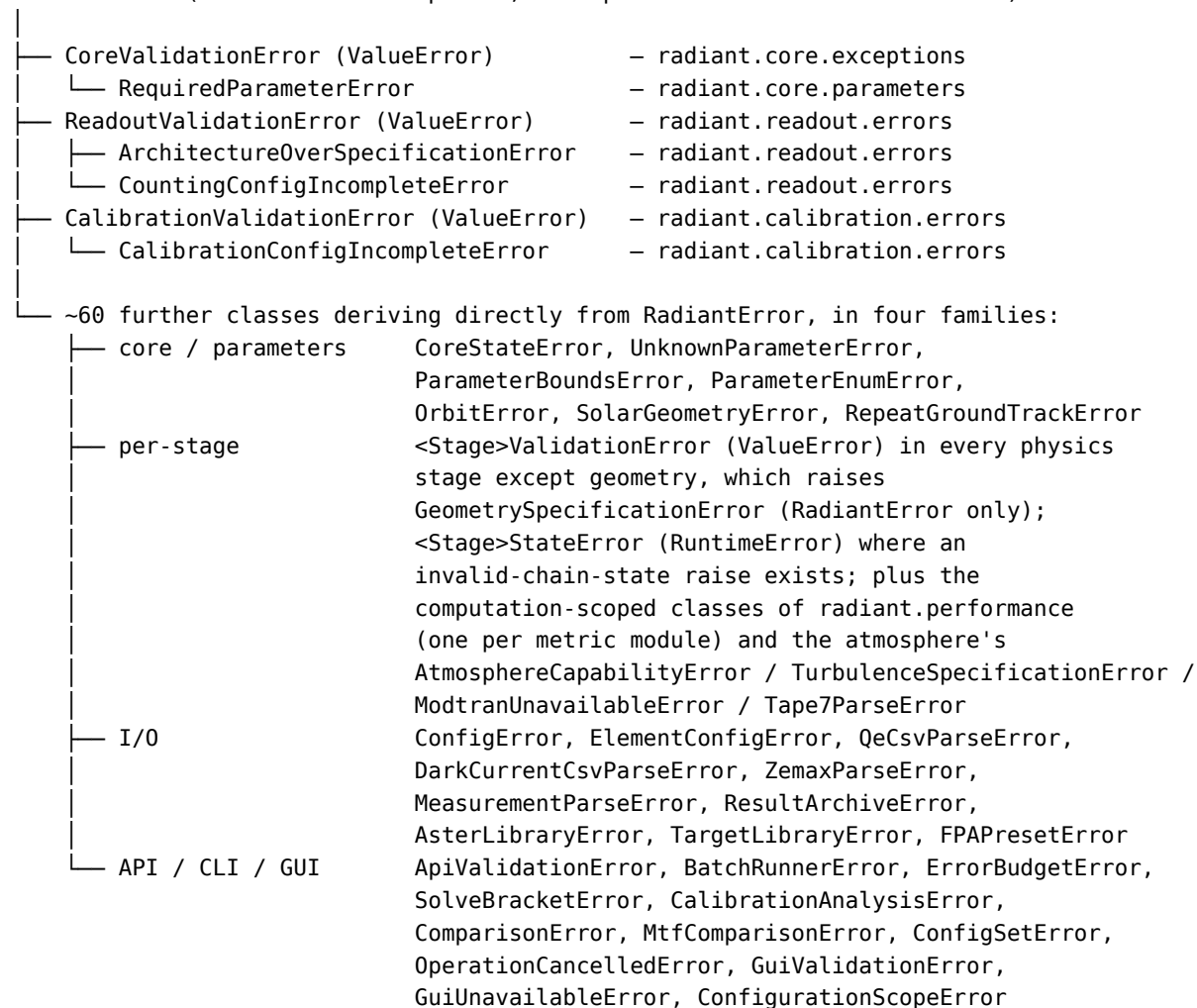
[DESIGN-TARGET] The two-level summary/detail rendering above is the intended error-presentation model. Today, RADIANT exceptions already carry the actionable what / why / action payload (via `ParameterBoundsError` and its structured context) and message strings, but there is no `sensor.validate(verbose=True)` API and no `--verbose` error-detail switch — those remain a design target, not a shipped surface.

13.8.5 Exception Hierarchy

Every framework-defined error derives from `RadiantError`, and most concrete classes co-inherit the built-in exception they historically raised as — `ValueError`, `RuntimeError`, `KeyError`, `NotImplementedError` — per the Rule 15 / CU-043 back-compat carve-out. New classes inherit from `RadiantError` only.

The hierarchy is **two-tier**, not flat: four classes specialize another RADIANT error rather than deriving directly from `RadiantError`.

`RadiantError` (`radiant.core.exceptions`; re-exported as `radiant.RadiantError`)



The **full enumeration** — every class, its bases, its module, and an example message — is the error-taxonomy chapter of the Technical Reference, which is generated from the code. It is not duplicated here; this section describes the *shape* of the hierarchy, which is what a test author needs. `tests/test_exceptions.py` pins the set.

`RadiantError` itself is importable from `radiant` (top-level re-export) and from `radiant.core.exceptions`. Each concrete subclass is importable from the module that raises it. Catching `RadiantError` catches every framework-defined error while letting unrelated bugs (`KeyError`, `AttributeError` from a buggy stage) propagate.

13.9 Test Infrastructure

13.9.1 Test Layout and Fixtures

Tests live in three places (there is no top-level `tests/conftest.py`, no `tests/fixtures/` tree, and no `tests/golden/` directory):

```
src/radiant/<stage>/tests/      # module-local Level 0 / Level 1 tests + fixtures,
                                #   co-located with the stage they exercise
tests/                          # cross-cutting tests: exceptions, provenance,
                                #   public-API surface (test_exceptions.py,
                                #   test_provenance.py, ...)
tests/integration/             # Level 2 full-chain tests
tests/integration/golden/      # frozen golden data
    └─ mwir_leo_minimal.json   #   the single current golden file
```

Fixtures are defined locally in the test modules / package-level `conftest.py` files that need them (e.g. a stage's own `src/radiant/<stage>/tests/conftest.py`), not in one session-scoped root `conftest`. This mirrors the “How to Find Things” table in `CLAUDE.md`.

13.9.2 Property-Based Testing

[**DESIGN-TARGET**] hypothesis is declared as a dev dependency in `pyproject.toml`, but it has **zero imports** anywhere in `src/` or `tests/` — no property-based tests exist today. Physics invariants (Planck positivity, $\sigma_{\text{shot}} = \sqrt{n} \geq 0$, $\text{MTF} \in [0, 1]$, etc.) are currently exercised by parametrized `level0` tests, not by generated inputs. Adopting Hypothesis for these invariants is a reasonable future target; until a test actually imports it, this section describes an aspiration, not shipped coverage. (Tracked as a cleanup item — the unused dependency should either be used or dropped.)

13.9.3 CI Configuration

The only CI workflow is `.github/workflows/ci.yml`. It runs on push/PR to main (and `workflow_dispatch`), on a single interpreter (Python 3.11, `ubuntu-latest`) — there is no Python 3.11 × 3.12 or `ubuntu` × `macos` matrix, and no `--cov` / `--cov-fail-under` gate. The jobs are:

Job	What it runs
static	<code>ruff check src/; ruff format --check src/; mypy --strict src/radiant/core src/radiant/api; lint-imports --config pyproject.toml; python scripts/check_org_rules.py; python scripts/check_physics_conversions.py; python scripts/check_approx_tolerances.py</code>
fast-tests	<code>pytest -m level0 --strict-markers -q</code> then <code>pytest -m level1 --strict-markers -q</code>
integration-tests	<code>pytest -m level2 --strict-markers -q</code> (needs fast-tests)
gui-tests	Geometry GUI v2 suite under xvfb-run (needs fast-tests; installs Qt/VTK system libs + dev_tools/geometry_gui_v2)
golden	<code>pytest -m golden --strict-markers -q — main only</code> ('if: github.event_name == 'push')

The golden job being main-only is what §1 refers to when it says golden tests run in a separate job on the main branch rather than on every PR.

Extending RADIANT

This chapter is the practical walkthrough the three preceding specifications support: how to add a stage, how to add a parameter, what the import rules permit, and where the plugin system stands.

Everything here assumes the constraints of Part 1 and Part 3. An extension that violates one of them is not an extension — it is a fork in disguise.

14.1 Adding a parameter

Every user-facing quantity is defined exactly once, as a `ParameterDef` in the owning stage's `_schema.py`. Nothing tuneable is hardcoded in a physics module; a magic number in a physics file is a bug, not a shortcut.

14.1.1 The steps

1. **Pick the owning stage.** The namespace is the stage that consumes the parameter — `optics.*` lives in `src/radiant/optics/_schema.py`, `detector.*` in `src/radiant/detector/_schema.py`. Do not put a parameter in a stage that merely forwards it.
2. **Write the `ParameterDef`** and add it to that module's `ALL_PARAMETERS` tuple. The central registry (`radiant.api._param_registry.build_parameter_set`) collects every stage's tuple, so there is nothing else to register.
3. **Read it in the stage** via `params.get("namespace.name")` — never from a raw dict, never from a module-level constant.
4. **Write the Level 0 test first**, against a known-good analytic value, before the implementation exists.
5. **Regenerate the parameter reference** (`python scripts/gen_param_reference.py`) — the generated chapter of this volume is freshness-gated, so a new parameter with a stale reference fails the check.
6. **Update the matching architecture document** in the same change. The parameter schema is a documented surface.

Note that the schema surface reaches further than it looks: the GUI builds its editors from `Sensor.parameter_defs()`, so adding one parameter can move a GUI test even though no GUI file was touched.

14.1.2 The fields

```
from radiant.core.parameters import ParameterDef
```

```
APERTURE_DIAMETER_M = ParameterDef(  
    name="optics.aperture_diameter_m",  
    description="Clear entrance-pupil diameter of the primary [m].",
```

```

dtype=float,
canonical_unit="m",
input_unit="m",
default=None,           # None means required
bounds=(1e-4, 20.0),    # in input_unit
group="fnumber",        # consistency group membership
tags=frozenset({"optics", "aperture"}),
)

```

Field	Meaning
name	Dot-path, namespace.parameter_name, lowercase with underscores
description	One sentence; it is what the Parameter Reference and the GUI tooltip print
dtype	float, int, str, or bool — nothing else
canonical_unit	The internal storage unit (μm, rad, s, m, ...)
input_unit	The unit the user enters and the YAML carries
default	In input_unit. None means required
bounds	(lo, hi) in input_unit; numeric dtypes only
enum_values	Closed set of legal strings; str dtype only
group	Consistency-group name, if the parameter is derivable from others
tags	Free-form labels for filtering and grouping
default_justification	Why this default and not another — required discipline for a non-obvious default
deprecated_aliases	Former names; they resolve with a DeprecationWarning
required_unless	Dot-path of a parameter that supersedes this one, letting a required parameter stay unset (e.g. detector.qe_value is required <i>unless</i> detector.qe_table_path is set)
is_file_path	The value names a data file; serialization stores it relative to the YAML's directory so configs stay portable. Not for system paths such as a MODTRAN binary

ParameterDef validates itself on construction: a non-scalar dtype, enum_values on a non-string, bounds on a non-numeric, a self-alias, or required_unless on a parameter that has a default all raise immediately.

14.1.3 Naming

Lowercase with underscores, no units *inside* the name but a unit **suffix** where one applies (_m, _um, _rad, _s, _K, _e_rms). The suffix names the canonical stored unit, which is not always the entry unit — geometry.solar_zenith_rad is entered in degrees through set(..., unit="deg").

14.1.4 Consistency groups

When a new parameter is derivable from existing ones, declare the relation instead of recomputing it at the call site. A group names its members, its constraint, a derivation per member, and a tolerance:

```

from radiant.core.parameters import ConsistencyGroup

_FNUMBER_GROUP = ConsistencyGroup(
    name="fnumber",
    parameters=("optics.aperture_diameter_m", "optics.focal_length_m", "optics.f_number"),
    constraint="f_number = focal_length_m / aperture_diameter_m",
    derivations={
        "optics.f_number": lambda kv: (

```

```

        kv["optics.focal_length_m"] / kv["optics.aperture_diameter_m"]
    ),
    # ... one entry per member
},
tolerance=1e-3,
)

```

Supply any two members and the third is derived with provenance DERIVED and a `derived_from` record. Supply all three inconsistently and resolution raises rather than silently preferring one. Groups are registered in `radiant.api._param_registry.build_parameter_set`.

The `ground_speed` group is worth studying as the other archetype: it is an *identity* group linking `platform.ground_velocity_m_s` and `geometry.ground_speed_m_s`, two names for one physical quantity. Setting either derives the other, so smear and access rate can never read two different velocities.

14.2 Adding a stage

A new stage is a substantial change — it extends the chain protocol — but the mechanics are small.

1. **Create `src/radiant/<stage_name>/.`**
2. **Create `_schema.py`** with every `ParameterDef` the stage owns, collected in `ALL_PARAMETERS`, and add it to `radiant.api._param_registry`.
3. **Create `stage.py`** implementing the Stage protocol:

```

from radiant.core.chain import ChainState
from radiant.core.parameters import ParameterSet

class MyStage:
    """One-line description of what this stage computes."""

    @property
    def name(self) -> str:
        return "my_stage"

    def run(self, state: ChainState, params: ParameterSet) -> ChainState:
        value = params.get("my_stage.some_parameter")
        ...
        return state.with_stage_output("my_stage", "result", value)

```

`name` is a read-only property, and `run` returns a **new** state — never the one it was handed.

4. **Create `tests/`** with Level 0 tests for the key physics equations, written against known-good analytic values — not against values another part of RADIANT computed.
5. **Register the stage** in the `ChainRunner` list in `radiant/api/session.py`, at the position the physics requires.

6. **Add the stage to the document map** in `RADIANT_Master_Architecture.md` and write its subsystem document.

14.2.1 What the stage may and may not do

Allowed	Forbidden
Read parameters via <code>params.get(...)</code>	Mutating state or params
Read earlier stages' <code>state.stage_outputs[...]</code>	Importing another physics stage
Return <code>state.with_frame / with_noise / with_mtf / with_stage_output</code>	Reading or writing any file
Raise an actionable <code>RadiantError</code> subclass	Calling another stage's <code>run</code>
Log through the logging module	<code>print()</code>
—	Returning <code>NaN / inf</code> silently, or holding state between runs

A stage needing a file-derived object does not open the file: the API layer builds the object before the chain and injects it through `ChainRunner.run(initial_stage_outputs=...)`. `atmosphere_config.model` and `optics_config.element_list` are the shipped examples, and `Sensor.set_stage_output(group, key, value)` is the user-facing door to the same mechanism.

14.2.2 If the stage has a spatial effect

Both spatial paths must move together. A new degradation needs a convolution kernel on the `EffectivePSF` **and** a corresponding term in the MTF product — or an explicit, documented reason why it is one of the deliberate MTF-only terms, like TDI mis-registration. Adding it to only one path is caught by the consistency check on every run, which is exactly what that check exists for.

14.3 Import rules

Module boundaries are machine-enforced by `import-linter`, and a violating change is blocked before it merges. The contracts:

Module	May import from
<code>core/</code>	<code>stdlib</code> , <code>numpy</code> , <code>scipy</code> only — no <code>radiant.*</code> imports at all
Physics stages (geometry ... performance)	<code>radiant.core</code> only
<code>data/</code>	<code>radiant.core</code> only, plus <code>stdlib</code> , <code>numpy</code> , <code>yaml</code>
<code>io/</code>	<code>radiant.core</code> plus any physics stage (read-only, for schema)
<code>api/</code>	<code>radiant.core</code> , all physics stages, <code>radiant.io</code> , <code>radiant.data</code>
<code>cli/</code>	<code>radiant.api</code> , <code>radiant.io</code> , and <code>radiant.gui</code> (lazily, for <code>radiant gui</code>)
<code>gui/</code>	<code>radiant.api</code> and <code>radiant.core</code> only — no physics stage, no <code>io</code> , no <code>cli</code>

```
# FORBIDDEN inside a physics stage:
from radiant.optics import psf          # cross-stage physics import
```

```
from radiant.source import blackbody # cross-stage
```

```
# ALLOWED:
```

```
from radiant.core.chain import ChainState
from radiant.core.parameters import ParameterSet
from radiant.core.spectral import SpectralData
```

Two consequences worth internalizing:

- **Stages communicate only through ChainState.** If a stage seems to need another stage’s function, the function belongs in core or the value belongs in stage_outputs.
- **The GUI is a view over the scripting API,** one action to one API call. When the GUI needs something a physics stage knows, the bridge is an api/ module — the pattern radiant.api.geometry_modes and radiant.api.metric_groups follow.

Verify locally with:

```
lint-imports --config pyproject.toml
```

14.4 Adding a metric

Each distinct metric gets its own module under radiant/performance/, with its own error class and its own Level 0 test. Bundling unrelated computations into one file because they share a stage is the thing this rule exists to prevent — a developer should find a calculation by scanning file names.

Bundling is acceptable only for computations that share internal state or helpers and would be meaningless apart (a cavity’s T_{sys} and ϵ_{eff} are one model, not two files).

A metric must also register its unit and description, so that metric_records() can return it with units; a metric key with no registry entry raises rather than reporting a bare number.

The metric layer carries one deliberate exception to the no-silent-failure rule: a computation under radiant.performance/ may return a result-typed failure with a named failure_reason field instead of raising, because “this metric is undefined for this scene” is a reportable outcome rather than a rejected input. The failure must appear in the result object callers already inspect. Silent NaN propagation remains forbidden, and physics-layer modules keep the universal raise rule.

14.5 Testing an extension

The test hierarchy is specified in full in the preceding chapter. In short:

- **Level 0** — the physics equation against a known-good analytic or literature value. Write it *before* the implementation. Never compare against a number another part of RADIANT produced.
- **Level 1** — module behavior, including the failure modes: zero, negative, very large, very small, wrong type, conflicting inputs.
- **Level 2** — full-chain integration and golden results.

`pytest.approx` always carries an explicit `rel=` or `abs=` tolerance; the default is never acceptable. Loosening a tolerance to make a failing test pass is a change to the test, not a fix to the physics — golden results have their own review protocol precisely so that baseline changes are deliberate and explained.

14.6 Plugins — deferred

`docs/architecture/RADIANT_Plugins.md` specifies a formal extension-point system: abstract base classes for source, atmosphere, metric, detector, and file-format plugins, discovered at run time through `pyproject.toml` entry points, so that an organization with a proprietary atmosphere model or a custom metric can extend RADIANT without forking it.

That design is deferred and not implemented. The `src/radiant/plugins/` package does not exist — an empty stub was removed rather than left to mislead — and none of the ABCs or registration machinery described in that document is importable. The specification is preserved as the v2 design, not as a description of current behavior. Do not write code against those symbols.

Until it lands, the supported extension routes are the ones in this chapter: a new stage, a new parameter, a new metric module — contributed into the tree, under the same rules as any other change.